Programming in ABCL/R2

A Guided Introduction to Objects,
Concurrency, and Reflection

For the September 4, 1998 distribution

October 20, 2026

Abstract

This book takes you from your first interactive session to concurrent, reflective objects in ABCL/R2. Each major concept is introduced through a small example, with exercises to help you apply what you have learned. You need experience with object-oriented programming and some Lisp, but not Common Lisp, actors, or reflection. The goal is to read and explain the original programs in the accompanying distribution, not just run the additional teaching examples.

How to use this book. Read the opening chapters in order, including the Lisp reading guide. Then work through futures, reflection, and the original-source reading projects. The original examples form the thread connecting concepts to complete programs. They live in `examples/`; experiment on copies, not the reference sources. Executable programs live in `tutorial/examples/`. The later sections contain worked exercises and a complete job-service solution. Start from the repository root unless a command says otherwise.

Contents

1 What Is ABCL/R2? 3
- 1.1 From Method Calls to Message Protocols . . . 3
- 1.2 An Actor-Like Mental Model . . . 3
- 1.3 Three Questions to Ask While Reading . . . 4

2 Installation and Your First Session 4
- 2.1 A Lisp Reading Guide . . . 5
- 2.2 The Boundary with Common Lisp . . . 8

3 Objects, Scripts, and Messages 9
- 3.1 A Small Useful Object . . . 9
- 3.2 Replying with `!` . . . 9
- 3.3 Message Patterns . . . 9
- 3.4 Parentheses, Brackets, and Quoting . . . 10
- 3.5 Matching Data Inside a Script . . . 10
- 3.6 Temporary Variables and Loops . . . 11

4 State and Assignment 11
- 4.1 Lexical Variables . . . 11

5 Two Kinds of Messages 12
- 5.1 Explicit Reply Destinations and Continuations . . . 12
- 5.2 Forwarding a Reply Without Waiting . . . 12
- 5.3 Object Identity and the Current Object . . . 13
- 5.4 Concurrency and Parallel Hardware . . . 13

6 A Complete Example: A Bank Account 13

7 Concurrency as a Programming Model 13

8 Creating and Organizing Objects 14

9 Files, Libraries, and Tools **15**
9.1 Options and File Selection . . . 15
9.2 Execution Context and Module Loading . . . 16

10 Futures and Express Messages **18**
10.1 A Future Is a Reply Stream . . . 18
10.2 Peeking, Consuming, and Readiness . . . 19
10.3 Express Messages . . . 20
10.4 An Express Exchange . . . 20

11 Reflection and Hybrid Group Architecture **20**
11.1 What Reflection Adds . . . 20
11.2 Two Reflective Towers, Two Kinds of Control . . . 21
11.3 Inspecting an Object's State . . . 22
11.4 Defining a Lazy Control Form . . . 22
11.5 Extension Parameters and Host Events . . . 24
11.6 A Safe Experiment with Synthetic Host Events . . . 24

12 Troubleshooting and Good Practice **27**

13 Working from the Original Examples **27**
13.1 Tree Leaves: The Original Future Protocol . . . 28
13.2 Lazy Evaluation: The Original IF with EF . . . 30
13.3 Interrupting a Reflective Control Form . . . 31
13.4 Inheritance Without a Superclass Declaration . . . 35
13.5 Reading the Built-in Protocol Sketches . . . 38
13.6 A Script Object with a Sender-sensitive Match . . . 39
13.7 MCP: Following a Meta-circular Interpreter . . . 40
13.8 Meta-meta: Reflecting on the Message Interpreter . . . 41
13.9 Guarded Scripts: The Original Heavy-Meta Counter . . . 42
13.10 Multicast: One Request, a Structured Result . . . 46
13.11 Object Copies Are Not Independent Computations . . . 49
13.12 Watch: Observe Sends without Replacing the Application . . . 53
13.13 Report: Observe Arrivals and Script Execution . . . 57
13.14 Retry: Repairing a Message Acceptance Condition . . . 61
13.15 Breakpoints: Inspecting a Suspended Lexical Environment . . . 64
13.16 The Original Express Clocks . . . 67
13.17 Pcontrol: Scheduling a Recursive Object Computation . . . 71
13.18 Time Warp: Virtual Time and Recovering from Late Events . . . 75
13.19 Navigating the Original Sources . . . 84

14 Final Project **84**
14.1 A Worked Job-Service Protocol . . . 85
14.2 Invariants and Limits . . . 86

15 Worked Exercise Solutions **87**
15.1 Exercise 1: Sum of Squares . . . 87
15.2 Exercise 2: An Account Factory and a Transfer . . . 87
15.3 Exercise 3: Count Only After Acknowledgment . . . 88
15.4 Exercise 4: A Tagged Stream . . . 89
15.5 Exercise 5: Unless at the Meta-Level . . . 90

16 Further Reading in the Source Tree **90**

A Building the Book and Checking Its Examples **91**

B Original Source Reading Map **91**

C Syntax at a Glance **93**

D Troubleshooting Program Protocols **95**

E Concept and Protocol Index **96**

1 What Is ABCL/R2?

ABCL/R2 is an object-oriented, concurrent, reflective programming language. A program consists primarily of objects that receive messages and execute scripts. Concurrency is part of the language: messages are placed in object queues, and computations in different objects can make progress independently.

The accompanying distribution (documented as version 0.60, September 1998) implements this model using partial compilation. The examples here use its SBCL implementation. We will unpack these implementation terms in the reflection chapter. Masuhara's 1992 thesis supplies the historical model [2].

Thinking in objects. Start from the familiar idea of an object with state and behavior:

1. It owns state.
2. It receives messages and matches them against patterns.
3. A matching script is selected for execution.
4. The script can compute, change state, create objects, and send further messages.

1.1 From Method Calls to Message Protocols

In a familiar OOP language, a call such as `account.balance()` selects a method, executes its body, and returns a value to the caller. ABCL/R2 separates these jobs. A sender constructs a data value, sends it to an object, and may arrange a destination for a later reply. The receiving object matches that data against its scripts. A *script* is a handler: a message pattern followed by expressions to execute when the pattern matches.

Familiar idea	ABCL/R2 counterpart and difference
Object fields	State bindings belonging to an object; outer lexical bindings can also be shared.
Method selection	First matching message pattern, not necessarily a method name plus a fixed argument signature.
Method arguments	Data inside the message; pattern variables extract the parts the handler needs.
Return value	An explicit reply message, not the handler's final value.
Inheritance	Can be implemented by a reflective protocol; the original bird/emu example does not declare a superclass.

A *protocol* specifies the messages an object accepts and what its clients may expect: replies, state changes, or further messages. For an account, `[:deposit 10]` and `:balance` can be two protocol messages. The colon is part of a Lisp keyword, not special method-call punctuation; the next chapter explains keywords.

1.2 An Actor-Like Mental Model

An *actor* is a useful introductory model of an independently addressable participant that reacts to messages and can send more messages. For an ordinary ABCL/R2 object, imagine an inbox and a handler processing one request at a time. Other objects have their own work. *Concurrent* means their activities can be outstanding together; it does not require simultaneous execution on different CPUs.

The distinction becomes concrete in this sequence:

1. A client sends a deposit request to an account.
2. The request enters the ordinary receiving object's message machinery.
3. The account selects a matching handler and updates its balance.
4. If the protocol requires it, the handler explicitly sends a reply.
5. The client uses that reply, or continues without waiting if it sent asynchronously.

Sending is therefore not the same event as finishing the requested work. An asynchronous sender does not automatically receive a completion notice. Conversely, a waiting sender cannot finish its request unless someone sends the required reply.

This is an actor-like explanation, not a claim that ABCL/R2 enforces pure actor isolation. Its lexical bindings can be shared between objects. *Reflection* means that a program can inspect or control parts of its own execution machinery. It can therefore change the ordinary selection and execution of handlers. Learn the ordinary model first; later chapters state where it is extended.

1.3 Three Questions to Ask While Reading

For every send in an original program, ask:

1. Which object receives this message, and which pattern matches it?
2. Does the sender wait, and where will any reply be sent?
3. What establishes completion: one reply, a count, or a terminal message?

In `examples/misc/future-ex.abclr2`, these questions distinguish the tree walker, the object consuming its replies, and the printer. In `examples/misc/inherit.abclr2`, they distinguish an application object from the metaobject running its dispatch protocol. You do not need to understand those files yet; these are our first original-source reading goals.

A route through the book. Work in stages rather than trying to absorb every library at once:

1. *Build the ordinary model.* Follow Sections 2–9, trying the small sessions and exercises. At the bank account (Section 6), explain which pattern handles each request and why the caller receives a reply.
2. *Follow results over time.* Study futures in Section 10.1, then trace the original tree walker in Section 13.1. Explain why its terminal message, not an empty buffer or a printed line, marks completion. Express messages are an advanced extension; you may return to that experiment after learning reflection.
3. *Open the execution machinery.* Read Section 11, then the original lazy-control, inheritance, and interpreter projects in Section 13. Keep the application reply separate from the interpreter's execution-completion message.
4. *Combine the mechanisms.* Work through the library and debugging projects, then Pcontrol and Time Warp (Sections 13.17 and 13.18). These are advanced reading projects, not prerequisites for ordinary object programming. Return to the file map in Appendix B when following their helper objects.

The job-service project (Section 14) is a parallel practice track: it combines ordinary objects and reply protocols, so you can try it after the foundations without first mastering the reflective projects. Use the worked exercise answers to check your reasoning, not just the final values. At each stage, being able to explain a message's recipient, reply destination, and completion condition is the checkpoint for moving on.

2 Installation and Your First Session

Build the interpreter and start it from the project directory:

```
make
./src/saved_abclr2
```

The prompt accepts ABCL/R2 expressions. Type `:q` to quit or `:reset` to restart the session. To make experiments reproducible, save your program files in a separate directory of your choice. For example, load a program named `demo.abclr2` from the session:

```
(inc "demo" :use-source t)
```

The Makefile uses SBCL and writes `src/saved_abclr2`. Running the saved image does not require installing it system-wide.

Omit the `.abclr2` extension when using `inc`. `:use-source t` forces translation from the ABCL/R2 source rather than reuse of an existing translation. Keep this option while trying the small sessions. Section 9 covers compilation, libraries, modules, and loading contexts once objects and messages are familiar.

2.1 A Lisp Reading Guide

This section teaches the Lisp notation used by the original examples. Try the small expressions at the ABCL/R2 prompt. Comments beginning with a semicolon explain the expected result; do not type the prompt itself. The guide is deliberately selective: it is not a Common Lisp course.

2.1.1 Expressions, Symbols, and Literal Data

An expression such as `(+ 2 3)` puts the operation first. For an ordinary function call, argument expressions are evaluated before the function is called. Numbers and strings are literal values. An unquoted ordinary symbol in an expression names a binding:

```
(+ 2 (* 3 4))                  ; => 14
(let ((amount 10)) (+ amount 2)) ; => 12
```

The inner multiplication finishes before the outer addition. `let` introduces a local binding; its body can refer to `amount`. A *binding* associates a name with a value. Its *lexical scope* is determined by the surrounding source structure: the body of this `let` is where its local name is available.

A symbol is not a string. `amount` is a name, `'amount` is that name as data, and `"amount"` is a string of characters. The quote character is shorthand for `quote`:

```
'amount                        ; => AMOUNT
'(a b c)                       ; => (A B C)
(list 'a 'b 'c)                ; => (A B C)
```

Quote prevents evaluation of its operand. Thus the symbols inside `'(a b c)` are not looked up as variables. Unescaped symbol names are normally printed in uppercase; `n` and `N` read as the same symbol in the same package.

A *keyword* begins with a colon and evaluates to itself. It is convenient as a protocol tag: `:deposit`, `:new`, and `:finish` are distinct symbols. No variable definition is needed to use them.

```
(list :deposit 10)             ; => (:DEPOSIT 10)
(let ((x 7)) [:point x (+ x 1)]) ; => (:POINT 7 8)
```

Here square brackets construct a list of evaluated elements. They also introduce ABCL/R2 operations such as object creation or sending; the recognized operator determines which interpretation applies. In a *pattern*, however, a symbol such as `x` binds a part of the incoming value. Construction and matching are different uses of the same readable list notation.

2.1.2 Truth, Tests, and Control Flow

`nil` means false and is also the empty list. `t` is true. Every value other than `nil`, including zero, counts as true.

```
(if nil :yes :no)             ; => :NO
(if 0 :yes :no)               ; => :YES
(null '())                    ; => T
(not t)                       ; => NIL
```

`if` is a *special form*: it does not eagerly evaluate both branches. It evaluates the test and only the selected branch. `progn` evaluates several expressions in sequence, with the final expression providing its value. `when` is a conditional sequence; `cond` chooses the first clause whose test succeeds. `and` stops at the first false operand; `or` stops at the first true operand. These are control operations, not ordinary eager calls.

Use `=` to compare numbers. Use `eq` for symbol or object identity, as in testing a terminal keyword or checking a self-transfer. Use `equal` for structural comparison of lists and strings. Do not assume that `eq` compares two independently constructed lists by their contents.

2.1.3 Lists, Cons Cells, and Dotted Tails

A *cons cell* has two parts: its `car` and its `cdr`. A proper list chains cons cells and ends in `nil`. `first` and `rest` are readable names for the head and tail of a list; `car` and `cdr` expose the underlying pair.

```
(cons 'a '(b c))              ; => (A B C)
(car '(a b c))                ; => A
(cdr '(a b c))                ; => (B C)
(cons 'd 'e)                  ; => (D . E)
(cdr '(d . e))                ; => E
```

The dot means that the final tail is the value after the dot. `(d . e)` is a pair, not the two-element proper list `(d e)`. Its tail is the symbol `e`, not the list `(e)`. `(a b)` is shorthand for `(a . (b . nil))`.

In a bracket pattern, `[A . B]` binds the head to `A` and the entire tail to `B`. `[[A] . B]` additionally requires the head to be a one-element proper list. These distinctions drive the original tree walker; do not replace dotted pairs by ordinary lists.

`append` concatenates lists, copying all but the final argument. `nconc` performs destructive concatenation. `length` counts elements of a proper sequence; `nth` selects a zero-based element of a list. Later original programs also use arrays: `make-array` creates an array and `aref` accesses an element. Do not pass an arbitrary dotted pair where a proper list is required.

2.1.4 Bindings and Updating Values

`let` evaluates its initializers before its new bindings are available to the body. `let*` introduces bindings one at a time, so a later initializer can refer to an earlier binding.

```
(let* ((x 3) (y (+ x 1))) (* x y)) ; => 12
(let ((x 3)) [x := (+ x 1)] x)    ; => 4
```

`[x := value]` changes an ABCL/R2 binding. `setq` is also available, but this parser permits only one assignment pair in a `setq` form. `setf` is a host macro for updating a *place*, such as the `cdr` of a pair or an array element. Do not confuse changing a binding with changing an existing data structure.

An *association list* stores bindings as pairs:

```
(let ((bindings '((n . 3) (balance . 20))))
  (cdr (assoc 'balance bindings))) ; => 20
```

`assoc` finds the pair whose key matches. `pairlis` pairs a list of names with a list of values. The original environment and script examples use association lists to describe bindings; those lists are data, whereas

an environment is an object that implements lookup and update messages. A local set of bindings for one scope is called a *frame*; an environment can link its frame to an enclosing environment.

2.1.5 Reading a Loop

ABCL/R2 implements `do` directly. Read its three parts as local bindings, a termination clause, and a body:

```
(do ((n 0 (+ n 1)))
    ((= n 3) n)
  (format t "~A~%" n))
; Prints 0, 1, 2 on separate lines; returns 3.
```

`n` starts at zero. Before each iteration the termination test is checked. If false, the body runs and the step expression updates `n`. If true, the result expressions of the termination clause run. With several step expressions, their new values are computed before the loop variables are updated. A variable with no step expression keeps its value unless the body changes it.

The tree walker uses `(do ((tree received-tree)) ((null tree)) ...)`. There is no step expression: its `match` clauses explicitly update `tree`. Its loop result is `nil`, but the handler subsequently sends `!:finish`. The loop result is not that reply.

2.1.6 Functions, Macros, and Reader Notation

`defun` defines a host Lisp function; its parameter list describes arguments and its body computes a result. `#'car` is shorthand for `(function car)` and denotes a function, not the symbol `'car`. `funcall` calls a function value; `mapcar` applies a function to corresponding list elements:

```
(mapcar #'car '((a . 1) (b . 2))) ; => (A B)
```

`apply` calls a function using a list for its final arguments. The priority queues use `(apply #'sort item-list predicate keys)`: when `keys` is `(:key #'first)`, this is a call to `sort` with that keyword pair appended to its arguments. `sort` can rearrange its input list destructively; use its returned list rather than assuming that the old head still names the result. The host function is called with values, not ABCL forms to execute later.

```
(apply #'+ '(1 2 3))             ; => 6
(let ((items (list :a :b)))
  (push :new items)
  (list (pop items) items))       ; => (:NEW (:A :B))
```

`push` changes a place to hold a new cons cell; `pop` returns its first element and changes the place to the remaining tail. `pushnew` inserts only when the item is not already present under its equality test. These macros update bindings or places; they are not just pure list constructors. `caar`, `cadr`, and `cddr` abbreviate `car` of `car`, `car` of `cdr`, and `cdr` of `cdr`, respectively.

`lambda` describes an anonymous host function. As explained below, it does not automatically capture ABCL/R2 state. A *closure* is a function value together with the lexical bindings it retains. Host Lisp closures retain host Lisp bindings, not arbitrary bindings in the ABCL/R2 interpreter's environment objects.

A macro transforms source expressions before the transformed program is translated or executed. Unlike a function, it receives argument forms, not their computed values. `defmacro` defines such a transformation. Backquote builds a source template; comma inserts a form into it, and comma-at splices a list of forms into the surrounding list. The original inheritance macro is a compact example:

```
(defmacro inherit (&rest args)
  `(reify-args-to-meta :inherit ,@args))
```

`&rest args` collects all argument forms into a list. Thus `(inherit Any Reply Sender bird)` becomes `(reify-args-to-meta :inherit Any Reply Sender bird)`. The macro inserts the tag and preserves the argument forms; it does not itself perform inheritance. You only need to recognize the transformation here; Section 11 explains what the reflective operation does.

Other parameter markers you will meet are `&optional` for optional positional arguments and `&key` for keyword arguments. `&aux` introduces initialized local variables in a host function or macro's parameter list. The macros in Copy and Report use it with `gensym`, which creates fresh symbols so generated temporary bindings do not accidentally capture a name in the caller's source. In `(inc "demo" :use-source t)`, the keyword selects an option and `t` supplies its value. `r2sys::create-primary-eval` names an internal symbol in the runtime package. A package groups symbol names; the double colon exposes an internal name rather than promising a portable public API. `#.` in a source file requests evaluation while reading; it is different from an ordinary expression executed later. Do not treat a source file containing it as inert quoted data. `#\>` is a character literal, distinct from the one-character string `">"`; Copy's display-name helpers inspect such characters in strings without changing the object's identity.

`#| ... |#` encloses a block comment; many original files put sample sessions there, so loading the file does not run those commands. `#+feature` includes the next form only if that reader feature is enabled, and `#-feature` does the opposite. The original sources use `#+COMMENT` for inactive alternatives in the normal build. This is a reader condition, not a block comment or an extra runtime branch: do not combine such alternatives with the active implementation.

`format` uses a control string: `~A` prints a value readably for a person, `~S` prints Lisp notation, and `~%` emits a newline. The first argument `t` selects standard output. `~D` prints a decimal integer and `~&` begins a fresh line when needed. An open output stream can replace `t`; the monitoring examples use `open` to create their log streams and `close` when recording ends. These explanations are enough to read the printer in the original future example without assuming familiarity with Common Lisp formatting.

Checkpoint, with answers. What are the values of `(cdr '(a b))` and `(cdr '(a . b))`? The first is the list `(B)`; the second is the symbol `B`. Why does `'n` work as an environment lookup key? Because the environment needs the variable's name as data, not its value. Why is the inheritance macro not an ordinary function? Because it constructs a reflective expression from unevaluated source forms.

2.2 The Boundary with Common Lisp

ABCL/R2 uses a Common Lisp host, but its scripts are not arbitrary Common Lisp programs. The following is a compatibility boundary, not a list of prerequisites you must already know:

Supported special forms include `declare`, `function`, `if`, `let`, `let*`, `progn`, `quote`, and `setq`. The `do` form is also implemented as a special form. This version does not provide the entire Common Lisp language: `tagbody/go`, `block/return`, and multiple-value facilities are unavailable. ABCL forms such as `wait-for` and `match-loop` are also unsupported. The original guide additionally excludes `wait-for-loop`, routine calls, the `suicide` form, and the brace-delimited form for sending several synchronous requests together [1]. This does not exclude an ordinary now-type send: `<==` still waits for one request's reply. Use explicit reply protocols or futures when coordinating several requests. The contents of `declare` are ignored by this implementation; do not rely on a declaration to validate data.

A boundary with host Lisp. A `function` form can embed a host Lisp lambda, but that lambda cannot capture the surrounding ABCL/R2 state or lexical environment. Its body cannot contain ABCL/R2 message sends or object definitions. Bindings introduced inside the lambda are ordinary host Lisp bindings. Use an object script, rather than a host lambda, when you need access to ABCL/R2 state or communication [1].

3 Objects, Scripts, and Messages

3.1 A Small Useful Object

Define an object using `[object ...]`. Its behavior is specified in a `script` block. The arrow `=>` connects a message pattern to the body that handles a matching message.

```
[object greeter
  (script
    (=> [:hello name]
        (format t "Hello, ~A!~%" name)))]
```

The pattern `[:hello name]` binds `name` to the argument of a message tagged `:hello`. The object is bound to the name `greeter`. To trigger its greeting, send a message:

```
[greeter <= [:hello "Ada"]]
; Prints: Hello, Ada!
```

3.2 Replying with !

The expression `!exp` sends the value of `exp` to the reply destination of the current message. Here is a simple calculator:

```
[object calculator
  (script
    (=> [:add x y] !(+ x y))
    (=> [:square x] !(* x x)))]

[calculator <== [:add 2 3]]       ; => 5
[calculator <== [:square 7]]      ; => 49
```

The exclamation mark denotes a reply in the message protocol. Use ordinary Common Lisp functions such as `format` for side effects such as logging or console output. An ordinary final expression does not automatically reply to the sender: use `!` or send explicitly to the bound reply destination. Conversely, `!` sends a message; it does not end the handler. Later expressions can still execute, and several `!` forms can send several replies. Use exactly one reply for an ordinary now-type request; a future stream can deliberately accept several.

3.3 Message Patterns

Patterns can contain variables, literals, and nested lists:

```
[object router
  (script
    (=> [:route :urgent text]
        (format t "URGENT: ~A~%" text))
    (=> [:route destination text]
        (format t "to ~A: ~A~%" destination text)))]
```

The first script matches the literal `:urgent`; the second binds `destination` and `text`. In this distribution, the `match` form does not support `where` constraints. For constraints on *script patterns*, load the `heavy-meta` library before defining the object. These are distinct facilities: loading the library does not add constraints to `match`. Section 13.9 follows the original counter and explains the required metaobject setup, including explicit groups.

The ordinary metaobject selects the first matching script in its script list. Put specific patterns before catch-all patterns, as the original bird/emu example in `examples/misc/inherit.abclr2` does. A message with no matching script is warned about and discarded; it is not retained until a suitable script appears. A now-type caller receives no automatic failure reply in that case. Check message tags and shapes before assuming that a wait will finish.

A header can also bind the sender separately from the payload and reply destination: `(=> pattern @ R from S body)`. Here `R` is where the reply should go and `S` is who sent the request; forwarding can make those different objects. The original inheritance example uses both fields.

3.4 Parentheses, Brackets, and Quoting

Parentheses normally express a Lisp function call: `(+ 2 3)` calls addition. Brackets have several roles. A recognized operator makes them an ABCL form, such as object creation, assignment, or message transmission. Otherwise, they construct a list of evaluated values. Thus `[:add 2 3]` constructs the data `(:ADD 2 3)`; it does not call an addition method by itself. The receiver's script gives that data meaning.

```
(let ((x 7))
  [:point x (+ x 1)])     ; => (:POINT 7 8)
'(red green blue)        ; Literal symbols, not variable lookups
[:label "red"]           ; A keyword and a string
```

A keyword such as `:point` evaluates to itself. An ordinary symbol such as `x` denotes a variable in an expression, but becomes a binding variable in a pattern. The same bracket syntax therefore serves as construction at the sending site and decomposition at the receiving site. Semicolons introduce comments to the end of a line.

3.5 Matching Data Inside a Script

Use `match` when an already selected handler needs to inspect a value. Clauses use `is`, and a final `otherwise` clause provides a fallback. The wildcard `_` matches a value without introducing a named binding. A dotted pattern separates a list's head from its remaining tail.

```
[object classifier
 (script
  (=> [:classify value]
      (match value
       (is [:point x y] !(list :point x y))
       (is [head . tail] !(list :nonempty head tail))
       (is nil !:empty)
       (otherwise !:atom))))]
```

```
[classifier <== [:classify [:point 2 3]]]
; => (:POINT 2 3)
[classifier <== [:classify [1 2 3]]]
; => (:NONEMPTY 1 (2 3))
[classifier <== [:classify nil]]   ; => :EMPTY
[classifier <== [:classify 9]]     ; => :ATOM
```

The point case must precede the broad head/tail case in this `match`: a point is also a nonempty list. Pattern matching tests structure; it does not validate that coordinates are numbers. Use `if` in the selected body for such a check.

3.6 Temporary Variables and Loops

An object's `state` survives successive messages. A `temporary` declaration at the beginning of a handler instead creates local variables for that invocation. Here the accumulator starts at zero every time:

```
[object list-sum
 (script
  (=> [:sum values]
      (temporary [total := 0])
      (do ((rest values (cdr rest)))
          ((null rest))
        [total := (+ total (car rest))])
      !total))]
```

```
[list-sum <== [:sum [2 3 4]]]   ; => 9
[list-sum <== [:sum nil]]       ; => 0
```

The `do` loop binds `rest`, tests for termination, runs its body, and updates `rest` using `cdr`. You can also use `let` for a local accumulator. This distribution provides `dotimes` and `dolist` macros that expand into supported `do` forms. Do not assume that every host Lisp macro expands into forms the ABCL parser understands.

Exercise 1. Define `sum-of-squares` with a `[:sum values]` handler. Reuse the loop structure, but add the square of each number. Check that an empty list returns zero. Compare your reasoning with Section 15.1.

4 State and Assignment

A state entry has the form `(state [variable := initial-value])`. Assignment uses `[variable := expression]`.

```
[object counter
  (state [n := 0])
  (script
    (=> :inc
        [n := (+ n 1)]
        !n)
    (=> :get !n)
    (=> [:add amount]
        [n := (+ n amount)]
        !n))]

[counter <== :inc]         ; => 1
[counter <== [:add 4]]     ; => 5
[counter <== :get]         ; => 5
```

State variables are directly accessible inside a script. Expressions in a script body execute in sequence. The example also shows that a message pattern can consist of a single keyword.

4.1 Lexical Variables

ABCL/R2 shares lexical variables among objects created within their scope. This differs from ABCL/1: a change to a shared variable can become visible to other objects. With concurrent accesses, do not assume a particular execution order. Encapsulate shared state in an object responsible for handling access to it.

5 Two Kinds of Messages

The syntax distinguishes two basic ways to send messages:

Form	Meaning	Typical use
`target <== message`	Now-type message: the caller waits for a reply.	Queries or computations whose result is needed immediately.
`target <= message`	Past-type message: delivery is asynchronous.	Events, starting work, and actor-style pipelines.
`target <= message @ reply`	Past-type message with an explicit reply destination.	Sending a later reply to a specified object.

```
; Query and wait for the result:
[calculator <== [:square 9]]

; Start work asynchronously:
[router <= [:route :urgent "done"]]
```

A past-type message lets the sending computation continue without waiting for a reply. This raises practical questions: When is the work complete? In what order do messages arrive? How is shared state accessed? The protocol between your objects should answer these questions.

5.1 Explicit Reply Destinations and Continuations

A reply destination is itself an object. A continuation object is convenient for a short reply handler:

```
[counter <= :get @ [cont value
                    (format t "The counter is ~A~%" value)]]
```

The form `[cont pattern body]` is an efficient shorthand for an object with one script. Its body should finish quickly and predictably. Loops or additional now-type messages in continuation bodies may lead to deadlock. Its body is compiled as non-reifying code, so `reify-args` and `reify-args-to-meta` cannot be used there [1]. These forms expose expressions and their execution context to a program; Section 11 introduces that reflective access. For now, use continuations for short, ordinary reply processing.

5.2 Forwarding a Reply Without Waiting

The `@ R` field in a script header binds the current reply destination. It is different from an argument in the message payload. This forwarding object keeps the original caller's destination:

```
[object forwarder
 (script
  (=> [:square x] @ R
      [calculator <= [:square x] @ R]))]
```

```
[forwarder <== [:square 8]]     ; => 64
```

The caller waits on the forwarder, but the forwarder's handler does not wait on the calculator. It sends a past-type message with `@ R`. The calculator's reply then goes directly to the original caller. Do not add an extra `!` to this handler: that would send a second reply. This style is useful when an object coordinates work but does not need the result itself.

5.3 Object Identity and the Current Object

Inside a script, `Me` denotes the current object. A handler can send a message to itself using `[Me <= message]`, usually to arrange work after the current handler finishes. Under the ordinary metaobject, a handler is processed to completion before another normal handler for that object begins. This remains true when the current handler is suspended waiting on a now-type reply.

Consequently, a now-type message to the same ordinary object can deadlock: the current handler waits for a reply from a handler that cannot begin. The same problem arises in a cycle: object A waits on B while B waits on A. Use forwarding or a later past-type phase to break that wait cycle. Ordinary queue serialization protects the object's handlers from overlapping; it does not make operations on several objects into one transaction. A *join* waits for the notifications required by an application protocol, such as both children's answers, before continuing. It does not mean that every object or host thread has stopped.

5.4 Concurrency and Parallel Hardware

Independent objects can be logically concurrent even when the host executes one computation at a time. The supplied runtime has a scheduling queue in `src/process.lsp`; its existence does not imply that an object is mapped to a dedicated operating-system thread or CPU. Long computations in a host Lisp function can also delay other work. Use the word *concurrency* for the protocol model, and measure the particular runtime before making claims about parallel speedup.

6 A Complete Example: A Bank Account

The following object combines state, patterns, replies, and a simple invariant. It rejects withdrawals that would make the balance negative.

```
[object account
  (state [balance := 0])
  (script
    (=> [:deposit amount]
        (if (> amount 0)
            (progn [balance := (+ balance amount)] !balance)
            !:invalid-amount))
    (=> [:withdraw amount]
        (if (and (> amount 0) (>= balance amount))
            (progn [balance := (- balance amount)] !balance)
            !:rejected))
    (=> :balance !balance))]

[account <== [:deposit 100]]     ; => 100
[account <== [:withdraw 30]]     ; => 70
[account <== :balance]           ; => 70
```

Exercise 2. Add a `[:transfer other amount]` handler. First check whether the withdrawal is possible, then send a deposit to `other`. Consider whether to use a now-type or past-type message, what should happen if the destination rejects the deposit, and when the transfer can be reported as complete. Compare your protocol with Section 15.2.

7 Concurrency as a Programming Model

A producer can send work to a consumer without waiting for an individual reply to each item:

```
[object consumer
  (script
    (=> [:item x]
        (format t "Processed: ~A~%" (* x x))))]

[object producer
  (script
    (=> :run
        (progn
          [consumer <= [:item 2]]
          [consumer <= [:item 3]]
          [consumer <= [:item 4]]
          !:sent)))]

[producer <== :run]
```

The three past-type messages are sent to `consumer`. The reply `:sent` confirms that the producer has sent the requests; it does not confirm that the consumer has completed them. Avoid relying on an assumed global execution order, especially with multiple senders. A clear protocol specifies both permitted messages and how completion is acknowledged.

Exercise 3. Give the consumer a state counter `n` that increases after each `:item`. Add a `:count` message that returns its current value. Experiment with querying the count while work is still outstanding. Then design an acknowledgment that tells the producer when all its items have been processed. Compare your completion condition with Section 15.3.

8 Creating and Organizing Objects

Objects can be created inside expressions. Start with an ordinary local object:

```
(let ((prefix "[log]"))
  [object local-logger
    (script
      (=> [:write text]
          (format t "~A ~A~%" prefix text)))]
  [local-logger <= [:write "start"]])
```

The object name is a binding in the surrounding environment. The object can use lexical variables from that environment. Because these variables are shared, take care when mutating them.

A *group* supplies an object-creation and execution policy for its members. It is not a class or a shared state container. An object normally uses its creation context's group; an explicit `(group manager)` selects another group manager. Groups and custom metaobject generators become important in Section 11.

```
[group logging-group]
[object logger-a (group logging-group) (script (=> :ping !:ok))]
[object logger-b (group logging-group) (script (=> :ping !:ok))]
(eq [group-of logger-a] logging-group)          ; => T
(eq [group-of logger-a] [group-of logger-b])     ; => T
(eq logger-a logger-b)                          ; => NIL
```

Here `group-of` returns the ordinary object's group manager. Sharing that manager does not make two objects identical or combine their state. Custom generators in the original projects must be read on their own terms; selecting a generator does not certify every introspection operation it might support.

9 Files, Libraries, and Tools

Loading a file is part of program execution in ABCL/R2. Its expressions run in an environment, using a group and evaluator; the same translated file can therefore behave differently when loaded by different callers. First separate preparing a file from executing it:

Expression	Purpose
`(inc "name")`	Translate if needed, then load and evaluate in the caller's environment, group, and evaluator context.
`(comp "name")`	Translate and compile if needed; does not run the ordinary ABCL application expressions.
`(inc-lib "name")`	Load and evaluate in the global environment, top-level group, and current top-level evaluator.
`(require 'name)`	Search for and load an unregistered module through `inc-lib`.
`(provide 'name)`	Record a module name in `*modules*`; does not load a file or validate its definitions.
`(pwd)` / `(cd "dir")`	Historical directory commands; not implemented for SBCL in this port.
`:q` / `:reset`	Quit the interpreter or reset the system.

Use a file's *base pathname*, without its type: `(inc "demo" :use-source t)` loads `demo.abclr2`. Translation writes a neighboring `demo.lsp`; compilation writes a host-specific compiled file, normally `demo.fasl` on SBCL. Do not pass the `.abclr2` type as part of the base name: the loader derives all three paths from that argument. The directory must permit writing the generated files. Use scratch copies when exploring the original examples, as the checks do.

Relative paths are resolved against the process's current directory, not automatically against the containing source file's directory. Start the interpreter in the required directory or use absolute paths. The host-specific implementations in `src/topfunc.lsp` have no SBCL branch for changing that directory. Calling `inc-lib` does not make its relative path relative to `lib/`.

9.1 Options and File Selection

The user's guide specifies the public options [1]; the active implementations are in `src/inc.lsp` and `src/user.lsp`. The defaults are:

Option	Default	Accepted by and effect
`:use-source`	`nil`	`inc`, `inc-lib`, `require`, `comp`. Non-nil forces translation from ABCL source. On a load it also bypasses the compiled-file branch; on `comp` it forces compilation.
`:verbose`	`t`	All four commands. Controls translation and the passed-through load/compile verbosity, not every warning or diagnostic.
`:compile`	`nil`	`inc`, `inc-lib`, `require`. Non-nil runs `comp` before loading. `comp` itself has no `:compile` option.

`inc` and `inc-lib` can prompt for a filename when it is omitted. `comp` requires a name, and `require` requires a module name. In scripts and reproducible sessions, always supply it explicitly. `provide` takes only its module-name argument.

Follow the three-stage choice. For an explicit `inc` or `inc-lib`, read the loader as follows:

1. If `:compile` is true, first perform the compilation command with the selected `:use-source` and `:verbose` values.
2. Require the `.abclr2` source to exist. Translate it if forced, if there is no `.lsp`, or if the source is newer than the translation. Even a cached compiled file does not remove this source-file requirement.
3. Load the compiled file only when `:use-source` is false, that file exists, and it is *strictly newer* than the Lisp translation. Otherwise load the translation. Execute its expressions in the selected loading context.

`comp` uses the same translation test, then compiles if forced, if the compiled file is absent, or if the translation is newer than the compiled file. Otherwise it reports that the compiled file is up to date. It does not execute an ordinary ABCL object definition or application send merely by compiling it. Translation still processes macros; do not equate compilation with an absence of all host-side effects.

Timestamp comparisons are not content comparisons. Equal timestamps do not satisfy "strictly newer" in the load test, so loading may select the Lisp file even after compilation. The combination `:compile t :use-source t` compiles first but still follows the forced-source load branch. Compilation is not a promise that this particular load executes the FASL. This is usually a performance/file selection distinction, not a different intended program result.

For day-to-day editing, prefer the explicit source path:

```
(inc "demo" :use-source t)          ; retranslate, load, and run
(comp "demo" :use-source t)         ; prepare compiled output; do not run
(inc "demo" :compile t)             ; update as needed, then load and run
```

These are three independent commands, not a required sequence. Do not repeat a loading command solely because a translation was cached: `inc` and `inc-lib` execute the file's expressions on each call, including assignments, object creation, and application sends. `:verbose nil` can reduce output while keeping those effects.

9.2 Execution Context and Module Loading

An included fragment sees its caller's environment. The supplemental `tutorial/examples/15-loaded-fragment.abclr2` contains just these two expressions:

```
[loaded-value := load-token]
[object loaded-object (script (=> :value !load-token))]
```

It deliberately depends on `load-token`; it is a fragment to load in a prepared environment, not a self-contained program. Try:

```
[load-token := :global]
[loaded-value := :unchanged]
[loaded-object := nil]
(let ((load-token :local) (loaded-value nil) (loaded-object nil))
  (inc "tutorial/examples/15-loaded-fragment" :use-source t)
  (list loaded-value [loaded-object <== :value]))
; => (:LOCAL :LOCAL)
loaded-value                        ; => :UNCHANGED
```

The fragment updates the local `loaded-value`, binds its new object locally, and that object's script retains the local `load-token`. The same fragment loaded by `inc-lib` instead uses the global environment:

```
(let ((load-token :local) (loaded-value nil) (loaded-object nil))
  (inc-lib "tutorial/examples/15-loaded-fragment" :use-source t)
  (list loaded-value loaded-object)) ; => (NIL NIL)
loaded-value                         ; => :GLOBAL
[loaded-object <== :value]           ; => :GLOBAL
```

Group context matters too. Ordinary objects created by `inc` inside a client's script inherit that caller's group; `inc-lib` evaluates the file in the top-level group using the current `*eval*`. An explicit `group` or `meta-gen` in the loaded definitions can still choose different machinery. Loading a library does not itself replace existing objects' evaluator references or compose its policy with every custom group. This is why the original examples' setup and loading order must be read as part of their protocols.

Require a module name, not an arbitrary file path. `*library-directories*` contains the directories searched by the ABCL/R2 `require` macro. It selects the first directory containing the named `.abclr2` source, then uses `inc-lib` to load it. The default value is `(list "")`: only the current working directory is searched. The distribution's `lib/` directory is not added automatically. Before running examples that require its modules, add it at the ABCL/R2 prompt, starting the interpreter at the repository root:

```
[*library-directories* :=
  (cons #P"lib/" *library-directories*)]
(require 'future)
```

Here `#P"lib/"` is Common Lisp's pathname literal for a directory. Keep the trailing slash: the search uses the directory component of each entry. Relative entries are resolved against the working directory, not the executable's location or the directory of the file being loaded. Use an absolute directory pathname when starting elsewhere. Explicit `inc` and `inc-lib` calls do not search this list; `(inc-lib "lib/future")` instead names the file directly. The path is session configuration; after `:reset`, set it again.

Symbols are converted to lowercase names. String arguments retain their case during search; prefer symbols or explicitly lowercase strings. `provide` normalizes both symbols and strings to lowercase.

For your own library directory, use a directory pathname, save the previous search path, and restore it after the experiment:

```
[saved-library-directories := *library-directories*]
[*library-directories* :=
  (cons #P"/absolute/path/to/my-libraries/" *library-directories*)]
; (require 'my-module) ; loads my-module.abclr2 from the search path
[*library-directories* := saved-library-directories]
```

The absolute example directory is a placeholder to replace, not a required installation path. Putting it first makes its source win over later directories. A typical module includes `(provide 'my-module)` to register its name in `*modules*`. `require` does not register a module automatically: without `provide`, a second require can execute the file again.

Once a name is registered, `require` skips loading entirely, even with `:use-source t` or `:compile t`. To intentionally reload a known file while editing, use its explicit base pathname with `inc-lib`. Be aware that this reexecutes definitions and effects; it is not an atomic replacement of every reference to earlier objects. If `provide` ran before a later failure in a module, its recorded name alone is not evidence that all initialization succeeded.

A missing module also returns `nil` without loading in this implementation. That return value is not a success certificate or a reliable distinction between "already registered" and "not found." Inspect the transcript, expected definitions, and search path. Direct `inc` and `inc-lib` instead signal an error for missing source.

Included libraries are `future`, `multicast`, `express`, `heavy-meta`, `copy`, `watch`, and `report`. Their walk-throughs explain setup and use. Historical alternatives with shared feature or generator names require separate fresh sessions, not an assumption that `require` can distinguish filenames by itself.

Evidence and check your understanding. Run `make -C tutorial loading-check`. Thirty-five probes check option defaults, compile-versus-load effects, independently marked Lisp and FASL cache selection, forced source, missing-source rejection, module registration/reload/search, and local versus global environment, group, and evaluator behavior. Tests modify only generated cache files in `build/loading-tests/`; original sources remain unchanged. The small fragment above is copied there before testing.

Why might changing a module and calling `require` show no change? *Answer:* The registered name bypasses the load, including its source option. Why can `comp` create output without creating the application object? *Answer:* Preparation does not execute its ordinary ABCL expressions. Why does a file see a local value with `inc` but the global value with `inc-lib`? *Answer:* Loading selects an execution environment, not just a file.

10 Futures and Express Messages

Future-type messages are a library feature. In a fresh session started at the repository root, configure the search path as explained in Section 9.2, then load the library:

```
[*library-directories* :=
  (cons #P"lib/" *library-directories*)]
(require 'future)
```

Futures allow a result to be requested and consumed later, separating the request from the point at which the result is needed. See `lib/future.abclr2` and `examples/misc/future-ex.abclr2` for the specific syntax and examples.

10.1 A Future Is a Reply Stream

The supplied future library can collect more than one reply from a request. The send syntax uses a dollar sign: `[receiver <== message $ future-object]`. Each `!` in the receiver's handler appends another reply to the future. This is more general than a promise that resolves to one value.

To run the following example from the repository root:

```
(inc "tutorial/examples/04-futures" :use-source t)
[range-client <== [:collect 1 4]]
; => (1 2 3 4)
```

The file loads the supplied future library. Here is the server:

```
[object range-server
 (script
  (=> [:range first last]
      (do ((i first (+ i 1)))
          ((> i last))
        !i)
      !:done))]
```

The server sends each integer and then `:done`. The terminal message is part of our protocol: futures themselves do not know that a producer has finished.

```
[object range-client
 (script
  (=> [:collect first last]
      (temporary [stream := (make-future)]
                 [items := nil]
                 [value := nil])
      [range-server <= [:range first last] $ stream]
      [value := (next-value stream)]
      (do ()
          ((eq value :done))
        [items := (cons value items)]
        [value := (next-value stream)])
      !(reverse items)))]
```

The initial send is asynchronous. Each `next-value` waits if no reply is currently available. Thus waiting occurs at the point where the client needs a particular element, rather than at the initial request. The sentinel is removed from the stream but excluded from the result. The range from 3 to 2 is empty and returns `nil`; the server still sends `:done`, so the client does not wait forever.

10.2 Peeking, Consuming, and Readiness

Operation	Behavior in the supplied implementation
`(make-future)`	Create and initialize an empty future.
`(ready? f)`	Return whether at least one reply is buffered.
`(next-value f)`	Wait for and remove the next reply.
`(next-value f :remove nil)`	Wait for the next reply without removing it.
`(all-values f :wait nil)`	Return the current buffered replies, or `nil` if none exist; removal defaults to true.
`(reset-future f)`	Send a reset request; use only when old producers and readers have finished.

The actual macros in `src/user.lsp` default `:remove` to true for `next-value` and `all-values`; an old comment in the library header gives a different default. `all-values` is a snapshot of buffered values, not an operation that waits for a terminal marker. Its `:wait t` default waits when the buffer is empty, but does not establish producer completion.

For stream consumers, use `next-value` and an explicit terminal marker. Use `all-values` with `:wait nil` to inspect the replies currently buffered; a snapshot is not a producer-completion signal.

```
(let ((f (make-future)))
  [range-server <= [:range 8 8] $ f]
  (list (next-value f :remove nil)
        (next-value f)
        (next-value f)
        (ready? f)))
; => (8 8 :DONE NIL)
```

The implementation records one waiting consumer destination. Treat each future as a stream with one active consumer; use a separate future per request rather than sharing one among unrelated readers.

Exercise 4. Change the server to send squares, keeping the terminal marker. Then change the payload to `[:value n]` so that a legitimate data value cannot be confused with `:done`. Compare the two message shapes with Section 15.4.

10.3 Express Messages

Express messages can preempt a script invoked by a normal message. They require the sender to use a suitable evaluator and the receiver to use a suitable metaobject generator. The `express` library provides these components. Once configured, the sending syntax is:

```
[receiver <= message :express]
[receiver <== message :express]
```

The documented purpose of `atomic` is to prevent express-message preemption while its subexpressions execute. The first experiment below studies routing to an idle receiver. Express messages are an advanced facility: first become comfortable with ordinary past-type and now-type messages.

10.4 An Express Exchange

An express send needs both ends configured: the sender uses an express evaluator, and the receiver uses an express metaobject generator. Begin with the original clock while it is idle. In a fresh session at the repository root, load its library and definitions:

```
(inc-lib "lib/express" :use-source t)
(inc "examples/misc/express-ex" :use-source t)
```

The example defines the group `express` and its clock. Place the client in that group too, so its evaluator understands the extra `:express` send argument:

```
[object finite-express-sender
 (group express)
 (script
  (=> :peek ![clock <== :peek :express])
  (=> :past-peek @ R [clock <= :peek @ R :express])
  (=> :atomic-result !(atomic 42)))]
```

Try these short exchanges:

```
[clock <== :peek]                          ; => 0
[finite-express-sender <== :peek]          ; => 0
[finite-express-sender <== :past-peek]     ; => 0
[finite-express-sender <== :atomic-result] ; => 42
```

The first query is ordinary. The second goes through an express-capable sender. The third forwards the caller's reply destination through a past-type express send; `@ R`, not `<=` alone, supplies that reply path. The last expression shows that `atomic` returns its body's value.

This exercise studies routing to an idle receiver, not interruption of a running computation. Section 13.16 follows the original clock's scripts, mode transitions, and evaluator front end.

11 Reflection and Hybrid Group Architecture

11.1 What Reflection Adds

So far, our messages have been about application data: a balance, a job, or a value in a stream. *Reflection* lets a program inspect and operate on representations of its own structure and execution. It is not merely another domain object or another name for inheritance. For example, an account's application protocol deposits money; a reflective protocol can inspect the environment holding its balance, the scripts available to it, or the evaluator executing those scripts.

Call ordinary application activity the *base level*. Activity that describes or controls it is at a *meta-level*. These are roles in a relationship, not dedicated CPU threads. A *metaobject* represents an individual object's execution machinery. An *environment* represents bindings and lexical lookup. An *evaluator* executes expression representations in an environment and context, reporting the result to a continuation. A *scheduling policy* decides which pending work to run next; evaluation describes how an expression is executed. The two can interact: an evaluator object can queue requests and apply its own scheduling policy. Do not confuse that object with the host runtime's scheduling machinery. A *metaobject generator* is a factory for metaobjects. A *group manager* represents a group and participates in creation using that group's generator and evaluator. The evaluator, metaobject generator, and group manager are the group's *kernel objects*.

Two distinctions will keep the examples readable. First, changing the machinery that executes a script is different from changing the application's message payload. Second, replying to the application caller is different from reporting that an expression has finished executing. The original inheritance reading project makes both visible.

Reification exposes an execution representation for reflective computation. ABCL/R2 avoids eagerly constructing an infinite stack of such representations: higher levels are created lazily. Its *partial compilation* compiles ordinary computations while keeping selected operations available through reflective protocols. The compiled representations passed to an evaluator are therefore not simply source strings to be re-read. A *non-reifying* object's script is compiled for direct execution rather than this reflective expression interface; it cannot use `reify-args` or `reify-args-to-meta`. This does not prohibit ordinary messages to other objects' reflective interfaces: the breakpoint inspector later uses environment objects in exactly that way. These implementation techniques are explained in Masuhara's thesis and the distribution's compiler documentation [2, 1].

An object definition can therefore contain additional parameters alongside `state` and `script`:

```
; Schematic: these kernel objects must be defined first.
[object named-object
  (meta-gen my-meta-generator)
  (group my-group-manager)
  (priority 10)
  (script
    (=> :hello !:ok))]
```

The interpretation of additional parameters depends on the chosen runtime objects; `priority` has no universal scheduling meaning. To explore a custom evaluator, study the files `express-ex.abclr2`, `built-ins.abclr2`, and `reify-args.abclr2` in `examples/misc/`.

The special forms `reify-args` and `reify-args-to-meta` send compiled argument expressions, together with their evaluation context, to an evaluator or to the current object's metaobject, respectively. They are unavailable in scripts of non-reifying objects.

This makes debugging, scheduling, resource management, and alternative evaluation strategies expressible as programs in the language. Reflection adds an abstraction level, so encapsulate it in small, well-understood libraries.

11.2 Two Reflective Towers, Two Kinds of Control

The Hybrid Group Architecture distinguishes reflection on an individual object from reflection on the group in which objects execute. Masuhara's thesis, Section 3.3, develops this distinction [2].

Level	What it controls	Example

Individual metaobject	The object's state environment, scripts, and normal message queue.	Inspect a counter's state or change its evaluator.
Group kernel objects	Shared evaluation and object creation policy.	Give a group a custom evaluator for a new control form.
Higher meta-levels	The corresponding behavior of metaobjects and meta-groups.	Reflect on the evaluator itself.

This conceptual hierarchy is not allocated as an infinite collection of objects. Chapter 4 of the thesis explains lazy creation and lightweight metaobjects: forwarding an ordinary message should not force expensive reification. Section 4.7 discusses non-reifying objects; Chapter 7 summarizes the use of partial compilation [2]. Sequential host computations can be compiled while selected operations still pass through the reflective protocols.

11.3 Inspecting an Object's State

Load the reflection example in a fresh session:

```
(inc "tutorial/examples/05-reflection" :use-source t)
[observed-counter <== :inc]                  ; => 1
[[[meta observed-counter] <== :state]
 <== [:value-of 'n]]                         ; => 1
[[[meta observed-counter] <== :state]
 <== [:set 'n 40]]                           ; => 40
[observed-counter <== :inc]                  ; => 41
```

The steps are significant. `[meta observed-counter]` obtains the counter's metaobject. The `:state` query returns an environment object, not a plain property map. Sending `[:value-of 'n]` to that environment retrieves a binding; `[:set 'n 40]` changes it. The quoted symbol is the variable's name as data. The next ordinary increment observes that change: the meta-level representation is connected to the computation it describes.

This bypasses the object's application protocol. Use it to understand reflection and debugging; ordinary clients should continue to use the counter's defined messages. Default metaobjects also answer `:scripts`, `:queue`, and `:evaluator`. Their precise replies are runtime objects or internal queue entries. The operation `[den metaobject]` returns the associated lower-level object, called its *denotation*. It is not a copy or a variable's value. In the original inheritance project we will check both identity directions and distinguish this relationship from group membership.

11.4 Defining a Lazy Control Form

An ordinary function evaluates its arguments before receiving them. A control form must instead decide which expression to evaluate. The next example implements a choice using the group's evaluator:

```
[group choice-group
 (evaluator
  [object
   (state [primary := (r2sys::create-primary-eval group)])
   (script
    (=> [:choose [predicate then else] Env Id Gid Eval] @ C
        [Eval <= [:do predicate Env Id Gid Eval]
              @ [cont result
                 [Eval <= [:do (if result then else)
                           Env Id Gid Eval] @ C]]])
    (=> any @ R
        [primary <= any @ R]))])]
```

The new evaluator handles the `:choose` protocol and forwards all other messages to a primary evaluator. The internal helper `r2sys::create-primary-eval` matches the helper used by the repository's examples; it is specific to this implementation. Inside a group definition, `group` names the group being constructed.

The expression variables `predicate`, `then`, and `else` contain compiled expression representations. They are not ordinary argument results. The `[:do expression Env Id Gid Eval]` protocol asks the evaluator to execute one representation in its original context. Read the context fields by role, not as application arguments:

Field	Meaning in this evaluation request
`Env`	Environment used for variable lookup.
`Id`	Metaobject identity of the object whose expression is being executed; evaluating `Me` produces `[den Id]`.
`Gid`	Group manager for that execution context; evaluating `group` produces this value.
`Eval`	Evaluator used for the expression's evaluation operations.
`C`	Continuation receiving this expression's result, not necessarily the application's original caller.

When a metaobject's own handler executes, `Me` is that metaobject, not the application object it manages. Likewise, `Gid` is not automatically the group of `Id`: a metaobject belongs to a group at its own level. Keep object, metaobject, and group relationships separate.

The first send evaluates the predicate. Its continuation selects one branch representation and forwards the final destination `C`. No unselected branch is evaluated.

```
[object choice-test
 (group choice-group)
 (state [selected := nil])
 (script
  (=> [:choose n]
      !(reify-args :choose
         (evenp n)
         (progn [selected := :even] (/ n 2))
         (progn [selected := :odd] (- n 1))))
  (=> :selected !selected))]
```

```
[choice-test <== [:choose 3]]   ; => 2
[choice-test <== :selected]     ; => :ODD
[choice-test <== [:choose 8]]   ; => 4
[choice-test <== :selected]     ; => :EVEN
```

The current evaluator receives compiled argument expressions through `reify-args`. The corresponding

form for the current object's metaobject is `reify-args-to-meta`. Both preserve the evaluation context, and neither is available in a non-reifying object's script. This example uses a reflective control form to add behavior; it does not change the meaning of the ordinary `if` operator.

The tag supplied to `reify-args` is evaluated first. The remaining argument expressions are passed as compiled representations in a message of the form `[Tag Expressions Env Id Gid Eval] @ C`; they have not already been reduced to argument values [1].

Exercise 5. Extend the choice group with a `:unless` protocol that evaluates one body only when its predicate is false. Forward the final result to the original continuation in both cases. Check both branches against Section 15.5.

11.5 Extension Parameters and Host Events

The user's guide identifies additional parameters on message transmission, object creation, and assignment as extension points [1]. They carry information to the execution machinery; they are not extra elements of the application's message payload. For example, in the schematic send `[target <= payload @ reply :RVT stamp]`, the evaluator representation is `[:send-past Target Payload Reply :RVT Stamp]`. Capitalized names here denote evaluated values. It is the selected evaluator's responsibility to interpret this extension. An ordinary receiver still receives `Payload`.

Likewise, `(priority n)` in an object definition becomes a `:priority` argument in its creation representation, rather than automatically creating a state variable or a universal scheduling rule. The priority-control examples supply machinery that gives this parameter meaning. An extended assignment `[x := value tag argument]` passes the evaluated extras to the environment's `:set` request. Custom environment objects can therefore participate in update policy. Do not assume the default environment acts on an arbitrary tag.

Two library facilities illustrate this principle without extending the core syntax. Script-pattern `where` constraints require `heavy-meta`; constraints inside an ordinary `match` form remain unsupported. Multicast requires `multicast` and a suitable evaluator created by `multicast-eval-gen`; merely loading the library does not replace every object's evaluator. The original `lib/multicast.abclr2` constructs a template for nested target lists and collects replies into their corresponding positions. This is a library-defined protocol, not the unsupported brace-delimited synchronous-send form. Section 13.10 works through its original example and reply paths. These libraries need their own setup and reply/completion analysis, just as the express library does.

The guide also describes a boundary for host events [1]. At Lisp level, `r2sys:send-signal` delivers its arguments as a `[:signal ...]` message to the object bound to `*signal-handler*`. The default handler ignores events; a `nil` handler disables delivery. Alternatively, the host can set `r2sys:*signal-request*`; the runtime checks and clears it before forwarding the request. This is a single pending-request slot, not a general event queue. The ABCL side handles an event as a message. It does not follow that every operating-system signal or timer is automatically connected to this interface: that requires host-specific integration. The interface also does not promise express preemption.

11.6 A Safe Experiment with Synthetic Host Events

The event interface turns a host notification into an ABCL message. You can explore that boundary without a timer, an operating-system signal, or another thread. The following supplemental exercise uses the interface described by the user's guide and implemented in `src/signal.lsp`; there is no corresponding runnable event example in `examples/`. It changes no original source and installs no operating-system callbacks.

Receive and inspect an event. The file `tutorial/examples/14-host-events.abclr2` defines an ordinary recording object. Loading it does not install it as a handler:

```
[object event-inbox
 (state [events := nil])
 (script
  (=> [:signal . parameters] @ R from S
      [events := (cons (list parameters R S) events)])
  (=> :events !(reverse events))
  (=> :clear [events := nil] !:cleared))]
```

The dotted pattern `[:signal . parameters]` binds the argument tail as a list. Each saved record contains that list, the reply destination, and the sender. A host notification arrives with both reply and sender `nil`: it is not an application call waiting for `!`. The separate `:events` query does reply, allowing us to inspect what the handler has processed.

Start a fresh interpreter at the repository root and save its current handler before temporarily replacing it:

```
(inc "tutorial/examples/14-host-events" :use-source t)
[saved-event-handler := *signal-handler*]
[*signal-handler* := event-inbox]
(r2sys:send-signal :tick 7)
[event-inbox <== :events]           ; => (((:TICK 7) NIL NIL))
```

The `r2sys:` prefix names the exported host interface in the ABCL/R2 system package. `send-signal` accepts an ordinary Lisp `&rest` argument list and constructs `(:signal :tick 7)`. It submits the notification; returning from that function is not an acknowledgment from your handler. This finite interactive exercise lets the runtime dispatch the event before inspecting the inbox. Do not infer a general ordering guarantee between independent producers, or use a notification as proof that unrelated work finished.

Passing one list is different from passing its elements as arguments:

```
[event-inbox <== :clear]
(r2sys:send-signal '(:tick 7))
[event-inbox <== :events]           ; => ((((:TICK 7)) NIL NIL))
```

The parameter list now has one element, itself a list. Similarly, `(r2sys:send-signal)` sends an event with an empty parameter list, while `(r2sys:send-signal nil)` sends one with parameter list `(nil)`. Neither call means "disable events."

Explore the single pending-request slot. The other interface is a *host Lisp variable*, `r2sys:*signal-request*`. It is distinct from a variable stored in an ABCL environment. An ABCL assignment such as `[r2sys:*signal-request* := value]` would not be the host-side write this interface requires. The teaching file supplies small host functions to make that boundary explicit:

```
(defun demo-event-slot () r2sys:*signal-request*)
(defun demo-post-event (value)
 (setf r2sys:*signal-request* value))
(defun demo-replace-event (first second)
 ;; No ABCL scheduler step occurs between these two host assignments.
 (setf r2sys:*signal-request* first)
 (setf r2sys:*signal-request* second))
```

These function bodies run in host Lisp. Their `setf` writes the actual host variable; `demo-event-slot` reads it. They are teaching helpers, not additional built-in ABCL/R2 operations. In the same fresh session, first check that the slot is empty:

```
(demo-event-slot)                  ; => NIL
[event-inbox <== :clear]
(demo-post-event '(:tick 7))
[event-inbox <== :events]           ; => ((((:TICK 7)) NIL NIL))
(demo-event-slot)                  ; => NIL: consumed by the runtime
```

A non-nil slot value becomes *one* argument following `:signal`. A list stored there is not spliced into multiple arguments. The original polling function makes this explicit:

```
(defun check-signal-request ()
  (when *signal-request*
    (let ((signal-value *signal-request*))
      (setq *signal-request* nil)
      (send-signal-internal (list *signal-message-tag* signal-value)))))
```

It saves the value, clears the slot, and forwards `(:signal value)`. The scheduler calls this function at the top of its dispatch loop in `src/process.lsp`. The slot is not a background service that keeps an otherwise idle interpreter polling, and its check does not interrupt a host Lisp computation halfway through execution.

Two writes before that polling point do not create two pending events:

```
[event-inbox <== :clear]
(demo-replace-event :first :second)
[event-inbox <== :events]           ; => (((:SECOND) NIL NIL))
```

Both writes happen inside one host function, with no ABCL scheduler step between them. Only the last value remains to be polled. This demonstrates overwrite, not queueing or reliable event aggregation. Writing `nil` means no pending request; use `(r2sys:send-signal nil)` if you actually need an event whose argument is `nil`. These helpers assume an isolated fresh session; do not overwrite a pending request owned by another event producer.

Disable delivery, then restore the saved handler. The handler binding controls both interfaces:

```
[event-inbox <== :clear]
[*signal-handler* := nil]
(r2sys:send-signal :dropped)
(demo-post-event :also-dropped)
(demo-event-slot)                  ; => NIL after polling
[*signal-handler* := event-inbox]
[event-inbox <== :events]           ; => NIL: no replay
[*signal-handler* := saved-event-handler]
(eq *signal-handler* saved-event-handler) ; => T
```

Disabling delivery does not buffer notifications for later. A polled slot value is cleared even when no handler is installed. Conversely, changing the handler binding is not a cancellation of messages already sent to a previous handler. Always restore the object you saved, rather than assuming that the default handler is still the desired one. The normal default handler prints an ignored-signal diagnostic; “ignored” does not necessarily mean silent.

No step in this exercise enables keyboard interrupts or timer delivery. The historical keyboard-interrupt wrapper in `src/signal.lsp` contains host-specific branches; its fallback simply executes its body. Do not infer an SBCL operating-system hook from the presence of this API. Nor is this an Express-message experiment: the original Interrupt and Express examples have their own reflective execution protocols.

Evidence and exercise. Run `make -C tutorial host-events-check`. Twenty-five probes in two fresh processes check handler preservation, direct argument shapes, nil sender/reply fields, slot consumption and overwrite, disabled delivery, absence of replay, and restoration. Inputs, copied teaching source and transcripts remain in `build/host-event-tests/`. Tests install no operating-system callbacks and modify no runtime or original sources.

Why does a list-valued slot notification differ from `(r2sys:send-signal :tick 7)`? *Answer:* The slot carries one value; the function accepts several arguments. Why is a zero-valued slot request still delivered? *Answer:* Only `nil` is false in Lisp; zero is non-nil. Why should the signal handler not rely on `!` to acknowledge the host? *Answer:* The notification has no application reply destination. Define a separate acknowledgment protocol if your application needs one.

12 Troubleshooting and Good Practice

1. Test handlers that return results with now-type messages first. Check that their patterns match and their replies are correct.
2. Then introduce past-type messages and specify when replies and completion acknowledgments are expected.
3. Keep shared state in a responsible object rather than spreading mutable lexical variables across many objects.
4. Use clear message tags, such as `:deposit`, `:withdraw`, and `:shutdown`.
5. After an error, this distribution may display a Lisp error message and attempt to continue execution. Use `:reset` when the session's state is unclear.
6. Build programs incrementally: objects, patterns, state, asynchronous communication, and finally reflection.

13 Working from the Original Examples

Read each project from the application's request to its result before following the machinery underneath. The tree walker uses the futures you already know. The next projects connect the reflection introduction to lazy control forms, inheritance, and interpreters; only then do we combine those mechanisms in libraries, debugging, and scheduling. Pcontrol and Time Warp are the longest projects and can be studied in separate sessions. Appendix B is a return map to their files, not an alternative order in which to learn the concepts.

These projects follow the distributed programs in `examples/`. The calculator, account service, and job service are teaching extensions; use them to prepare for reading the original applications. Line references use code-only views, omitting source-maintenance comments.

Read each result in its own protocol. The reflective examples contain several overlapping conversations. Use this key when following a destination or an apparent completion:

Conversation	What its response establishes
Application request	A reply answers the application's caller. Its destination is often named `R` or `Reply`.
Expression evaluation	A result sent to an evaluation continuation, often `C`, finishes that expression, not necessarily the work it has requested elsewhere.
Object dispatch	A `:end` message in the ordinary dispatch cycle lets the metaobject advance its queue; it is not a reply to the client.
Object creation	A generator's reply supplies a new metaobject to the creation machinery; it is not an application-method result.

The names are conventions in the examples, not fixed roles for every variable with that spelling. A now-type send can use its evaluation continuation as the application's reply destination, deliberately joining two of these paths. Follow the actual sends before deciding that two destinations are different, or that one result means all work is complete.

13.1 Tree Leaves: The Original Future Protocol

`examples/misc/future-ex.abclr2` supplies a useful starting point: a producer walks a nested tree and sends its leaves through a future. Here is its original producer, without adaptation:

```
[object tree2leaves
  (script
   (=> received-tree
       (do ((tree received-tree))
           ((null tree))
         (match tree
           (is [[A] . B]
               [tree := [A . B]])
           (is [[A . B] . C]
               [tree := [A B . C]])
           (is [A . B]
               !A
               [tree := B])))
       !:finish))]
```

The first two patterns reshape nested list structure; the third emits a leaf and advances. Notice that `!A` occurs more than once for one request. The final `!:finish` terminates the application protocol, not the future mechanism itself. The original consumer makes this explicit:

```
[object print-leaves
  (script
   (=> tree
       (temporary [leaf := nil]
                  [leaves := (make-future)])
       [tree2leaves <= tree $ leaves]
       [leaf := (next-value leaves)]
       (do ()
           ((eq leaf :finish))
         [printer <= leaf]
         [leaf := (next-value leaves)]))))]
```

In a fresh session using the rebuilt port at the repository root, add the library directory before loading the unchanged original source. Its `(require 'future)` uses this path, not the example's directory:

```
[*library-directories* :=
  (cons #P"lib/" *library-directories*)]
(inc "examples/misc/future-ex" :use-source t)
[print-leaves <= '((a b) (c (d . e) f) g)]
; Prints A, B, C, D, E, F, G on separate lines.
```

Loading interactively writes generated translations next to the sources. For an isolated run that writes only under `build/`, use the example checks in Appendix A. This book's range producer deliberately removes the tree-walking detail while retaining this reply-stream and sentinel protocol. Once you understand it, return to the original and trace each pattern against `((a b) (c (d . e) f) g)` by hand.

13.1.1 Read the Original in Three Passes

First identify the participants. `print-leaves` coordinates one request, `tree2leaves` produces leaf replies, and `printer` performs output. A future object buffers the replies between producer and consumer. The file's first expression loads that library; the three object definitions then establish the application's protocol.

Second read the consumer. Its `temporary` bindings belong to one handler invocation: `leaf` holds the current value and `leaves` is a fresh future. The dollar send starts the walker without waiting for the entire traversal. Each `next-value` removes one reply or waits until one arrives. The consumer forwards ordinary leaf values to the printer and stops when it sees the keyword `:finish`.

Third read the producer's loop. Its `tree` variable is a working representation of the remaining traversal, not an unchanged input tree. The first two `match` clauses rewrite the remaining work. The third sends one reply and removes the emitted head. Only one matching clause executes on an iteration.

Pattern	How to read it
`[[A] . B]`	The head is a one-element proper list. Remove that extra layer, keeping the outer tail `B`.
`[[A . B] . C]`	The head is a nonempty pair. Make its head and tail the next two pending items, followed by `C`.
`[A . B]`	The outer head is the next leaf. Send `A` and continue with the outer tail `B`.

These patterns overlap. A one-element list is also a pair, so the first clause must precede the second. A nested pair is also an outer head, so the structural clauses precede the emission clause. Uppercase names in the source are binding variables, not literal symbols that the tree must contain.

13.1.2 Trace a Small Tree Completely

Use the data `'((a b) (c . d))`; it exercises both a proper nested list and a dotted pair. The following is a logical trace of the walker, not a claim about how its messages interleave with printing. Unquoted uppercase letters in the table denote the already quoted input symbols, not new variable lookups.

Before iteration	Clause	After iteration	Reply
`((A B) (C . D))`	2	`(A (B) (C . D))`	none
`(A (B) (C . D))`	3	`((B) (C . D))`	`A`
`((B) (C . D))`	1	`(B (C . D))`	none
`(B (C . D))`	3	`((C . D))`	`B`
`((C . D))`	2	`(C D)`	none
`(C D)`	3	`(D)`	`C`
`(D)`	3	`nil`	`D`

On the first iteration, pattern variables are `A = 'a`, `B = '(b)`, and `C = '((c . d))`. The construction `[A B . C]` puts the value of `A` first, the value of `B` second, and uses `C` as the final tail. It therefore preserves the remaining work while exposing the next leaf. When the working list becomes `nil`, the loop exits and the handler sends one more reply: `:finish`.

The original sample's longer input adds further rewrites but follows the same rules. It yields `A B C D E F G`. No recursive call to `tree2leaves` is necessary: the loop and rewrites implement the traversal.

13.1.3 Where Does Completion Travel?

Distinguish three message paths:

1. The top-level request goes to `print-leaves`. Its past-type send does not wait for a return value.
2. Leaf values and the terminal keyword go from `tree2leaves` to the future's reply destination, then to the waiting consumer.
3. The consumer sends each leaf to `printer` without requesting a reply. Printing is a side effect, not the future stream.

The terminal marker tells the consumer that no more leaf values are part of this request. It does not acknowledge completion of every printer handler, and the consumer does not reply to its original sender. Changing the outer invocation to a now-type send would therefore require adding an application completion protocol; merely changing the arrow would not create a reply.

Understanding check. Why is a leaf equal to the keyword `:finish` problematic? The consumer would treat that data value as the terminal marker. The original example assumes its data does not contain that marker. Tagged values such as `[:value x]` solve the ambiguity, as in the later stream exercise. Why not use `all-values` as a completion test? Its buffer snapshot does not say whether the producer has finished.

A boundary of the original algorithm. The loop expects an outer proper list of pending items. An empty nested list can be emitted as a `nil` leaf by the final pattern; it is not automatically skipped as an empty subtree. Read the actual patterns rather than assuming the behavior of a general-purpose tree-flattening function.

13.2 Lazy Evaluation: The Original IF with EF

The first example in `examples/misc/reify-args.abclr2` contrasts an ordinary function with a reflective control form. The function `ef` selects between already evaluated arguments:

```
(defun ef (pred then else)
  (if pred then else))
```

The custom evaluator instead receives representations of expressions. It evaluates the predicate first and then uses `ef` to select which branch representation to execute:

```
[group if-with-ef
 (evaluator
  [object
   (state [primary-eval := (create-primary-eval group)])
   (script
    (=> [:IF [pred then else] Env Id Gid Eval] @ C
        [Eval <= [:do pred Env Id Gid Eval]
              @ [cont result
                 [Eval <= [:do (ef result then else) Env Id Gid Eval]
                       @ C]]])
    (=> any @ R
        [primary-eval <= any @ R]))])]

;; (my-if a b c) is shorthand for (reify-args :IF a b c)
(defmacro my-if (&rest body)
  `(reify-args :IF ,@body))
```

The original sample session calls `[:ef 3]` and `[:my-if 3]`. Both return 2, but the ordinary function prints both *Even* and *Odd*; the reflective form prints only *Odd*. Equal return values alone would miss the evaluation-order distinction. This book's `choice-group` is a teaching extension of this original protocol, using a state marker to make branch selection testable.

Study the two experiments separately. The first occupies code-only lines 1–51; Section 13.3 follows the interrupt experiment. The example checks prepare separate executable excerpts for these projects. After running them, load the first experiment and compare the calls:

```
(inc "build/original-tests/reify-first" :use-source t)
[my-if-test <== [:ef 3]]     ; prints Even and Odd; returns 2
[my-if-test <== [:my-if 3]]  ; prints Odd; returns 2
```

13.3 Interrupting a Reflective Control Form

The second experiment in `examples/misc/reify-args.abclr2` defines `with-interrupt`. Unlike the preceding `my-if`, it customizes an individual object's metaobject rather than the group's evaluator. It illustrates *abandoning* an evaluation and selecting an alternative result. It is not an operating-system interrupt handler, an exception mechanism with stack unwinding, or state rollback.

13.3.1 Read the Application Before the Machinery

The original worker counts, then returns one of two strings:

```
[object test-with-interrupt
 (group with-interrupt)
 (script
  (=> [:run n]
   !(with-interrupt
      (dotimes (i n)
        (format t "~&Count ~D." i))
      "Yeah! I did it!"
      "Time is up.")))]
```

The three arguments are the body, the success expression, and the failure expression. They are passed as compiled representations, not evaluated argument values. The body prints `Count 0.` through `Count n-1.` if allowed to finish. Its ordinary `dotimes` result is not the application's answer: the selected branch supplies the value of `with-interrupt`, and the outer `!` sends that value to the waiting caller.

The original controller starts the worker and later interrupts it:

```
[object test-with-interrupt-2
 (group with-interrupt)
 (script
  (=> [:run n m] @ R
      [test-with-interrupt <= [:run n] @ R]
      (dotimes (i m)
        ;; dummy loop
        )
      [[meta test-with-interrupt] <= [:interrupt]]))]
```

The worker's request is asynchronous with `@ R`, so the worker answers the original caller directly. The controller does not wait for the worker before running its dummy loop. After `m` empty iterations, it addresses `[meta test-with-interrupt]` directly. Sending an ordinary interrupt request to the application would queue it behind the running script; the direct meta-level request can change evaluation policy while that script is active. It has no application reply of its own, and the controller does not execute a second `!`.

The commented historical session demonstrates these two outcomes:

```
[test-with-interrupt-2 <== [:run 10 3]]
; Count 0., Count 1., Count 2.
; Being interrupted... / Context is caught.
; => "Time is up."

[test-with-interrupt-2 <== [:run 10 30]]
; Count 0. through Count 9.
; Interruption is ignored.
; => "Yeah! I did it!"
```

The checked excerpt reproduces those counts and branch results on the current image. The parameter `m` is a workload measured in empty loop iterations, not seconds or a timeout deadline. Do not generalize the exact point of interruption to another evaluator or scheduler. Even diagnostic output order needs care: in the current transcript the late interruption's "ignored" message appears after the check has already received the success result. The worker's reply does not join every outstanding action in the controller.

13.3.2 How the Control Form Reaches the Metaobject

The group's `meta-gen` selects `meta-with-interrupt-gen`. The generator itself belongs to `[group-of group]`, the surrounding group's meta-group. For each application object it constructs a non-reifying evaluator wrapper and a non-reifying metaobject. These machinery objects use direct message handlers, while the application script can use reflective forms. The macro exposes the protocol:

```
(defmacro with-interrupt (&rest forms)
  `(reify-args-to-meta :with-interrupt
     ,@forms))
```

Its expansion sends `[:with-interrupt [Body Success Fail] Env Id Gid Eval] @ C` to the current application's metaobject. `Env` is the captured evaluation environment; `Id` identifies that application's metaobject; `Gid` is its group context; `Eval` is its current evaluator wrapper. `C` receives the result of the control form. In this worker, that continuation leads to the outer `!`; it is not simply the controller's application reply destination `R`.

Read the metaobject's control handlers in this order:

```
(=> [:interrupt]
    (if (eq mode :active)
      (progn (format t "~&Being interrupted...")
             [evaluator <= [:interrupt Me]])
      (format t "~&Interruption is ignored.")))
(=> [:context _]
    (format t "~&Context is caught.")
    (match context-for-recover
      (is [C Env Id Gid Eval]
        [Eval <= [:do fail-form Env Id Gid Eval] @ C])))
(=> [:with-interrupt [Body Success Fail] Env Id Gid Eval] @ C
    [fail-form := fail]
    [context-for-recover := [C Env Id Gid Eval]]
    [Eval <= [:do Body Env Id Gid Eval]
          @ [cont Result
             ;; finished without interruption
             [Eval <= [:do Success Env Id Gid Eval] @ C]]])
```

On entry to `:with-interrupt`, it stores the failure representation and the result/context tuple `[C Env Id Gid Eval]`. It requests evaluation of `Body`; the attached continuation receives `Result`, ignores that body result, and requests evaluation of `Success` with destination `C`. The caller waits for one selected branch, not for all three arguments.

An active metaobject accepts `[:interrupt]` and tells its evaluator wrapper where to report an intercepted request. A dormant metaobject only prints that interruption is ignored. The mode comes from the ordinary message-dispatch lifecycle, not from a timer or a stack of active `with-interrupt` regions.

13.3.3 Capture the Next Request, Then Abandon It

Here is the wrapper inside the original generator:

```
(temporary
 [interruptable-eval
  := [object
      (meta-gen non-reifying-meta)
      (state [interrupted := nil]
             notification-address)
      (script
       (=> [:interrupt R]
           [notification-address := R]
           [interrupted := t])
       (=> any @ R
           (if interrupted
             (progn
               [interrupted := nil]
               [notification-address <= [:context [any . R]]])
             [Eval <= any @ R])))]])
```

The final delimiters also close the enclosing temporary-variable binding. Initially `interrupted` is `nil`. A normal request `any` is forwarded to the underlying group evaluator, retaining its reply destination `R`. Here the capitalized `Eval` in the wrapper's fallback comes from the generator's creation request: it names the underlying evaluator. It is different from the incoming control handler's `Eval`, which names the wrapper through which the application is currently evaluated.

The `[:interrupt R]` handler saves its notification address and sets a flag. It does not immediately stop host Lisp code or return a context. The next ordinary evaluation request consumes the flag. Instead of forwarding that request, the wrapper sends `[:context [any . R]]` to the saved address. The dotted

pair retains the request and its completion destination. Interruption occurs at an evaluation-message boundary; a host operation already executing is not preempted inside its machine instructions.

The metaobject's `[:context _]` handler intentionally ignores that intercepted request. It evaluates `fail-form` using the separately saved `context-for-recover`, with the same `C` as the control form's original result destination. This bypasses the body continuation that would have selected success. The interrupted computation is not resumed later. Contrast this with the newer Express library, which stores a suspended evaluation request and resumes it after the express script has finished (Section 13.16).

Follow an early interruption through five distinct messages:

1. The controller sends the worker `[:run n] @ R`.
2. The worker's reflective form asks its metaobject to evaluate the body and remember a failure alternative.
3. The controller sends that metaobject `[:interrupt]`; the metaobject sends its wrapper `[:interrupt Me]`.
4. The wrapper intercepts a later evaluation request and reports `:context`; the metaobject substitutes failure evaluation at the control form's saved destination `C`.
5. Failure returns through the worker's `!` to `R`. The script's separate execution-completion continuation eventually sends `:end` to its metaobject, advancing the ordinary queue.

The remaining generator handlers reproduce the familiar queue protocol: `:message` enqueues, `:begin` matches and starts one script, and `:end` starts another request or returns to dormant mode. State initialization uses `:do-evlis`, `pairlis`, and `env-gen`, as in the inheritance walkthrough. Extra creation parameters are matched by `Extra` but not used here. The `:evaluator`/`:set-evaluator` handlers expose the evaluator choice; they do not themselves define interruption.

13.3.4 Effects, Scope, and Safe Reproduction

Printed counts remain printed. A supplemental worker using the original generator increments persistent state in its body and returns either `(:SUCCESS count)` or `(:FAILED count)`. Its early interrupt leaves a positive count below the requested 50 increments. A subsequent ordinary state query sees that same count, and a new successful run adds two. Thus branch substitution abandons further body work but does not undo earlier effects or automatically reinitialize the worker.

The implementation has one `fail-form` and one recovery tuple per metaobject, not a stack of nested regions. Use it around one body interval rather than as a general nested cancellation API. There is no timeout clock, cleanup protocol, or guarantee of arbitrary resource release.

For a separate executable version of this experiment, run `make -C tutorial interrupt-check`. Then load the prepared definitions in a fresh ABCL/R2 session:

```
(inc "build/interrupt-tests/interrupt-excerpt" :use-source t)
```

The excerpt contains the interrupt generator, its group, the worker and the controller. Thirteen checks exercise the two controller calls, empty and normal bodies, reuse, and the supplemental state experiment. Sources, translations, inputs and logs stay in `build/interrupt-tests/`.

Check your understanding. Why not use `reify-args` here? The handler is on the application's metaobject, not the group evaluator. Why does the failure path not send the intercepted request back to the wrapper? It substitutes another expression rather than resuming that request. Does `m = 3` mean three seconds? No. Does interruption restore the count? No; prior effects remain. Why do both success and failure use `@ C`? Both must complete the same control expression and resume its caller's remaining evaluation, including the application's reply.

13.4 Inheritance Without a Superclass Declaration

The control-form projects changed which expressions run. The original `examples/misc/inherit.abclr2` now uses the same reflective access to choose behavior from another object. Familiar OOP gives us the question: can an emu reuse a bird's behavior while overriding its flight behavior? ABCL/R2's answer here is an explicit *script-acquisition protocol*, not a class declaration or a built-in superclass search.

13.4.1 Start with the Application, Not the Generator

Load the original file in a fresh session:

```
(inc "examples/misc/inherit" :use-source t)
```

The file defines a macro, a generator, and a group. Loading it does not create bird or emu; enter their definitions from its closing comment:

```
[object bird
 (script
  (=> :has-feather? !t)
  (=> :can-fly? !t))
 (group inherit-group)]

[object emu
 (script
  (=> :can-fly? !nil)
  (=> Any @ Reply from Sender
      (inherit Any Reply Sender bird)))
 (group inherit-group)]
```

Both objects belong to `inherit-group`, which arranges for the custom metaobject generator to be used. The bird answers two questions locally. The emu answers the flight question locally and forwards every other request through `inherit`. The source's sample results are:

```
[bird <== :can-fly?]           ; => T
[emu <== :can-fly?]            ; => NIL
[emu <== :has-feather?]        ; => T
```

`nil` in the second result is a reply containing false, not a missing reply. The local emu handler sends it explicitly.

Check the identities separately from those application results:

```
(eq [den [meta emu]] emu)             ; => T
(eq [meta emu] emu)                   ; => NIL
```

Emu's metaobject is another object, and its denotation leads back to emu. Neither operation is a superclass lookup. These identity checks and the three application replies pass on the current image. Here `[group-of emu]` returns `inherit-group`. The generator exposes its creation argument as `Group` for identity creation, retaining `Group-manager` for execution. MCP, Meta-Meta, Fault-Tolerant, and the Time-Warp application generator use the same convention. This is application ownership, not the group of its metaobject or inherited method source.

For non-reifying metaobjects, the generator assigns its known creation group to the denoted identity before initialization. The host expression `(setf (r2sys::object-data-gid [den Metaobject]) GMgr)` writes that identity's group slot; it does not inspect or expose the NRM state.

For the feather request, `Any` becomes `:has-feather?`. `Reply` holds the caller's reply destination and `Sender` holds the request's sender. These are context fields, not extra elements inside the keyword message. The fallback asks for bird's matching script with that context. It does not first send the ordinary question `:has-feather?` to bird and then copy bird's answer.

13.4.2 The Reflective Boundary

The macro from the Lisp reading guide expands the fallback call into:

```
(reify-args-to-meta :inherit Any Reply Sender bird)
```

`reify-args-to-meta` sends compiled representations of these four argument expressions, with their evaluation context, to emu's metaobject. That is the object controlling emu's script execution. The keyword `:inherit` selects a handler added by the custom generator. The representations are not yet the four application values.

Read the core original handler now:

```
(=> [:inherit Args Env Id Gid Eval] @ C
 [Eval <= [:do-evlis Args Env Id Gid Eval]
  @ [cont [message reply sender inherit-source]
     (match [[meta inherit-source]
             <== [:find-script message reply sender]]
       ;; If a script is found, strat execution of the script.
       (is [bindings scriptbody]
           [Eval
            <= [:do scriptbody [env-gen <== [:new bindings Env]]
                Id Gid Eval]
            @ C])
       (otherwise
        (warn "Inheritance from ~S is failed." inherit-source)
        [C <= nil]))]])
```

The handler proceeds in three stages:

1. `:do-evlis` asks the evaluator to evaluate the argument representations in their original environment. Its continuation receives the list `[message reply sender inherit-source]`.
2. It asks the source's metaobject to find a script matching that message, reply destination, and sender. A successful reply contains pattern bindings and an executable script representation.
3. It extends the current environment with those bindings and asks the evaluator to execute the acquired script in the current execution context, forwarding the meta-level continuation `C`.

`Env` supplies the current bindings; `Id` identifies the execution context used for `Me`; `Gid` supplies its group context; `Eval` names the evaluator. The acquired feather script sends `t` to the application's `Reply`, answering the feather question. Separately, `C` tells the reflective computation that the acquired expression has finished. If no script is found, the failure branch warns and completes through `C` with `nil`. It does not thereby send an application failure reply to `Reply`.

13.4.3 Reconstruct the Metaobject's Ordinary Dispatch

Return to the generator's state and its normal-message handlers:

```
(=> [:new StateSpec LexEnv Scripts Eval GMgr . Extra] @ R
    [object metaobject
     (state [queue := [queue-gen <== :new]]
            [state := nil]
            [scriptSet := scripts]
            [evaluator := Eval]
            [Group-manager := GMgr] [Group := GMgr]
            [mode := ':dormant])
```

The application's initializers are described by `StateSpec`. `LexEnv` names its outer lexical environment and `Scripts` its script list. The group's execution components are `Eval` and `GMgr`. The dotted `Extra` collects additional parameters. The generated metaobject has its *own* state: application queue, application environment, scripts, evaluator, group manager, and mode. The variable `state` is initially `nil`; it is filled by the initialization code, not assumed to be the application's data already.

Read lines 49–71 of the original file with this state machine in mind:

1. `[:message Message Reply Sender]` queues one request. A dormant metaobject marks itself active and schedules `:begin`.
2. `:begin` dequeues the request and finds its first matching script. The evaluator executes it in an environment containing the pattern bindings over the application's state.
3. The execution continuation sends `:end` to the metaobject. That handler starts another queued request, or becomes dormant.

`Me` in these metaobject handlers denotes the metaobject itself. `[den Me]` obtains the application object it describes. The custom `:inherit` handler is added to this machinery; it does not replace the ordinary application inbox by a superclass hierarchy.

13.4.4 Initialization and Reflective Queries

Lines 73–92 expose lookup and update operations such as `:state`, `:scripts`, `:find-script`, and `:set-evaluator`. In particular, the three-field `:find-script` query preserves sender and reply information; the one-field query uses `nil` for those fields. That difference matters for patterns that depend on the sender.

Finally, lines 95–104 evaluate state initializers with `:do-evlis`. `stateSpec-varNames` supplies names and `pairlis` associates them with the resulting values. `env-gen` creates an environment with those bindings and the lexical parent `LexEnv`. The generator sets that environment on the new metaobject and replies with the metaobject to its creation caller. Read initialization last, once you know what its state must support.

Understanding check, with answers. Why does emu's `:can-fly?` not consult bird? Its specific local pattern is first and matches, so the catch-all never runs. Would moving the catch-all first preserve that override? No. Does the example copy bird's state or permanently attach bird's scripts to emu? No: it finds a script for each inherited request and executes it using the current environment and context. It is not a guarantee of the field or superclass semantics of another OOP language. Why is `from Sender` retained? Acquired script matching may depend on who sent the original request, not just its payload.

Reading milestone. You should now be able to locate the application reply, the expression completion, the queue transition, and the creation reply independently in this file. The original baseline checks the three bird/emu queries without changing any original expression. That success does not certify arbitrary scripts, runtime script mutation, or every use of the generator.

13.5 Reading the Built-in Protocol Sketches

Inheritance used an environment, scripts, and a queue. Before reading another metaobject generator, inspect those components in `examples/misc/built-ins.abclr2`. It describes environments, script objects, and queues in familiar object notation. It is a *protocol sketch*, not a replacement runtime to load: angle-bracket placeholders, spelling mistakes such as `Nmae` and `Vlaue`, an extra bracket, and the `:bound` fallback prevent that interpretation. Consult `src/primary-env.lsp` and `src/primary-queue.lsp` for the supplied implementations.

An environment is an object representing variable bindings. Its local frame is an association list: `((n . 2) (limit . 10))` binds `n` to 2. A second environment, `lexEnv`, represents the enclosing lexical scope.

```
(=> [:new alist lexEnv]
    ![object env
      (script
       (=> [:value-of Name] @ R
           (let ((p (assoc Name alist :test #'eq)))
             (if p [R <= (cdr p)]
                 [lexEnv <= [:value-of Name] @ R])))
       (=> [:set Name Value] @ R
           (let ((p (assoc Name alist :test #'eq)))
             (if p
               (progn (setf (cdr p) Value)
                      [R <= Value])
```

Trace a lookup of `limit`: `assoc` finds its pair, and `cdr` supplies 10 to the reply destination. If no local pair exists, the request goes outward, preserving `@ R`. A local binding whose value is `nil` still counts: the test examines the pair, not its value. Assignment by `:set` updates the nearest existing binding; it does not necessarily create a new local binding. The actual implementation also accepts `[:set-local Name Value]`, updating or creating a binding in this frame. The evaluator uses this when naming a new object. `:alist` and `:lexEnv` inspect the frame and its parent. The global environment ends the chain and reports an unbound lookup. The sketch's outer `:bound` request is a typo, not a second protocol: the implementation forwards `:boundp`.

A script object separates two jobs usually hidden by a handler declaration. `[:match Message Reply Sender]` returns either `nil` for failure or `[Bindings ExecutableExpression]` for success. The bindings are an association list; the executable expression may be `[:script ScriptObject]`. The evaluator then asks that object to execute with `[:exec Env Id Gid Eval] @ C`. Here `Env` contains the bindings, `Id` is the current metaobject identity, `Gid` the group manager, `Eval` the evaluator, and `C` the execution continuation. Pattern and body inspection are separate requests, not commands to run the script.

The queue sketch stores arrivals by `cons` at the front and removes the last element. Arrivals A, then B produce `(B A)`, but dequeue returns A first: this is *first in, first out* (FIFO), not a stack. Its `waiters` list similarly retains reply destinations for dequeue requests made while empty. Such a request need not stop the queue from accepting a later enqueue. The built-in queue uses lower-level machinery instead; do not infer that its storage representation is this pair of lists. It additionally provides `:listify`, used by the metaobjects below.

Check your understanding. An inner frame is `((n . nil))`; its parent binds `n` to 7. What does looking up `n` return? *Answer:* `nil`, because the local binding shadows the parent's binding. Would storing A with `cons` make it the next queue item? *Answer:* only if no older item exists; removal is from the opposite end.

13.6 A Script Object with a Sender-sensitive Match

The sketch separated script matching from execution; now follow an executable script object in `examples/misc/user-script.abclr2`. Loading it creates four objects; the installation commands at the end are inside a block comment and must be issued separately. `Monkey` initially replies *Scratch!* to any message. `Monkey-Showman` and `Poor-Audience` each handle `:try` by sending `:play-trick` to Monkey and forwarding their caller's reply destination. Monkey sees different *senders* even though the requests have the same content.

The fourth object, `monkey-script`, implements the script-object contract. It is not Monkey's application handler until installed in Monkey's script set. For `:play-trick`, its match method tests `(eq Sender Monkey-Showman)`. On success, it returns two elements:

```
[ [ ['R . Reply] ] [:script Me] ]
; First element: association list binding R to Reply.
; Second element: expression requesting this script's execution.
```

These are nested lists, not three bindings. `R` is quoted because it is a binding name; `Reply` is evaluated because its value is the destination being bound. Here `Me` is the script object, not Monkey or the Showman. For a different sender the match method replies `nil`. The next catch-all `:match` method also replies `nil`; without it, script search could wait rather than move on to Monkey's original catch-all handler.

```
;; should be returned to the reply destination C when the execution
;; terminates.
(=> [:exec Env Id Gid Eval] @ C
    ;; This script object first looks up the reply destination (which
    ;; is bound to variable R in the given environment).
    ;; Then, it sends a message directly to the reply destination
    ;; without using given Evaluator.
    [Env <= [:value-of 'R]
         @ [cont Reply-dest
                 [Reply-dest <= "Hansei" ]]] ; well-known trick !?
    ;; After the execution specified above, this script object sends
    ;; Nil to the reply destination C.  This message notifies the
    ;; end of the script execution.  Note that C is not the reply
    ;; destination of the message to object Monkey, but the one of
    ;; (metalevel) message to this script object.
    [C <= nil])
```

The execution method asks the environment for `R`. Its continuation sends *Hansei* to that application reply destination. Separately it sends `nil` to `C`, completing the evaluator's request. These destinations have different meanings. There is an ordering boundary: the lookup uses `<=`, not `<==`. The source does not wait for its continuation to deliver *Hansei* before sending completion to `C`. Completion is not a receipt from the application caller.

Try these commands in a fresh session, preferably with a scratch copy:

```
(inc "examples/misc/user-script" :use-source t)
[Monkey-Showman <== :try] ; "Scratch!"
[Poor-Audience <== :try]  ; "Scratch!"
[[meta Monkey] <= [:add-script monkey-script]]
[[meta Monkey] <== :scripts] ; inspect installation
[Monkey-Showman <== :try]
[Poor-Audience <== :try]
```

The final replies are *Hansei* and *Scratch!*. Installation puts the new script first. It handles the Showman's request; its failed match for Audience lets search reach Monkey's original handler. Removing a script

with `[:remove-script message]` removes only the first script matching that message with `nil` reply and sender fields, not every matching script or a script selected by identity. The custom sender test here matters: a removal request with a `nil` sender does not select `monkey-script`. Script-list changes affect later dispatch, not a handler already being evaluated. `:add-script` does not send a reply: changing its arrow to `<==` waits for a reply which the protocol never promises.

Check your understanding. How can the script distinguish Showman and Audience without adding a field to `:play-trick`? *Answer:* matching receives the sender as well as content. Why must a failed match reply? *Answer:* search needs an explicit failure before trying another candidate.

13.7 MCP: Following a Meta-circular Interpreter

We have followed individual environment and script protocols. Now combine them: `examples/misc/MCP.abclr2` expresses a metaobject generator, an evaluator, and a group manager in ABCL/R2 itself. *Meta-circular* means that the interpreter is described using the language it interprets. It does not mean this file removes all dependencies on host Lisp and existing kernel objects. Read it in three passes: the group manager at the bottom, the generated metaobject, and finally the evaluator's expression cases.

The ordinary object `Meta-circular` implements the group-manager protocol. Its state selects `MC-meta-gen` and `MC-eval`. Its `:new` handler delegates to the generator, adding the evaluator and `Me` as group manager. `:replace-meta-gen` and `:replace-evaluator` change these configuration fields, not necessarily every previously created object. `:new-group` delegates to the existing group machinery and sends `:initialize` to the resulting manager.

The generator receives state specifications, lexical environment, scripts, evaluator, and group manager. Extra arguments are warned about and ignored. It creates a metaobject holding queue, state environment, script set, evaluator, manager, and a mode initially `:dormant`. Initializer expressions are evaluated by `:do-evlis`; `pairlis` pairs variable names with the returned values. An environment extending `LexEnv` becomes the state, and the generator replies with the metaobject.

For application traffic, follow this cycle:

```
(=> :begin
    (match [queue <== :deq]
      (is [Message Reply Sender]
          (match (find-script Message Reply Sender scriptSet)
            (is [Bindings ScriptBody]
                [evaluator
                  <= [:do
                      ScriptBody [env-gen <== [:new Bindings state]]
                      Me Group-manager evaluator]
                  @ [cont _
                     [Me <= :end]]]))
```

1. `[:message Message Reply Sender]` enqueues the three fields. If dormant, the metaobject becomes active and sends itself `:begin`.
2. `:begin` dequeues one request, finds its script, and extends the state environment with match bindings.
3. The evaluator executes the body. Its completion continuation sends `:end` to the metaobject.
4. `:end` starts another queued request or returns to dormant mode.

Application replies use `Reply`; advancement of the cycle uses the execution continuation. Reflective handlers expose state, scripts, queue, and evaluator. The one-field `[:find-script Message]` query supplies `nil` for reply and sender; ordinary dispatch uses the actual fields.

In `MC-eval`, `Exp` is a tagged representation, not necessarily original source. `:compiled` invokes a compiled function with the context. `:variable` looks up ordinary names in `Env`; `Me` and `Group` yield `[den Id]` and `Gid`. `:self-evaluate` invokes a supplied function. `:script` invokes script-object execution. An unknown representation warns; this is not an unrestricted Common Lisp evaluator.

The send cases expose the difference between reply and completion:

Expression tag	Completion and delivery
`:send-past`	Sends `nil` to `C`, then forwards the request to the target's metaobject with its separate `Reply` field.
`:send-future`	Sends `nil` to `C`, obtains the future's reply destination, and forwards the request with that destination.
`:send-now`	Uses `C` as the application's reply destination; the target's reply supplies the expression's result.

These describe the non-null-target paths. A now-send to `nil` does not reply in this definition. `:object-def` chooses the specified group or the current one, creates an object, and, if named, installs its denotation through `:set-local`. An explicit generator receives the group's evaluator; otherwise creation goes through the group manager. `:group-def` creates and optionally binds a group manager.

`:do-progn` evaluates expressions sequentially, returning the last result, or `nil` for an empty body. `:do-evlis` evaluates sequentially and reconstructs a list of results. Both use continuations to wait for each expression.

Here is a small teaching probe of the interpreter, not another original example:

```
(inc "examples/misc/MCP" :use-source t)
[object mc-counter
  (group Meta-circular)
  (state [n := 0])
  (script
    (=> :increment [n := (+ n 1)] !n)
    (=> :value !n))]
[mc-counter <== :increment] ; 1
[mc-counter <== :increment] ; 2
[mc-counter <== :value]     ; 2
[[meta mc-counter] <== :evaluator] ; MC-eval
```

All four observations pass on the current image. They establish this creation/state/reply path, not coverage of every evaluator case.

Check your understanding. Why can a past-send complete before the receiver answers? *Answer:* its evaluator sends `nil` to `C` independently of the application reply. A now-send instead lets the receiver answer directly to `C`.

13.8 Meta-meta: Reflecting on the Message Interpreter

`examples/misc/meta-meta.abclr2` takes the queue/dispatch cycle one level higher. Its application object, `twice`, replies with `(+ x x)`. The interesting change is which objects interpret its requests.

Read the four numbered source steps in order. `the-meta-meta-gen` is placed in `(group-of (group-of group))`, and `the-meta-gen` in `(group-of group)`. Each `group-of` selects the group of its argument, climbing the group relationship in this setup; `group` is the current-group pseudo-variable. The second generator creates a metaobject explicitly configured with `(meta-gen the-meta-meta-gen)`. That object's own execution is therefore interpreted by a custom meta-metaobject. The third step creates `a-group` using `the-meta-gen`; the fourth puts `twice` in that group.

Keep three identities distinct:

```
twice                  ; application object
[meta twice]           ; interprets application messages
[meta [meta twice]]    ; interprets interpreter messages
```

For `[twice <== 3]`, the first interpreter receives a `:message` envelope with application content 3. The higher interpreter sees messages involved in executing that interpreter, including its begin/end cycle. The generators retain the familiar state initialization, matching, environment extension, and completion protocol. Compare their repeated bodies with `MC-meta-gen`: logging is in the generated objects' `:message` handlers, not merely in the factories' creation handlers.

The actual labels are *Meta-gen receievs message* and *Meta-meta receievs message*; spelling is the original's. Initialization also produces traffic. Not every log line is another application request, and numeric object identifiers are not stable.

```
(inc "examples/misc/meta-meta" :use-source t)
[twice <== 3] ; 6, with meta-level diagnostic output
```

The commented historical transcript refers to `foo`, although the file defines `twice`, and one logging label differs. Treat it as an old illustration, not the literal session to reproduce. The isolated check confirms 6, both actual logging markers, and distinct identities at all three levels. It does not require a fixed log order or number of lines.

Check your understanding. Does a meta-metaobject make `twice` a subclass of its interpreter? *Answer:* no. These are separate objects with an execution-control relationship, not an inheritance hierarchy.

Reproducing the checks. Run `make -C tutorial reflection-check`. Eighteen checks use distributed copies of `user-script`, `MCP`, and `meta-meta`; Their metaobjects retain distinct application and execution-group bindings. Inputs, translations, and logs stay in `build/reflection-tests/`. An additional 38 checks exercise addition, first-match removal, duplicate scripts, reinstallation, retained state, and queued replies on both primary dispatch paths. They guard script-list/cache consistency, not arbitrary concurrent modification of an executing handler.

13.9 Guarded Scripts: The Original Heavy-Meta Counter

We now return from interpreter construction to a useful change in application dispatch. A structural pattern can bind `n`, but it does not establish that `n` is a positive integer. A script's `where` clause supplies an additional acceptance test, called a *guard*. This is part of script selection, not an `if` inside an already selected body. If the guard is false, the metaobject tries the next script for the same request. The user's guide describes this facility as a library extension [1]; it does not add guards to the ordinary `match` form.

The original example is inside the closing block comment of `lib/heavy-meta.abclr2`, not in a separate example file:

```
[object counter
  ;; (meta-gen heavy-meta) <-- attached automatically
  (state [c := 0])
  (script
   ;; increment positive number only.
   (=> [:add n] where (and (integerp n) (plusp n))
       [c := (+ c n)])
   (=> [:add _])                        ; ignored
   (=> [:value]
       !c)
   (=> [:reset]
       [c := 0]))]
```

Loading the library defines the generator `heavy-meta`, not this commented counter. In a fresh session, load the library before entering the counter definition above. Then replay its original commands:

```
(inc-lib "lib/heavy-meta" :use-source t)
; Enter the original counter definition after loading the library.
[counter <= [:add 123]]
[counter <= [:add -100]]
[counter <== [:value]] ; => 123
```

The two add requests are past-type sends. Neither add script replies; do not replace them by now-type sends expecting an acknowledgement. The value request is now-type and explicitly replies with `c`. The reset request likewise changes state without replying. The automated `guards-check` extracts original lines 133–144 byte-for-byte and replays lines 146–148 separately, leaving the selected code unchanged.

Trace both structural success and guard failure. For `[:add 123]`, the first pattern binds `n = 123`. `integerp` is true and `plusp` is true, so the body adds 123 to `c`. For `[:add -100]`, that same pattern still matches and binds `n = -100`. Only the guard fails. The metaobject then tries `[:add _]`, whose wildcard accepts the argument without binding it. That script has no body and intentionally ignores the request, but its execution still finishes the dispatch cycle.

The `and` evaluates left to right and stops at the first false result. Consequently an argument such as `"7"` fails `integerp` without asking `plusp` to compare a string. Zero is an integer but not positive, so it also reaches the fallback. Lisp's `nil` is false; other values, including zero, are true when used directly as guard results. A guard need not return exactly `t`.

For `[:value]`, neither add pattern matches; their guards are not evaluated at all. The value script has no guard and replies immediately once selected. Ordering still matters: placing the unguarded add fallback first would accept every add request before the guarded script could be considered.

Ensure that the object actually uses heavy-meta. The compiler records the guard separately from the body. In the ordinary counter definition, it selects `heavy-meta` automatically because neither a metaobject generator nor a group is specified. The current `src/translate.lsp` makes that condition explicit. The guide's shorter description of automatic selection should not be read as overriding any custom group or metaobject configuration.

If you specify a group, explicitly select a compatible generator:

```
[group guarded-group]
[object grouped-choice
 (group guarded-group)
 (meta-gen heavy-meta)
 (script
  (=> [:choose n] where (and (integerp n) (plusp n)) !:positive)
  (=> [:choose _] !:rejected))]
```

Loading the library alone does not replace the generator used by every group. A diagnostic object with an explicit ordinary group but no `meta-gen` accepts its first structural match even when its stored guard is `nil`; the ordinary metaobject does not perform this guard protocol. Likewise, the non-reifying object translator warns that a `where` field is ignored. Do not use a non-reifying object as a guarded receiver. If another reflective generator is needed, it must implement compatible guard-aware selection; naming both features does not compose their algorithms automatically.

Read the library's continuation chain. The heavy metaobject retains the familiar ordinary queue, application state environment, scripts, evaluator, group manager, and active/dormant mode. Its `:begin` dequeues one request as `MRS`, a triplet of message, reply destination, and sender. Selection is then a series of asynchronous requests and result continuations:

1. Ask the candidate script `[:match . MRS]`. Structural failure immediately moves on to the remaining scripts.
2. On success, the result contains `[Bindings ScriptBody]`. Ask `env-gen` for a frame containing those bindings, with the object's persistent state as its outer environment.
3. Ask the script for `:where-exp`. A `nil` reply means there is no guard. Otherwise this reply is a compiled expression representation, not its boolean result.
4. Evaluate that representation with `[:do ...]` in the new environment. Its result continuation either starts `ScriptBody` in that same environment or tries the remaining scripts.
5. Attach `[cont _ [Me <= :end]]` to body execution. Completion advances the ordinary queue or restores dormant mode.

The central decision is visible in the original implementation:

```
(if whereExp
  ;; if exists, test the where expression
  [evaluator <= [:do whereExp env Me Group-Manager
                     evaluator]
   @ [cont whereResult
      (if whereResult
       ;; good, evaluate the body
       [evaluator <= [:do scriptBody env Me
                          Group-Manager evaluator]
        @ [cont _ [Me <= :end]]]
       ;; where exp failed, try next script
       [Me <= [:find-Match MRS restScripts]])]]
  ;; no where expression, just start exec.
  [evaluator <= [:do scriptBody env Me Group-Manager
                     evaluator]
   @ [cont _ [Me <= :end]]])]]]])
```

Its two `if` tests answer different questions: whether a guard representation exists, and whether evaluating it accepted the request. An explicitly false guard such as `where nil` still has a non-nil representation; it must be evaluated and rejected, not treated as absent. The completion continuation is also separate from the script's application reply. The empty fallback can finish execution without sending anything to the original caller.

The library first presents a synchronous sketch under `#+COMMENT`, then the active continuation-passing version. Continuation-passing style, abbreviated CPS in its comments, makes each next step a handler of the preceding result. Read the active chain as the implementation; the sketch is explanatory, not an additional code path executed before it.

Which bindings can the guard see? The guard runs after pattern matching and environment creation. It can use pattern variables, persistent state, outer lexical bindings, and the bound reply/sender fields of its script header. The supplemental example checks the last two explicitly:

```
[object guarded-header
 (script
  (=> [:who expected] @ R from S where (and (eq S expected) R)
      !(not (null R)))
  (=> [:who _] !nil))]

[object guard-driver
 (script
  (=> :self ![guarded-header <== [:who Me]])
  (=> :other ![guarded-header <== [:who nil]]))]
```

In the first driver request, `Me` denotes the driver, which is also the sender received as `S`. The guard succeeds and the body confirms that `R` is a non-nil reply destination. The other request passes `nil` as the expected sender, so the guard fails and the fallback replies `nil`.

Body-local `temporary` bindings are different: their initialization belongs to body execution, after guard selection. Do not refer to those bindings as though they were available before the body starts. The parser's header order is pattern, optional `@ R`, optional `from S`, then optional `where expression`.

A failed guard does not undo its effects. The supplemental `guarded-choice` increments `checks` inside its guard before testing the argument against state binding `limit`. Requests for 5, 11, and the symbol `wrong` return `:rejected`, `:accepted`, and `:rejected`, while `checks` becomes 3. Even failed candidates have executed their guards. The example demonstrates the mechanism, not a recommendation to hide side effects in acceptance tests.

```
[object guarded-choice
 (state [limit := 10] [checks := 0])
 (script
  (=> [:choose n]
      where (progn
              [checks := (1+ checks)]
              (and (integerp n) (> n limit)))
      !:accepted)
  (=> [:choose _] !:rejected)
  (=> :checks !checks))]
```

Prefer simple predicates. A guard that waits on an ordinary request to its own receiver can deadlock: that request cannot start while the current guard is still selecting a script. Other objects can also change shared state; guard evaluation is not a multi-object transaction.

Reflective script lookup is not the same as guard-aware dispatch. The heavy metaobject's `[:find-script Message]` query calls `search-script`, which checks structural matches. In the original counter it returns the first add script even for `[:add -1]`. Its `:where-exp` remains available for inspection, but the lookup query does not certify that evaluating that guard would accept the request.

Evidence and exercise. Run `make -C tutorial guards-check`. Twenty-four checks cover the original counter, script/guard inspection, state and sender/reply bindings, failed-guard effects, explicit-group setup and non-reifying restrictions. Translations, extracted sources and transcripts stay in `build/guard-tests/`. The runnable supplemental definitions are in `tutorial/examples/09-guards.abclr2`.

Why does `[:add -100]` not raise an error or block later value queries? Its structural match succeeds, its guard fails, and the empty fallback completes dispatch. Why not put the same test inside the first

body's `if`? That would select the first script regardless of the test; it would not ask dispatch to try the next script. Why must a guard's side effects be considered separately from acceptance? Evaluation has already happened when its false result selects another candidate.

Which Heavy-Meta library should you load? Use `lib/heavy-meta.abclr2` for guarded programs. The distribution also contains `lib/heavy-meta-old.abclr2`, a historical variant whose guard protocol does not match the current translator. It is not needed by the original programs in `examples/` and is not an interchangeable alternative. Both files define `heavy-meta` and provide the feature `heavy-meta`; do not load both into the same session. The old file is historical reference material, not another step in this book's learning path.

13.10 Multicast: One Request, a Structured Result

An ordinary send has one receiver. The original `lib/multicast.abclr2` adds a library-defined operation: send the same message to every object in a target tree, and optionally reconstruct a tree of replies. This is not network multicast, a transaction, or the unsupported brace-delimited send syntax. Its targets are ordinary ABCL/R2 objects in Lisp cons structures. Where guards changed the receiving object's script selection, multicast changes how the sender's evaluator interprets a target. Keep the future reply-stream model and the evaluator context from earlier sections in mind.

13.10.1 Replay the Original Library Example

The library ends with a commented example, rather than a separate file in `examples/`. Its three objects are:

```
[object twice
  (script
   (=> x !(+ x x)))]

[object square
  (script
   (=> x !(* x x)))]

[object identity
  (script
   (=> x !x))]
```

In a fresh session at the repository root, load the library, wrap the top-level evaluator, enter those definitions, and replay the original send:

```
(inc-lib "lib/multicast" :use-source t)
[*eval* := [multicast-eval-gen <== [:new *eval*]]]
; Enter the original twice, square, and identity definitions above.
[[twice square identity] <== 12]
; => (24 144 12)
```

The bracket expression `[twice square identity]` constructs a list of three object values. Each receives 12: `twice` returns 24, `square` returns 144, and `identity` returns 12. The result list uses the targets' positions, not the order in which replies arrive. The same top-level installation is documented in the user's guide [1]. Loading the library alone only defines its machinery; it does not enable list-valued targets in every evaluator.

13.10.2 Follow the Shape, Not Just the Values

Try a target tree containing both objects and literal data:

```
[[twice [square :label] identity nil] <== 12]
; => (24 (144 :LABEL) 12 NIL)
[[twice twice] <== 3]                  ; => (6 6)
```

Only object leaves receive the message. The keyword `:label` and `nil` are retained as data; they are not receivers. Repeating an object creates two target occurrences and two sends, not a deduplicated recipient set. Each occurrence has its own result position.

The host Lisp function `pickup-target-objects` recursively walks `car` and `cdr`. It constructs a fresh cons template with `nil` in every object position, and records each object with a path into that template. For the nested example:

Receiver	Path from the template root	Stored reply
`twice`	`(:car)`	24
`square`	`(:cdr :car :car)`	144
`identity`	`(:cdr :cdr :car)`	12

The template is `(nil (nil :label) nil nil)`. A path is a sequence of operations: `(:cdr :car :car)` means take the outer tail, its first nested list, and that list's first cell. The function builds its target records with `push`, so their order is reversed relative to the traversal. That does not change the reconstructed result. `store-returned-value` follows a path and updates the final `car` or `cdr` with `setf`.

This is a cons-tree walk, not arbitrary container traversal. A dotted pair of object targets also works: `(cons twice square)` with message 3 yields `(6 . 9)`. Arrays are not recursively decomposed. Use finite, acyclic target trees. A receiver that sends `!nil` has replied; a `nil` result is not the same as a missing reply.

13.10.3 Separate Submission from the Aggregate Reply

The evaluator wrapper intercepts two ordinary send representations:

```
(=> [:do [:send-now Targets Message] Env Id Gid Eval] @ C
    (if (consp Targets)
        [multicaster <= [:send Targets Message C]]
        [original-eval <= [:do [:send-now Targets Message]
                               Env Id Gid Eval] @ C]))
(=> [:do [:send-past Targets Message Reply] Env Id Gid Eval] @ C
    (if (consp Targets)
        [multicaster <= [:send Targets Message Reply] @ C]
        [original-eval <= [:do [:send-past Targets Message Reply]
                               Env Id Gid Eval] @ C]))
```

For a nonempty cons target, a now-type request sends the evaluator's continuation `C` as the aggregate reply destination. A past-type request instead supplies its explicit application destination `Reply`, and preserves `C` separately as the submission continuation. Single-object targets and other evaluator requests are forwarded to `original-eval`. Future-type sends and extended send representations are not intercepted by these two patterns; do not assume that every extension composes automatically with multicast.

Read the `multicaster` handler in this order:

1. Decompose the target tree into template and object/path records.
2. Send `nil` to its own continuation `C`, if present. This is submission progress, not a collected application result.
3. Without an aggregate destination, send the payload to each object and collect nothing.
4. Otherwise create one collector, initialize it with the template, target count, and aggregate destination, then send to each object with a position-specific reply receiver.

`partial-value-receiver-gen` creates a `cont` object for each position. An application reply becomes `[:result value position]` to the collector. Notice another forwarding effect: the target's sender binding is `multicaster`, not the original application client. Sender-sensitive scripts must take this into account.

13.10.4 Put a Multicast Client in Its Own Group

The supplemental `tutorial/examples/10-multicast.abclr2` keeps this policy local to a client group instead of replacing the interactive evaluator:

```
[group multicast-group
 (evaluator [multicast-eval-gen
             <== [:new (r2sys::create-primary-eval group)]])]

[object multicast-client
 (group multicast-group)
 (script
  (=> [:now targets value] ![targets <== value])
  (=> [:past targets value reply]
      [targets <= value @ reply]
      !:submitted)
  (=> [:fire targets value]
      [targets <= value]
      !:submitted))]
```

The targets may remain ordinary objects: it is the *sender's* evaluator that interprets a list-valued target. The internal `r2sys::create-primary-eval` helper is the same implementation-specific helper used earlier. In a fresh session, enter the original three object definitions, then load the supplemental file:

```
(inc "tutorial/examples/10-multicast" :use-source t)
[multicast-client <== [:now [twice square] 3]]
; => (6 9)
[multicast-client
 <== [:past [twice square] 3 multicast-inbox]]
; => :SUBMITTED
[multicast-inbox <== :status]           ; => (1 (6 9))
```

The inbox stores an arrival count and the last aggregate. Inspect its short definition in the supplemental file. `:submitted` is the client handler's reply to the interactive caller; `(6 9)` travels separately to the inbox. The submission reply does not acknowledge that every recipient has finished. The checked session observes the aggregate after the finite sends have run; it does not establish an arrival-time guarantee for other schedules.

For an existing group, the original `multicast-installer` accepts `[:install group-manager]` as a past-type request. The manager must support `:replace-evaluator`. Install before creating clients that will use multicast: replacing a group's evaluator does not automatically rewrite every existing object's captured evaluator. The guide's `*eval*` assignment is the explicit top-level alternative.

13.10.5 Know What the Collector Can and Cannot Promise

The essential collector handlers are:

```
(=> [:init RH C] @ R
    [result-holder := RH]
    [result-counter := C]
    [reply-dest := R])
;; one of returned values
(=> [:result value position]
    ;; the value is stored into result-holder
    (store-returned-value result-holder value position)
    [result-counter := (1- result-counter)]
    (if (zerop result-counter)
        [reply-dest <= result-holder])))]))]
```

Initialization stores the template, remaining-result count, and reply destination. Here `C` names the numeric count, not the evaluator continuation used earlier; `R` is the aggregate destination. Each `:result` stores a value and decrements the count. Only the transition to zero sends the aggregate. This assumes *exactly one reply per target occurrence.*

A nonempty target list with no object leaves initializes a count of zero, but initialization sends no aggregate. A silent recipient leaves the count above zero. Neither case has a library timeout or failure result; a now-type caller cannot obtain the missing aggregate. An empty list is `nil`, not a cons, so it does not enter the multicast branch at all. Use a nonempty set of object recipients for collected requests, and make each recipient's one-reply contract explicit.

The collector does not remember which positions have already replied. A controlled probe initializes two positions, supplies two results for the first position, and observes a premature `(9 nil)` aggregate. A later result for the second position mutates that same delivered list to `(9 10)` without a second aggregate message. These are observed protocol limits, not a supported duplicate-delivery or retry mechanism. Do not use a multi-reply stream producer as a one-reply multicast recipient. For fire-and-forget sends, replies are not collected and there is no aggregate completion acknowledgment.

Evidence and exercise. Run `make -C tutorial multicast-check`. Twenty-four checks replay the original commented session and test nested/dotted targets, duplicate occurrences, `nil` replies, sender identity, group setup, explicit reply routing, fire-and-forget effects, and the collector boundaries above. The runner extracts original lines 171–185 and copies the library byte-for-byte; inputs and transcripts stay in `build/multicast-tests/`. No original source or runtime is patched.

Why does `:label` survive in the result? It occupies a literal template position rather than an object position. Why are two instances of `twice` not merged? Positions, not distinct identities, define the requests. As an exercise, add a one-reply negation object to a nested target tree, predict the template and paths, then compare the collected result. Keep the original library unchanged.

13.11 Object Copies Are Not Independent Computations

The original `lib/copy.abclr2` creates another object from an existing object's scripts, state information, and pending messages. For an OOP reader, the important question is not just "is the identity different?" but "which references and pending effects remain shared?" This library also underlies `report` and can move an object into the `watching` group defined by `watch`.

What is shared. Copying preserves local state values but is neither deep nor atomic: new local bindings retain the same referenced data, while outer lexical environments, scripts, and queued reply destinations remain shared. The examples below separate these effects; do not interpret them as independent replay of an active computation.

13.11.1 Read the Original Fibonacci Experiment

The library's final block comment defines a recursive Fibonacci object. It creates a collector for each nontrivial request and sends two requests to itself: both target `Me`, not newly created worker objects. The later Pcontrol project (Section 13.17.2) uses newly created children instead; you do not need that project to follow this one.

```
[object fib
 (script
  (=> 0 !0)
  (=> 1 !1)
  (=> n @ R
      (temporary
       [collector :=
        [object collector
         (state [half-result := nil])
         (script
          (=> result
              (if half-result
                [R <= (+ half-result result)]
                [half-result := result])))]])
      [Me <= (- n 1) @ collector]
      [Me <= (- n 2) @ collector]))]
```

The collector captures `R`, the reply destination of the original request. Its local `half-result` stores the first answer; the next answer is added to it and sent to `R`. The base cases are 0 and 1. Without copying an active computation, `[fib <== 5]` returns 5. The library then proposes this experiment:

```
(let ((x 5) (n 10))
  [fib <= x @ [cont result (format t "~&Fib(~D)=~D" x result)]]
  (dotimes (dummy n)
    ;; do nothing
    )
  [fib-copy := [copier <== [:copy fib]]]
  [[meta fib-copy] <= :begin])
```

To replay it, load `lib/copy` with `inc-lib`, enter the original `fib` definition, then enter this expression. The automated `copy-check` target extracts both source ranges without editing them. The `dotimes` loop is a scheduling opportunity, not a timed delay. The copy can inherit requests already waiting for `fib`; those requests still carry references to the original reply continuations and collectors. Starting the copy can therefore deliver additional answers into the original computation. The original comment warns that the result can seem "funny."

Do not expect one fresh, independent Fibonacci answer from this session. In the checked run the same `Fib(5)=` label appeared several times with values including 4, 5, and 6. Those values are observations, not a specified sequence to reproduce on every schedule. A collector designed for two answers cannot distinguish extra answers caused by duplicated work. A successful ordinary `[fib <== 5]` after the experiment does not certify independent continuation copying.

13.11.2 Make Sharing Visible with a Small Object

The supplemental `tutorial/examples/11-copy.abclr2` puts a counter binding in an *outer lexical environment*, not in an object's local `state` frame:

```
[copy-source := (let ((shared 0))
 [object copy-subject
  (script
   (=> :increment [shared := (1+ shared)] !shared)
   (=> :value !shared)
   (=> [:echo value] !value)
   (=> :sender from S !S))])]
```

The outer assignment retains the object in the top-level binding `copy-source`. The name `copy-subject` inside the `let` is local to that environment; a printed object name is not a guarantee that a top-level variable of that name exists. In a fresh session, try:

```
(inc "tutorial/examples/11-copy" :use-source t)
[clone := [copier <== [:copy copy-source]]]
(eq clone copy-source)                 ; => NIL
[copy-source <== :increment]           ; => 1
[clone <== :value]                     ; => 1
[clone <== :increment]                 ; => 2
[copy-source <== :value]               ; => 2
```

The objects and their metaobjects have distinct identities, but the scripts are reused and the outer environment is retained. Both objects therefore update the same binding `shared`. This is not a deep copy of all reachable Lisp data or of captured application objects. It is also not the proposed `COPY` environment semantics discussed as future work in the historical guide.

13.11.3 Follow the Copier's Construction Protocol

Start with the `:group` handler, which the default copy request also uses:

```
(=> [:copy anObject :group GMgr] @ R
    (temporary [messages := [[meta anObject] <== :queue]]
               [state := [[meta anObject] <== :state]]
               [scripts := [[meta anObject] <== :scripts]])
    (let ((stateSpec (alist->stateSpec [state <== :alist]))
          (lexEnv [state <== :lexEnv]))
      [GMgr
       <= [:new stateSpec lexEnv scripts]
       @ [cont newMeta
          (set-copied-name newMeta [meta anObject])
          [message-setter <= [:set messages newMeta] @ R]]]))
```

It first queries the original metaobject for its queue, state environment, and scripts. It asks that environment for its local association list and outer `:lexEnv`. Then `alist->stateSpec` builds an object creation specification, and the selected group manager receives `[:new stateSpec lexEnv scripts]`. The reply is a *metaobject*. The continuation gives the new object a display name, arranges its pending messages through `message-setter`, and forwards the final object to the copier's caller.

Part	Copy protocol
Object/metaobject identity	Creates new identities; renaming does not redirect existing application references.
Scripts	Passes the existing script objects to the new construction.
Local state	Reconstructs an initializer specification from current values; creates separate bindings but retains referenced data.
Outer lexical environment	Reuses the original environment reference.
Pending messages	Enqueues the original message/reply/sender records on the copy; leaves the original queue in place.
Active execution	Does not clone an executing evaluator context or recursively replace captured continuations and collectors.

These are several reflective requests, not an atomic snapshot protocol. If an application continues to change while they run, do not assume the collected queue and state describe one indivisible instant. Custom metaobjects must support the queried reflection protocol.

How local-state values are preserved. For a binding `(n . 7)`, the helper captures 7 in a zero-argument function and uses that function as the operand of `:self-evaluate`. The primary evaluator calls it with `funcall` to obtain the value. This also preserves a symbol or function as data rather than accidentally calling it. The binding is new, not the referenced value: assigning the copy's numeric slot leaves the source's slot unchanged; mutating a shared list is visible through both. Rebinding that list slot does not rebind its aliases. Initializer order and the outer environment are unchanged.

13.11.4 Choose the Group and Evaluator Deliberately

The library accepts these ordered message patterns. In the table, `obj`, `g`, `mg`, and `ev` stand for evaluated objects:

Request to `copier`	Construction policy
`[:copy obj]`	Original group, using that group's current generator and evaluator.
`[:copy obj :group g]`	Selected group's generator and evaluator.
`[:copy obj :meta-gen mg]`	Selected generator, original object's current evaluator, original group.
`[:copy obj :meta-gen mg :eval ev]`	Selected generator and evaluator, original group.

In particular, the simplest copy does *not* necessarily retain an object-specific evaluator override: it goes through the group's current creation policy. The explicit `:meta-gen` form retrieves the source object's evaluator instead. The tests distinguish these cases by comparing evaluator identities. These are message patterns, not unrestricted Common Lisp keyword arguments; retain the indicated order and combinations.

The macro `(copy name)` assigns the copy back to the binding `name`. It also accepts `:group` or `:meta-gen`, but not both, and has no `:eval` keyword of its own. For example, if `[old-reference := copy-source]` precedes `(copy copy-source)`, `old-reference` still denotes the original object. Rebinding one name does not update all references held by other objects.

13.11.5 Copy Pending Work without Silently Starting It

The original queue installer is short enough to read completely:

```
[object message-setter
 (meta-gen non-reifying-meta)
 (script
  (=> [:set messages newMeta]
      (dolist (message messages)
        [newMeta <= [:enqueue message]])
      ;; [newMeta <= :begin]
      ![den newMeta]))]
```

It sends `:enqueue` for each saved record and replies with `[den newMeta]`, the new application object. The `:begin` send is deliberately commented out. Copying pending work therefore does not start that work; it also does not remove the original requests. When a copy already has a nonempty queue, send `:begin` to its metaobject to start it. Do not send `:begin` blindly to an empty queue. An empty-queue copy can receive ordinary application requests normally. An ordinary query is not a passive queue probe: it can start execution. Inspect the metaobject's `:queue` instead when checking pending work.

The bounded queue exercise stages one pending record explicitly, so it does not depend on interrupting a running computation:

```
[object copy-inbox
 (state [values := nil])
 (script
  (=> :values !(reverse values))
  (=> value [values := (cons value values)]))]
[[meta copy-source]
 <= [:enqueue [[:echo 42] copy-inbox copy-source]]]
[queued-clone := [copier <== [:copy copy-source]]]
[copy-inbox <== :values]                ; => NIL
[[meta queued-clone] <= :begin]
[copy-inbox <== :values]                ; => (42)
[[meta copy-source] <= :begin]
[copy-inbox <== :values]                ; => (42 42)
```

Here `[:enqueue ...]` takes the internal triple `[message reply sender]`, not an ordinary application payload. Both copies retain `copy-inbox` as the reply destination. The original queue still held its request after the clone ran; starting it produces the second 42. Enqueueing this way is a controlled reflective exercise, not the recommended way to submit normal application work. Do not interpret copying as exactly-once processing, migration that deletes the source, or independent replay of an active computation.

Evidence and exercise. Run `make -C tutorial copy-check`. Fifty-two successful probes cover the original Fibonacci session, identities, shared lexical bindings, script reuse, queue copying/startup, alias rebinding, and group/evaluator selection. Twenty-three state checks cover values, reference identity, independent bindings, shared mutation, and both construction routes. Source extracts, copies, inputs, and transcripts stay in `build/copy-tests/`.

Why do both copies increment `shared`? They retain the same outer environment. Why does the inbox receive two replies? Both queues contain a request addressed to it; only the execution start is separate. As an exercise, keep the original and a named copy, then change which binding you query. Predict identity and lexical effects separately. Do not infer deep-copy semantics from separate object identities.

13.12 Watch: Observe Sends without Replacing the Application

The original `lib/watch.abclr2` is a practical debugging aid. It wraps an evaluator, reports message-send requests, then forwards them to the original evaluator. It does not copy the application object when you

use `watch`. In particular, it can observe an ordinary stateful object without invoking the state-copying path discussed in Section 13.11.

The key distinction is *outgoing sends*, not incoming requests. Watching a caller reveals the requests it sends and the replies it sends onward. It does not show every arrival at that caller, every operation inside the receiver, or every host Lisp computation. The separate `report` library observes a different boundary: message arrival and script execution through a replacement metaobject.

13.12.1 Start with One Sender

The supplemental `tutorial/examples/12-watch.abclr2` defines an echo target and a driver with now-, past-, and future-type sends:

```
[object watch-target (script (=> value !value))]

[object watch-driver
 (script
  (=> [:now value] ![watch-target <== value])
  (=> [:past value] [watch-target <= value] !:submitted)
  (=> [:future value]
      (temporary [replies := (make-future)])
      [watch-target <= value $ replies]
      !(next-value replies)))]
```

The file loads `watch` and `future`; the latter is needed for this driver's future request, not for the basic watch facility. Run this in a fresh session:

```
(inc "tutorial/examples/12-watch" :use-source t)
(watch watch-driver)
[watch-driver <== [:now 7]]             ; => 7, plus trace output
(nowatch watch-driver)
[watch-driver <== [:now 8]]             ; => 8, no watch trace
```

These are the original library's public macros. Common Lisp reader case folding makes `noWatch` and `nowatch` the same macro name. The trace contains a send to `watch-target` with value `7` and a send of the reply onward to the driver's caller. The top-level request `[:now 7]` itself was sent by the interactive evaluator, not by the watched driver, so it is not the driver's outgoing request being reported.

The default reporter prints lines shaped like `[target <== payload from sender]`. Object serial numbers are not stable expected output. For a past-type send with a reply destination it also prints `@ reply`. Its historical future display uses `<<=`; that is a trace label, not a replacement for this book's `[target <= payload $ future]` source syntax. Trace lines are asynchronous reports, not acknowledgments that a receiver has finished processing the message.

13.12.2 Read the Evaluator Wrapper

Here is the complete original generator:

```
[object eval/watch-gen
 (meta-gen non-reifying-meta)
 (script
  (=> [:new original-eval :reporter reporter]
      ![object eval/report
        (meta-gen non-reifying-meta)
        (script
         (=> [:do [:send-past . Arg] . Context] @ C
             [reporter <= [:send-past Arg Context C]]
             [original-eval <= [:do [:send-past . Arg] . Context] @ C])
         (=> [:do [:send-now . Arg] . Context] @ C
             [reporter <= [:send-now Arg Context C]]
             [original-eval <= [:do [:send-now . Arg] . Context] @ C])
         (=> [:do [:send-future . Arg] . Context] @ C
             [reporter <= [:send-future Arg Context C]]
             [original-eval <= [:do [:send-future . Arg] . Context] @ C])
         (=> :original-evaluator !original-eval)
         (=> any @ C
             [original-eval <= any @ C]))]))]
```

The dotted patterns bind the rest of the send representation as `Arg`, and the remaining evaluation context as `Context`. For each of the three send types, the wrapper sends the reporter four fields: the type tag, evaluated send arguments, evaluation context, and continuation `C`. It then forwards the original evaluator request with the same continuation. The application reply path is not redirected through the reporter.

For the standard send protocol, `Context` is `(Env Id Gid Eval)`. Its second element is the sender's meta-level identity; `[den Id]` denotes the application sender. This is how the default formatter obtains the displayed sender. The wrapper also answers `:original-evaluator`, which is useful for checking or explicitly undoing one layer of instrumentation.

The catch-all forwards other evaluator requests unchanged. This is not an instruction-by-instruction profiler, and operations performed by host Lisp or a non-reifying script need not cross these evaluator patterns. Observation adds messages and work of its own; do not infer uninstrumented scheduling or timing from a watched run.

13.12.3 Collect Structured Events Instead of Printed Names

The macros select `default-watch-reporter`. To supply another reporter, use the original manager's explicit protocol. The teaching file includes this small recording object:

```
[object watch-recorder
 (state [events := nil])
 (script
  (=> :events !(reverse events))
  (=> :reset [events := nil] !:reset)
  (=> [type args context continuation]
      [events := (cons (list type args context continuation) events)]))]
```

Each event retains the same four fields the wrapper sends. The helper `watch-target-events` filters those records by target identity; it avoids comparing unstable printed names. In a fresh session after loading the teaching file:

```
[watch-manager <= [:watch-object watch-driver watch-recorder]]
[watch-driver <== [:now 7]]
[watch-driver <== [:past 8]]
[watch-driver <== [:future 9]]
[events := (watch-target-events
             [watch-recorder <== :events] watch-target)]
(mapcar #'first events)
; => (:SEND-NOW :SEND-PAST :SEND-FUTURE)
(nowatch watch-driver)
```

There are additional events for future creation, future consumption, and the driver's replies. Filtering for `watch-target` isolates the three intended sends. The driver returns 7, `:submitted`, and 9: the past-type result still acknowledges submission, not target completion.

Manager commands are past-type requests. Their handlers do not promise application replies: do not replace `<=` with `<==` merely to wait for installation. The bounded sessions check observable evaluator identities and finite traces, not a general synchronization protocol. Keep a recording reporter outside the watched sender's evaluator policy, and bound or clear its retained events in longer experiments.

For printed output with filtering, the original `reporter-gen` accepts two host Lisp functions. The predicate decides whether to print; the formatter returns a string. For example:

```
[filtered := [reporter-gen <== [:new
 :pred #'(lambda (type args context cont) (eq type :send-now))
 :format #'(lambda (type args context cont)
             (format nil "WATCH-NOW ~S" (second args)))]]]
[watch-manager <= [:watch-object watch-driver filtered]]
[watch-driver <== [:now :probe]]
(nowatch watch-driver)
```

The formatter sees evaluated arguments, not source text. These functions are host lambdas: do not place ABCL/R2 sends in their bodies or assume they can capture arbitrary surrounding ABCL bindings. Use ordinary Lisp parameters or explicit Lisp closure construction as in the earlier lambda-boundary discussion.

13.12.4 Choose the Scope and Understand Restoration

Public operation	Scope of the change
`(watch obj)` / `(nowatch obj)`	Replaces/restores this object's evaluator through its metaobject. Requires compatible reflective operations.
`(watch-group g)` / `(nowatch-group g)`	Changes the group's evaluator for subsequent object creation; does not rewrite existing objects.
`(watch-toplevel)` / `(nowatch-toplevel)`	Changes/restores `*eval*`, the interactive evaluator; not every object's evaluator.

The manager saves the previous evaluator before installing the wrapper. For an object it sends `[:set-evaluator wrapper]` to the metaobject; for a group it sends `[:replace-evaluator wrapper]` to the group manager. The top-level operation assigns `*eval*`. These are three different places to install the same kind of wrapper.

In a group experiment, create one sender before `watch-group` and another after it. The first retains its original evaluator; the second captures the wrapper. After `nowatch-group`, a third sender gets the restored group policy, but the second still holds its wrapper. Group restoration is not a broadcast repair of all existing objects.

There is also an original, preconfigured `watching` group. An object defined with `(group watching)` uses its tracing evaluator from creation. Alternatively, after loading `copy`, the library suggests `(copy name :group watching)`. That rebinds `name` to a *different object*; it is not equivalent to `(watch name)`. All the queue-sharing and local-state-copy restrictions of Section 13.11 still apply. The checked copy example has no local state entries and does not establish general copying compatibility.

Wrappers obtained from a group's creation policy are not entries in the manager's watched-object list. Thus `(nowatch obj)` alone cannot remove such a preconfigured wrapper. For a known single wrapper, an explicit object-level restoration is:

```
[wrapper := [[meta obj] <== :evaluator]]
[underlying := [wrapper <== :original-evaluator]]
[[meta obj] <= [:set-evaluator underlying]]
```

Use this only when you know that the current evaluator implements this watch-wrapper protocol. It removes one layer from that object, not the group policy. Do not replace an unrelated custom evaluator blindly.

Avoid repeated installation. The original manager does not make `watch` idempotent. Calling it twice adds two wrappers. On disabling the object, it restores the most recent saved evaluator but deletes all saved entries for that object. A second `nowatch` therefore leaves the inner wrapper installed. The tests observe this boundary and explicitly restore the evaluator saved before the experiment. Pair one installation with one restoration; do not rely on repeated object, group, or top-level requests behaving as a balanced instrumentation stack.

Evidence and exercise. Run `make -C tutorial watch-check`. Forty-one probes check now/past/future results and event payloads, sender context, default and filtered reporters, restoration, group creation policy, top-level control, the preconfigured group, stateful watching without copying, an empty-local-state watched copy, and repeated installation boundaries. Libraries are copied unchanged; transcripts and generated files stay in `build/watch-tests/`.

Why watch the driver rather than just the echo target? The driver issues the application requests of interest. Why does turning off the group not silence every existing sender? Each already has an evaluator reference. As an exercise, record only `:send-past` events and explain which are application requests and which are replies forwarded by the driver.

13.13 Report: Observe Arrivals and Script Execution

Where `watch` observes outgoing evaluator send requests, `lib/report.abclr2` observes the receiving object's dispatch cycle. It supplies a replacement metaobject generator, `meta/report-gen`, that reports arrivals, dispatch attempts, and completed script bodies. The public `report` macro copies the application object using this generator and rebinds its name to the copy. This is not an evaluator wrapper installed on the original object.

Read Section 13.11 first. The copy retains references to scripts and the outer lexical environment, but has a new object identity and queue. The original Copy library's state-initializer limitation also applies: do not assume that `report` can instrument arbitrary stateful objects on this image. Our first client deliberately has no local state.

13.13.1 Start with the Public Macros

The supplemental `tutorial/examples/13-report.abclr2` explicitly loads Copy before Report, whose original source requires Copy. Its application and saved alias are:

```
[object report-subject
 (script
  (=> [:echo value] !value)
  (=> [:twice value] !(* 2 value))
  (=> :silent nil))]

[report-original := report-subject]
```

The `:silent` handler intentionally sends no application reply. Start with a request that does reply:

```
(inc "tutorial/examples/13-report" :use-source t)
(report report-subject)
(eq report-subject report-original)       ; => NIL
[report-subject <== [:echo 7]]            ; => 7, plus reports
(noreport report-subject)
(eq report-subject report-original)       ; => T
```

The macro takes a symbol naming an object, not an arbitrary expression. Its three keyword options, `:message`, `:begin`, and `:end`, all default to `t`. For example, `(report report-subject :message nil :end nil)` selects only begin reports. Disabling all three options suppresses printed reports but still creates a copy; it does not make `report` a no-op.

Here is the original macro's construction sequence:

```
`(let* ((,tmp [copier <== [:copy ,objectSymbol :meta-gen meta/report-gen]])
        (,df [[meta ,tmp] <== :reporter])
        (,nf [standard-filter-gen <== [:new ,df :message ,message
                                                :begin ,begin
                                                :end ,end]]))
   (push (cons ,tmp ,objectSymbol) *reporting-objects*)
   [[meta ,tmp] <= [:set-reporter ,nf]]
   [,objectSymbol := ,tmp]))
```

The copier creates a new object with `meta/report-gen`; the macro retrieves its default reporter, wraps it in a filter, saves the pair `(new-object . previous-object)` in `*reporting-objects*`, installs the filter, and finally rebinds the supplied name. Other aliases still denote the previous object. Requests through `report-original` therefore bypass this new reporting metaobject.

13.13.2 Follow One Dispatch Cycle

Compare the original receive and begin handlers:

```
(=> [:message Message Reply Sender]
    [reporter <= [:report Me :message [Message Reply Sender]]]
    [queue <= [:enq [Message Reply Sender]]]
    (when (eq mode ':dormant)
      [mode := ':active]
      [Me <= :begin]))
(=> :begin
    (match [queue <== :deq]
      (is [Message Reply Sender]
          [reporter <= [:report Me :begin [Message Reply Sender]]]
          (match (find-script Message Reply Sender scriptSet)
            (is [Bindings ScriptBody]
                [evaluator
                  <= [:do
                      ScriptBody [env-gen <== [:new Bindings state]]
                      Me Group-manager evaluator]
                  @ [cont _
                     [reporter
                      <= [:report Me :end [Message Reply Sender]]]
                     [Me <= :end]]])
```

The reporting protocol is `[:report metaObject type [Message Reply Sender]]`. Its identity field is the application's *metaobject*; `[den metaObject]` gives the application object. The final triple retains the received payload, application reply destination, and sender. These are not the evaluator's completion continuation.

For a matching request, follow three distinct boundaries:

1. `:message`: the metaobject receives a `:message` envelope and reports it before enqueuing the triple. If dormant, it becomes active and sends itself `:begin`.
2. `:begin`: it dequeues a triple and reports it *before* looking for a matching script. Bindings from a successful match extend the object's state environment for body evaluation.
3. `:end`: the evaluator calls the supplied `cont` after the body finishes. That continuation sends an end report and the metaobject's internal `:end` request. The latter either starts another queued request or makes the object dormant.

Thus a begin report means a dispatch attempt, not proof that a script matched. An end report means that this body returned through evaluator completion, not that all downstream work finished. A script may reply early and continue working; another may finish without replying at all. In our `:silent` case, a past-type send produces all three events although there is no application answer. Do not call it with `<==` and expect the end report to satisfy the waiting caller.

Reports themselves are past-type messages. The event order for one completed request is arrival, begin, end, but reports from different objects may interleave. They add work and cannot establish the timing or scheduling of an uninstrumented run.

13.13.3 Record the Actual Protocol Fields

The teaching file includes a non-reifying recorder. This keeps the diagnostic handler simple and avoids recursively instrumenting it:

```
[object report-recorder
 (meta-gen non-reifying-meta)
 (state [events := nil])
 (script
  (=> [:report identity type triple]
      [events := (cons (list identity type triple) events)])
  (=> :events !(reverse events))
  (=> :reset [events := nil] !:reset))]
```

In a fresh session after loading the file, replace the copy's reporter:

```
(report report-subject :message nil :begin nil :end nil)
[[meta report-subject] <= [:set-reporter report-recorder]]
[report-subject <== [:echo 7]]
[events := [report-recorder <== :events]]
(mapcar #'second events)                ; => (:MESSAGE :BEGIN :END)
(mapcar #'(lambda (e) (first (third e))) events)
; => ((:ECHO 7) (:ECHO 7) (:ECHO 7))
```

Each record is `(metaObject type triple)`. The lambda only transforms its ordinary Lisp argument; it neither captures ABCL variables nor sends messages. Pass context explicitly if a host helper needs it.

`:set-reporter` replaces the *whole* reporter chain. The macro's earlier keyword filter is no longer in that path, so the recorder receives all three event types. To select only end events while retaining structured records, install a new filter:

```
[only-end := [standard-filter-gen <== [:new report-recorder
                        :message nil :begin nil :end t]]]
[[meta report-subject] <= [:set-reporter only-end]]
[report-recorder <== :reset]
[report-subject <== [:echo 8]]
(mapcar #'second [report-recorder <== :events])
; => (:END)
(noreport report-subject)
```

The generator's message requires all three named fields in this order; it is an object protocol, not a Common Lisp keyword-argument function. The filter forwards enabled events unchanged and drops disabled ones. It does not suppress application messages or change script execution. Reporter installation uses `<=`: the setter promises no application reply, so changing it to `<==` would introduce a wait with no answer.

Reading the default report. Prefer structured records when interpreting reply and sender identities. The `default-reporter` receives `[M R S]` and passes those fields to `default-reporting-function` in the same order. Its `from` field prints the sender; `reply=` prints the application reply destination, not the evaluation continuation. Either field can be `nil`. Printed objects and messages are abbreviated by `*print-length*` and `*print-level*`. Do not use the displayed names or labels as a substitute for the recorded triple.

13.13.4 Restoration and the Limits of Instrumentation

`noreport` looks up the current object in `*reporting-objects*`, removes its association, and rebinds the supplied name to the saved object. It does not destroy the reporting copy, remove other aliases, drain its queue, or merge changes back into the previous object. A retained alias to the copy still runs and reports after `noreport`.

Repeated `report` creates successive copies, not multiple reporting layers around the same object. Each copy has its own reporting metaobject; it does not forward dispatch through the previous copy's metaobject. With an intact registry, successive `noreport` calls restore the previous copy and then the original. This differs from Watch's repeated installation limit (Section 13.12). Do not confuse this name history with a general facility for rolling back application effects.

For an active application, copying is not an atomic snapshot. Existing references still target the old object. Pending queue entries preserve their original reply and sender references; Copy installs them with `:enqueue`, not `:message`, so they do not acquire new arrival reports merely by being copied. They also do not start simply because the copy was returned. Follow the explicit `:begin` rules in Section 13.11, and do not manually start an already active metaobject. This book uses an idle, empty-state subject to keep those separate issues out of the first experiment.

Check your understanding. Will `report-original` start reporting when the other name is rebound? *Answer:* No; it retains the old object identity. Does an end report replace `!` for the application caller? *Answer:* No; evaluator completion and application replies have separate destinations. Does `noreport` stop a reporting copy held under another alias? *Answer:* No; it restores one binding, not every reference.

What the checks establish. Fifty-nine bounded probes check identities, alias preservation, event triples, all eight filter combinations, repeated copying/restoration, completion without an application reply, and dispatch state. The formatter checks cover arguments and output for all three event types and `nil` reply/sender fields. Run `make -C tutorial report-check`; copies, inputs and transcripts remain in `build/report-tests/`. These are bounded observations, not certification of arbitrary stateful copying or transparent instrumentation.

13.14 Retry: Repairing a Message Acceptance Condition

Watch and Report observed execution. This project changes dispatch when a request cannot yet be accepted. The question mark in the opening comment of `examples/misc/fault-tolerant.abclr2` is worth keeping in mind. This is a small experiment in reflective recovery, not a distributed fault-tolerance service. Its receiver refuses ordinary requests until it has received an initialization message. The sender's metaobject repairs that condition and retransmits the refused request. No machine crash or lost network packet is involved.

13.14.1 Start with the Application

Read the two application objects before their generator:

```
[object server
  (meta-gen FT-meta-gen)
  (state [amount := nil])
  (script
   (=> [:initialize]
       [amount := 0])
   (=> [:count-up n]
       [amount := (+ amount n)]
       (format t "~&Count is ~D." amount)))]

[object client
  (meta-gen FT-meta-gen)
  (script
   (=> [:start server]
       [server <= [:count-up 10]])
   (=> [:initialize]
       ;; do nothing
       ))]
```

The server owns `amount`, initially `nil`. Its initialization handler resets that value to zero, and `[:count-up n]` adds an amount and prints the resulting count. The client has no application state: `[:start server]` simply sends `[:count-up 10]` asynchronously. Both explicitly select `FT-meta-gen`, so both participate in the recovery protocol. The client's empty initialization handler is still important: its metaobject must accept the client before it can run `:start`.

In a fresh session at the repository root, the file's own entry sequence is:

```
(inc "examples/misc/fault-tolerant" :use-source t)
[client <= [:initialize]]
[client <= [:start server]]
; Prints Count is 10.
```

The server was deliberately not initialized by these commands. A successful count therefore depends on recovery. Do not change `amount`'s initializer to zero: that would conceal the problem the original example demonstrates. For scratch-copy checks that leave the original directory untouched, run `make -C tutorial retry-check`.

13.14.2 Trace the Rejection and Retransmission

Follow one request in six steps. This is a causal explanation, not a promise about the order of unrelated console output.

1. The client's metaobject accepts `[:initialize]`, sets its private acceptance flag, and executes the empty client handler.
2. The accepted `[:start server]` handler sends `[:count-up 10]` to the server. Its sender is the client.
3. The server's metaobject dequeues the request. Its acceptance flag is false, so it does not execute the arithmetic on `nil`.
4. It sends `[:reject Receiver Message Reply Sender]` to the client's metaobject. Here `Receiver` is the server and `Message` is the original `[:count-up 10]`.
5. The client's metaobject sends `[:initialize]` to the server, then forwards the original message envelope to the server's metaobject.
6. The server processes initialization before the retransmitted request. Initialization sets `amount` to zero; the accepted count request then sets it to 10 and prints the count.

The original recovery handler is short:

```
(=> [:reject Receiver Message Reply Sender]
    ;; An initialization request.
    [Receiver <= [:initialize]]
    ;; Message retransmission.
    [[meta Receiver] <= [:message Message Reply Sender]])
```

Notice the two levels of addressing. The initialization goes to the application object `Receiver`. Retransmission goes directly to `[meta Receiver]`, using the interpreter's `:message` envelope. This preserves the old `Reply` and `Sender` fields rather than creating a new application request with the recovering metaobject as sender. In this trace the recovered request was a past-type send, so it has no waiting application reply to deliver.

13.14.3 Read the Acceptance State Machine

The generator constructs the familiar queue, environment, script set, evaluator, group manager, and `:dormant`/`:active` dispatch mode. Its extra `initialized` flag belongs to the *metaobject*; it is not the server's application variable `amount`.

```
(=> [:message Message Reply Sender]
    [queue <= [:enq [Message Reply Sender]]]
    (when (eq mode ':dormant)
      [mode := ':active]
      [Me <= :begin]))
(=> :begin
    (match [queue <== :deq]
      (is [Message Reply Sender]
```

The `:message` handler enqueues a three-field application envelope. If dormant, the metaobject activates itself and schedules `:begin`. That handler removes one envelope and applies the acceptance condition before looking for an application script:

```
(match Message
  (is [:initialize . _]
      [initialized := t]))

;; The following expression tests whether a message
;; is acceptable or not.  In this example, any messages
;; cannot be accepted before initialization.
(if (not initialized)
    (progn
      ;; A rejection message is sent to the metaobject
      ;; of the sender.
      [[meta Sender] ;;?
       <= [:reject [den Me] Message Reply Sender]]
      ;; Process next messages
      [Me <= :end])
```

`[:initialize . _]` matches a list beginning with the keyword `:initialize`; the wildcard ignores its remaining tail. The flag is set *before* the application handler is found or executed. Thus the comment saying that initialization has executed is stronger than the code: the flag establishes that an initialization-shaped message was encountered. It does not validate a successful initialization result.

For rejection, `Me` is the server's metaobject and `[den Me]` recovers the application server it represents. `[meta Sender]` selects the sender's recovery machinery. The rejected request has already been removed

from the queue; `:end` advances dispatch rather than putting it back locally. The sender is responsible for retransmission.

For acceptance, lines 83–90 use `find-script` to obtain bindings and a script body. The evaluator executes that body in an environment extending the application's state. Its completion continuation schedules `:end`. This continuation is an interpreter completion notice, not a client/server acknowledgment. Lines 94–97 either process the next queued request or return to dormant mode. The generator's final `:do-evlis` and `pairlis` sequence builds the initial application environment, as in the earlier inheritance and MCP generators; that construction is separate from the application's `[:initialize]` protocol.

13.14.4 What the Experiment Does and Does Not Guarantee

A second client start prints *Count is 20.*: the server is already accepted, so no recovery reset is needed. An explicit server initialization resets the count to zero; the next client start prints *Count is 10.* Six isolated checks verify these state transitions, the printed counts, and an empty final server queue using the distributed example. Reflective state queries inspect `amount`; they are test probes, not new server API methods.

The example has no retry budget, timeout, persistent log, duplicate detection, or exactly-once delivery mechanism. A sender without the expected `:reject` handler cannot supply this recovery. Multiple rejections can cause multiple initialization messages, each resetting server state. The checked two-object sequence demonstrates one repaired precondition, not correctness for every interleaving or failure.

Check your understanding, with answers. Why initialize the client even though its handler does nothing? Its metaobject also gates application requests; the empty handler satisfies the chosen protocol without adding application work. Why preserve the original sender on retry? Script matching can depend on the sender, and subsequent rejection must reach the original sender's recovery machinery. Would automatically resetting the server after every request be equivalent? No: it would lose the accumulated count. Recovery belongs to the failed acceptance path, not the normal successful path.

13.15 Breakpoints: Inspecting a Suspended Lexical Environment

Retry repaired a precondition and resent a request; here the application deliberately waits while we inspect its current bindings. `examples/misc/break.abclr2` implements an interactive debugger in ABCL/R2 itself. It does not invoke the Common Lisp debugger or install an operating-system interrupt handler. An application calls a macro, waits for a reply, and lets another object inspect its lexical environment before execution continues. This makes reflection useful for a concrete task rather than merely printing an object's representation.

13.15.1 Reproduce the Original Session

Load the debugger before defining code that uses its macro. At the repository root, enter the following. The definition of `foo` comes from the original file's commented session, not from a replacement example.

```
(inc "examples/misc/break" :use-source t)
[object foo
 (state [x := 0])
 (script
  (=> y
      (temporary [z := (+ x y)])
      (abclr2-break)
      !z))]
[foo <== 123]
```

The handler binds `y` to 123 and computes the temporary `z` as `x + y`, initially 123. The breakpoint now

prints an object identifier and an `Inspect(...)` prompt. Object numbers and generated reply-variable names vary between sessions; they are not expected results. At this prompt you are talking to the environment inspector, not the ordinary `ABCL/R2>` reader.

Use the following sequence. For `:set`, the next two inputs answer the `Name =` and `Value =` prompts.

```
:l            ; lists Z in the temporary frame
z             ; prints Z=123
:u
:l            ; lists Y and a generated reply binding
y             ; prints Y=123
:u
:l            ; lists X in the application-state frame
:set
x
10
:d
:d
:set
z
999
:r
0
```

The first `:u` reaches the handler's pattern bindings, and the second reaches the object's state. The two `:d` commands retrace those steps. The resumed handler sends 999, because you changed its current `z`. The value 0 supplied after `:r` is the result of `(abclr2-break)`, which this handler ignores before executing `!z`.

Now send `[foo <== 123]` again. At the new inspector prompt, enter `:r`, then `t`. The handler returns 133: `x` remained 10, but a new invocation computed a new temporary `z = 10 + 123`. This is the original demonstration's distinction between persistent state and an invocation-local binding. The inspector changes existing bindings; it does not automatically recompute an already calculated `z` when you change `x`.

13.15.2 The Inspector's Small Command Language

Input	Effect
`:l`, `:list`	List the names in the selected local frame.
`:a`, `:alist`	Print that frame's association list of names and values.
A variable name	Look up its value, searching outward if necessary; print an unbound notice if lookup fails.
`:u`, `:up`	Select the outer lexical environment, if one exists, and remember the current frame.
`:d`, `:down`	Return to the most recently remembered frame, if there is one.
`:s`, `:set`, `:=`	Read a name and a value, then update or create that binding in the selected local frame.
`:h`, `:help`, `?`	Print the built-in command help.
`:r`	Read a return value, reply to the breakpoint caller, and finish the inspector handler.

These commands are read by Lisp's `read`, but are interpreted by the inspector's `case`, not evaluated as arbitrary Lisp forms. Enter `x`, not `'x`, when asked for a name. Entering `(+ 1 2)` as a value supplies a list; it does not calculate 3. In particular, `:r` is a command followed by a separate value input. It is missing from the original printed help, although the loop's termination test implements it.

Local listing and outward lookup are different operations. You can look up `x` while positioned at the temporary frame even though `:l` there lists only `z`. However, `:set` uses `:set-local`, not the outward-searching `:set` environment operation. Setting `x` in the temporary frame would introduce a local binding there, not update the object's existing state. Navigate to the state frame first when you intend the persistent change shown above.

13.15.3 How the Macro Obtains the Environment

Read the original monitor:

```
[object break-monitor
 (script
  (=> [dummy-object format args] @ R from sender
      (format t "~&Object ~S breaks." sender)
      (when format
        (format t "~&~?" format args))
      [environment-inspector
       <= [[[[meta dummy-object] <== :state] <== :lexenv]
           nil]
       @ R]))]
```

The macro at lines 65–66 expands into a now-type request to this monitor. It creates an anonymous, empty-script *dummy object* at the call site and includes that object, an optional format string, and a list of format arguments in the message. The macro uses backquote, comma, and comma-at as introduced in the Lisp reading guide: it builds code while inserting the caller's forms.

Why create an object that handles no messages? Its state environment retains the lexical environment in which it was created. `[meta dummy-object]` obtains its metaobject; the `:state` query obtains its state environment; and `:lexenv` follows that environment's outer link. That link is the application environment at the breakpoint. It is a live environment object, not a copied map or a snapshot of a machine stack.

The monitor receives the breakpoint request's reply destination as `R` and its application sender as `sender`. It prints that sender, then sends `[environment nil]` to `environment-inspector`, forwarding `@ R`. The initial `nil` is an empty navigation history. The monitor does not manufacture the final return value or wait for a second result: the inspector replies directly to the waiting caller. If a format string is present, `?` applies it to the supplied argument list to print a contextual message.

13.15.4 Navigation, Mutation, and Resume

The inspector explicitly selects `non-reifying-meta`. Its handler is compiled for the non-reifying execution path rather than interpreted by a custom reflective evaluator. It can still send messages to the environment objects it was given. Non-reifying does not mean that the supplied environment has become inaccessible.

```
(=> [environment env-record] @ R
    (temporary command)
    (do ((command :skip
                  (progn (format t "~&Inspect(~S)> " environment)
                         (read))))
        ((eq command :r)
         (progn
           (format t "~&Return value = ")
           [R <= (read)])))
```

The loop begins with the command `:skip`, whose case does nothing. Its step expression prints the prompt and reads the next command. The termination test recognizes `:r`; it reads one more datum and

sends it to R. Until this happens, the application's now-type breakpoint request has not received its reply. The suspended application handler resumes after that reply, not after a local-listing or edit command.

```
  ((:u :up)
   (let ((lexenv [environment <== :lexenv]))
     (when lexenv
       [env-record := [environment . env-record]]
       [environment := lexenv])))
  ((:a :alist)
   (format t "~&~S" [environment <== :alist]))
  ((:l :list)
   (format t "~&~S" (mapcar #'car [environment <== :alist])))
  ((:d :down)
   (when env-record
     [environment := (car env-record)]
     [env-record := (cdr env-record)]))
((:s :set :=)
 (let* ((name
         (progn (format t "~&Name = ") (read)))
        (value
         (progn (format t "~&Value = ") (read))))
   [environment <= [:set-local name value]]))
```

For upward navigation, `env-record` is a stack of previously selected environments, represented as a Lisp list. The bracket expression `[environment . env-record]` constructs a new head/tail pair. Downward navigation retrieves its `car` and removes it with `cdr`. It therefore retraces your own upward moves; it does not search for every inner scope in the program. An upward move with no outer environment or a downward move with empty history does nothing.

The setting branch uses `let*` so the name is read before the value, then sends the local-binding update asynchronously. The original session and isolated checks show those updates visible when the handler resumes. Do not extend that observation to arbitrary debugger clients or scheduler policies without checking their message ordering.

Evidence and scope. Run `make -C tutorial break-check`. Six checks replay the original definition and editing session from unchanged source copies, verify 999 followed by 133, inspect persistent `x`, and confirm that a supplementary handler receives `(+ 1 2)` as raw breakpoint-return data. Transcript assertions also check local listings, variable values, unbound-name handling, help, and an optional formatted message. They do not certify every command alias or concurrent breakpoint arrival. The single inspector reads shared interactive input; this example is not a multi-user debugger, a rollback facility, or a whole-system pause.

Check your understanding, with answers. Why does the second call not return 999? The first call's `z` was temporary; the second creates a fresh binding. Why can its result still depend on the first session? The state binding `x` was changed in the persistent state frame. Does entering `:r` and then 0 force the application to return 0? No. It returns 0 from the breakpoint expression; the handler still decides what application reply to send afterwards.

13.16 The Original Express Clocks

The breakpoint could wait for an inspector's reply. A running clock poses a different problem: how can a new request affect a handler that has not finished? An ordinary object finishes one script before starting the next queued request. This creates a problem for a clock whose `:start` script loops until another request sets `stop`: an ordinary stop request waits behind the very script it must stop. Express messages provide an alternative dispatch policy, not faster transport or operating-system threads.

Read `examples/misc/express-ex.abclr2` with `lib/express.abclr2`; its older companion uses `lib/express-old.abclr2`.

Configuration and the application first. The original group installs both halves of the protocol:

```
[group express
 (meta-gen meta/express-gen)
 (evaluator [eval/express-gen
             <== [:new (r2sys::create-primary-eval group)]])]
```

The evaluator recognizes express sends; the metaobject generator equips receivers to accept them. Both `clock` and `test` belong to this group. A keyword in a send does not by itself change the primary evaluator or the receiver's ordinary queue policy. The top-level only sends an ordinary request to `test`; express sends occur inside that object.

The clock starts with `time = 0` and `stop = nil`. Its `:start` script resets only `stop`, not `time`:

```
(=> :start
    [stop := nil]
    (do () (stop)
      (format t "~&Count up")
      (atomic                   ; count up tiwce
       [time := (1+ time)]
       (dotimes (i 5))          ;dummy loop
       [time := (1+ time)]
       )
      )
    (format t "~&The clock is stopped.")
    !:finished
    )
```

In Common Lisp, `(do () (stop) ...)` has no loop variables and tests `stop` before each iteration. `(dotimes (i 5))` runs five empty iterations; it is neither a delay in seconds nor a clock measurement. Each completed pair adds two. The intermediate value between increments is odd, so excluding express observation of that intermediate state is the point of `atomic`. Without it, an odd sample is permitted, not guaranteed: a particular schedule may still produce only even samples.

```
(=> :peek                               ;express
    (format t "~&Tick of the clock is inspected by an express message.")
    ;; Surrounding ATOMIC form is no longer needed.
    !time)                              ; Even number should be returned.
(=> :stop                               ;express
    (format t "~&The clock got an order to stop.")
    [stop := t]))]
```

The comments calling these scripts "express" do not declare a special kind of script. The *send* chooses the mode; an ordinary `:peek` uses the same matching script. `:peek` replies with the current count. `:stop` changes a binding but does not reply. Stopping is completed when the resumed loop notices that binding and replies `:finished`.

Following the controller's two phases. The controller supplies the orchestration rather than the clock itself:

```
(=> :start @ R
    (format t "~&Test 1:(atomic (incf time) (dotimes (x 5)) (incf time))~
               ~&Only even numbers should be shown.")
    [clock <= :start @ [cont _ [Me <= :start-non-atomic @ R]]]
    (dotimes (i 20)
      ;; Thanks to the atomic form, we will see only even numbers.
      (print [clock <== :peek :express]))
    [clock <= :stop :express])
(=> :start-non-atomic @ R
    (format t  "~&Test 2: withou atomic form~
                ~&Either even and odd numbers would be seen.")
    [clock <= :start-non-atomic @ [cont _ [Me <= :end @ R]]]
    (dotimes (i 20)
      ;; We may see odd numbers as well as even ones.
      (print [clock <== :peek :express]))
    [clock <= :stop :express])
(=> :end !:finished))]
```

Trace the original `[test <== :start]` as follows:

1. `R` is the caller's reply destination. The controller sends `clock` an ordinary asynchronous `:start`, attaching a continuation instead of waiting for the loop to finish.
2. Twenty now-type express `:peek` requests each wait for one count and print it. They are sequential observations by this controller, not twenty simultaneous requests.
3. A past-type express `:stop` requests the state change without waiting for a reply that the stop script never sends.
4. The clock's `:finished` reply invokes the continuation. Its `_` discards that value and sends `Me` a request for the second phase, explicitly retaining `R`.
5. The second phase repeats the experiment without the paired increments' atomic wrapper. The count continues from the first phase. Its completion continuation sends `:end` to the controller.
6. Only `:end`'s `!:finished` answers the original caller.

Thus the immediate controller script finishing, an express peek replying, and the entire two-phase test completing are three different events. Replacing the asynchronous start by a now-type start would prevent this controller from issuing its peeks and stop. Replacing stop by a now-type stop would wait for a nonexistent application reply.

How the newer library interrupts evaluation. The generator creates a dispatcher, a metaobject, and an evaluator front end for each receiver. The dispatcher finds the script and constructs its binding environment over the object's persistent state. The front end forwards ordinary evaluation to the group's evaluator, but can retain the next `[:do . Context]` request instead of executing it. This is cooperative interruption at an evaluator boundary: it cannot stop a host Lisp operation halfway through its machine instructions.

Mode	Intended handling of an arriving express request
`:dormant`	Dispatch it directly; there is no normal script to suspend.
`:active`	Ask the front end to `:grab`; save the normal evaluation request and its completion destination, then dispatch the express script.
`:express`	Queue it; one express script does not preempt another.
`:lock`	Queue it until the atomic region releases the lock.

Normal arrivals during active, express, or locked execution enter the normal queue. At an express script's completion, the intended order is pending express requests, resumption of the saved normal context, and then queued normal requests. The saved context includes the destination `R` of the evaluator request, not necessarily the application caller's reply destination. Resuming it must preserve evaluation's completion path as well as expression and environment.

The `atomic` macro in `src/user.lsp` asks `[meta Me]` for a now-type `:lock`, evaluates its body, sends a past-type `:release`, and returns the body's last value. In the newer library, locking saves the previous mode and sets `:lock`; releasing restores that mode and can start an express request deferred during normal execution. This is object-local protection against this express protocol, not a transaction over all objects. The macro uses no unwind-protection: do not infer automatic release after an error or a non-local exit.

Why the older peek locks too. The July version has no grabbing front end or suspended normal context. It dispatches express scripts without putting them into the normal message queue, while normal evaluation can continue. Its `express-count` tracks outstanding express executions. A separate lock object stores waiting reply destinations and grants them in order. Both a writer and an observer must acquire that same receiver-local lock:

```
(=> :peek                               ;express
    ;;(print "Peek!")
    !(atomic time))                     ; Even number should be returned.
(=> :stop                               ;express
    (print "Stop!")
    [stop := t]))]
```

An unlocked read in that version could run between the two protected increments, because the reader does not participate in the lock protocol. The newer example deliberately removes `atomic` from `:peek`: under its intended suspension policy, the normal writer cannot advance while the express reader executes, and a reader arriving during a locked pair must wait. These are different algorithms, not interchangeable files.

Both libraries define the same public generator names and provide the feature `express`. Compare them in separate fresh sessions, explicitly loading the matching library first. The old example's `require` otherwise names `express`, not `express-old`; its commented sample also still names `express-ex`. Use the actual old filename:

```
; New version, in a fresh session:
(inc-lib "lib/express" :use-source t)
(inc "examples/misc/express-ex" :use-source t)

; Old version, in a different fresh session:
(inc-lib "lib/express-old" :use-source t)
(inc "examples/misc/express-old-ex" :use-source t)
```

Connect the reading to the first experiment. The idle-clock exchange introduced earlier isolates sender configuration, reply forwarding, and the receiver's express dispatch. Read `:start-non-atomic` next: it starts normal clock work, samples the value, sends stop, and waits for the clock's completion reply. Follow the roles of the two reply destinations separately before tracing the atomic phase.

The queue-handling branches dequeue the selected envelope and bind `[Message Reply Sender]` before dispatch. Compare this with the saved execution context: an envelope describes an application request, whereas the saved context describes suspended expression evaluation. The `express-check` sessions provide short examples of both library versions, queue order, and reply routing.

Check your understanding. Why is `:stop` sent asynchronously? Why is `time` not reset between phases? Could the non-atomic phase print only even values? Why does configuring only the clock fail to establish express sending? Answers: stop has no reply; only `stop` is reset; scheduling need not expose an odd intermediate state; and the sender's evaluator must translate the extra send argument into the express meta-level protocol.

13.17 Pcontrol: Scheduling a Recursive Object Computation

The programs in `examples/pcontrol/` move reflection from an individual object's behavior to a group's execution policy. A recursive algorithm can create many objects before earlier work has finished. The example asks whether choosing small pending computations first can reduce this growth. Here, *parallelism control* means controlling the progress of concurrent work, not creating operating-system threads. Read the application first, then follow the machinery that schedules it.

13.17.1 A Checked Session and the Loading Order

Start a fresh interpreter with its working directory set to a scratch copy of `examples/pcontrol/`. Relative names in `start.abclr2` refer to that directory. Loading creates translations and compiling creates compiled files; the experiments also overwrite their own log files. The automated reading check creates this scratch copy for you:

```
make -C tutorial pcontrol-check
```

For an interactive session in such a copy, use:

```
(inc "start" :use-source t)
(inc "fib" :use-source t)
[test-pc <== [:start 4]]
; Prints Fibonacci(4) = (:ANS 3); replies T.
[test-np <== [:start 4]]
; Prints Fibonacci(4) = (:ANS 3); replies T.
```

The printed `(:ANS 3)` is the Fibonacci answer message, whereas `T` is the test object's final reply. These are different protocol results. Use the argument shapes in `README.abclr2` and the actual handlers: some commands in the commented block at the end of `start.abclr2` have obsolete extra arguments. Keep inputs small; the original README warns that these experiments are expensive.

`start.abclr2` compiles both applications but loads only the four machinery files. Their order supplies the dependencies:

1. `para-monitor.abclr2` defines the monitor.
2. `pqueue-eval.abclr2` defines the queue and evaluator factories.
3. `priori-meta.abclr2` defines application metaobjects.
4. `pcontrol-group.abclr2` connects them into a group.

The subsequent `inc "fib"` or `inc "qsort"` chooses the application. Both application files define `test-pc` and `test-np`; loading the second replaces those bindings. Use a fresh session when changing applications.

13.17.2 Fibonacci: Create Children, Then Collect Replies

The convention is `F(1) = F(2) = 1`. The source does not provide a base case for zero or negative inputs. A factory `fib-gen-gen` creates a `fib-gen` in the `pcontrol` group. That generator creates one `fib` object per recursive subproblem, with its problem size as its priority. The child objects inherit the creation context's group.

```
(=> [:ans n]
    (if sub-node-value
        (progn
          [reply <= [:ans (+ sub-node-value n)]]
          [p-monitor <= :object-dead])
        [sub-node-value := n]))
(=> 1
    ![:ans 1]
    [p-monitor <= :object-dead])
(=> 2
    ![:ans 1]
    [p-monitor <= :object-dead])
(=> n @ R
    [reply := R]
    [[fib-gen <== [:new (- n 1)]] <= (- n 1) @ Me]
    [[fib-gen <== [:new (- n 2)]] <= (- n 2) @ Me]))]))]))]
```

The literal patterns `1` and `2` precede the general variable pattern `n`. A base case sends `[:ans 1]` to its request's reply destination, then reports logical completion to the monitor. A larger problem saves its caller's destination in state `reply`. Each nested `fib-gen <== [:new ...]` waits for the newly created child object, not for its Fibonacci result. The following `<=` starts that child asynchronously with `@ Me`, so the child sends its answer back to the parent.

The parent does not block while collecting answers. Its first `[:ans n]` stores the value in `sub-node-value`; its second adds both values and forwards `[:ans sum]` to the saved destination. Because all valid results are positive, `nil` can safely mean no first answer yet. This is a one-computation-per-object protocol: the state is not reset for a second request.

Trace the five application nodes for input 4:

1. The root for 4 saves its caller and creates children for 3 and 2.
2. The node for 3 saves the root's identity and creates children for 2 and 1.
3. Each base node replies `[:ans 1]` to its parent.
4. The node for 3 combines two answers into `[:ans 2]`.
5. The root combines 2 and 1 into `[:ans 3]`.

This is a dependency trace, not a guaranteed temporal ordering. At each join either child may reply first. Addition makes the aggregation independent of that order. The test waits on the root, prints its answer, requests that monitoring stop, and replies `T`. Its `<==` is therefore a completion boundary for the computation, not just for dispatching the first child.

13.17.3 Where the Scheduling Policy Actually Lives

The group definition is short enough to read in full:

```
[group pControl
  (meta-gen priori-meta-gen)
  (evaluator [multi-eval-gen <== [:new primary-eval-gen 4]])
  (script
   (=> [:new StateSpec LexEnv ScriptSet :priority priority] @ C
       [meta-gen <= [:new StateSpec LexEnv ScriptSet evaluator Me
                            priority] @ C]))
  ]
```

The group uses `priori-meta-gen` for each application object's metaobject. Its evaluator is a `multi-eval` created with four primary evaluators. The group forwards an object-creation request, including

`:priority priority`, to its metaobject generator. Four evaluators here are four runtime objects; this declaration does not establish execution on four processors.

There are two queues to distinguish. Each application metaobject in `priori-meta.abclr2` has its own ordinary FIFO message queue. On `:begin`, it selects a script and submits a `[:do ...]` request to the shared evaluator, adding the object's priority after the usual context fields. A continuation sends `:end` back to that metaobject when script evaluation completes. That signal advances the object's queue; it is separate from the script's application reply.

The shared `multi-eval` has a metaobject made by `PQ-meta-gen`. That metaobject queues evaluation requests as `[Priority [Message Reply Sender]]`. An explicitly supplied priority is associated with the requesting object's `Id`; requests without an explicit priority use the recorded association, or zero if none exists. The priority is thus scheduling information in the evaluation context, not an extra field in `[:ans value]`.

Both tests keep this machinery and the `pcontrol` group. They replace the shared evaluator metaobject's queue:

```
[[meta [pcontrol <== :evaluator]]
 <= [:set-queue [pqueue-gen <== [:new #'< :key #'first]]]]
```

The `pc` variant installs a queue sorted by `<` on each entry's `first` element. The `np` variant instead uses `[queue-gen <== :new]`, the ordinary FIFO queue. In the nested expression, `[pcontrol <== :evaluator]` obtains the evaluator object; `meta` obtains its metaobject; and `[:set-queue ...]` changes that metaobject's pending-work policy. This does not change each Fibonacci object's inbox into a priority queue.

```
(=> [:enq item]
    [item-list := (cons item item-list)]
    [item-list := (apply #'sort item-list predicate keys)])
(=> :deq
    !(first item-list)
    [item-list := (rest item-list)])
(=> :empty? !(null item-list)))]))]
```

`cons` inserts an entry into the list and `sort` reorders that list. `:key #'first` tells sorting to compare the priority field. Smaller problem sizes are selected before larger ones among entries pending in this queue. Do not infer a stable order for equal priorities: the source uses `sort`, not a stable tie-breaking policy. Nor is this express interruption: it selects pending requests rather than preempting an already running script.

Finally, read `multi-eval`'s two handlers near the end of `pqueue-eval.abclr2`. One recognizes an object-definition representation carrying `:priority` and forwards creation to the chosen group manager, then arranges the resulting name binding and reply. The catch-all handler forwards other evaluation requests to the next primary evaluator in a cyclic list, preserving their completion destination `C`. This explains how the example changes creation and scheduling without rewriting the Fibonacci application's reply protocol.

13.17.4 Quicksort: A Message-driven State Machine

`qsort.abclr2` uses the same group but a different application. Its opening comment explains why scanning is implemented through self-messages rather than a `do` loop. Read those self-messages as transitions between phases, not as recursive host function calls.

Each sorting object captures `array`, `left`, and `right` from its generator's lexical scope. Its priority is `(- right left)`. For a nontrivial range, `:start` saves its caller's destination, chooses the middle element as `pivot`, initializes indices `i` and `j`, and sends `:scan-left` to itself.

1. `:scan-left` advances `i` until its element is not smaller than the pivot, then requests `:scan-right`.
2. `:scan-right` moves `j` left until its element is not larger than the pivot, then requests `:test-swap`.
3. `:test-swap` exchanges elements if `i <= j`, advances both indices, and either resumes scanning or creates two child sorting objects for the remaining ranges.
4. Each child sends `:fin` to its parent. The parent's `wait` flag records the first completion; the second forwards `:fin` to the parent's saved caller.

`aref` reads an array element; `setf` changes that element in place. The descendants share this array but work on partitioned ranges. The first completion means only that one side is sorted; the second establishes completion of both. A range with `left >= right`, including an empty range, immediately replies `:fin`.

```
          [[qsort-gen <== [:new array left j]]
           <= :start @ Me]
          [[qsort-gen <== [:new array i right]]
           <= :start @ Me])))
(=> :fin
    (if wait
        [wait := nil]
        (progn [reply <= :fin]
               [p-monitor <= :object-dead])))))]))))]
```

For `(3 1 3 2)`, the root covers indices 0 through 3 and chooses the element at index 1, namely 1, as its pivot. The left scan stops at index 0. The right scan moves from 3 to 1. The swap produces `(1 3 3 2)`, with `i = 1` and `j = 0`. Scanning ends, so the children cover the already trivial range 0–0 and the remaining range 1–3. Their completions join at the root; the final array is `(1 2 3 3)`.

In a fresh session with the same setup, a deterministic test avoids the random initialization object:

```
(inc "start" :use-source t)
(inc "qsort" :use-source t)
[input-array := (make-array 4 :initial-contents '(3 1 3 2))]
(coerce [test-pc <== [:start input-array 4]] 'list)
; => (1 2 3 3)
(coerce [test-np <== [:start input-array 4]] 'list)
; => (1 2 3 3)
(coerce input-array 'list)
; => (3 1 3 2)
```

`coerce ... 'list` makes an array's contents convenient to inspect. Both test handlers first copy their input with `make-array :initial-contents`; they return the sorted copy, not the caller's modified array. The original random entry point `[init <= N]` instead stores an array in the outer binding `a`; follow its `sample-gen` self-message loop to see how it fills each index.

13.17.5 Reading the Monitor Without Overinterpreting It

`para-monitor.abclr2` records two columns: a logical `clock` and a counter. The evaluator sends `:tick` when handling an evaluation request; this clock is not elapsed wall-clock time. Application metaobjects announce creation and transitions between dormant and active dispatch. The application explicitly sends `:object-dead` at logical completion. Nothing in that message destroys a Lisp object or measures garbage collection.

The active count tracks announced dispatch activity, not the number of physical CPUs busy. A node waiting for child answers can be dormant yet still represent unfinished application work. The object

count combines creation announcements with explicit logical-death announcements. For example, generator objects are also created through the machinery. It is not simply a count of the recursive leaves, allocated bytes, or all reachable host objects. The `:record` messages are asynchronous observations of these counters.

For Fibonacci input 4, the actual source names the files `fib-active-pc-4.log`, `fib-object-pc-4.log`, and the corresponding `np` files; quicksort uses a `qsort-` prefix. Repeating the same run supersedes those files. The directory README's generic names omit these application prefixes. Compare traces as experiment output, not as proof that one policy always reduces memory or is faster.

Check your understanding. Does `test-np` remove priority-bearing metaobjects? *Answer:* No. It replaces the shared evaluator's work queue with a FIFO queue while retaining the group and application metaobjects. Why can a parent accept either child's answer first? *Answer:* Its state records the first answer and combines it with the second; Fibonacci uses addition, while quicksort counts two completion signals for disjoint subranges.

Reproducing the reading checks. `make -C tutorial pcontrol-check` runs twelve checks in fresh sessions using copies of the distributed sources. They check priority-queue ordering for distinct priorities, both Fibonacci policies at 4 and 6, both quicksort policies on a duplicate-containing input, and preservation of the caller's array. Translations, monitor files, inputs, and transcripts stay in `build/pcontrol-tests/`. These results establish the small examples' values and completion, not general performance, queue fairness, or faithful historical parallel execution.

13.18 Time Warp: Virtual Time and Recovering from Late Events

Pcontrol changed the order in which pending computations run. Here we also give application events simulated times and recover when their arrival order disagrees with that history. The `examples/time-warp/` directory implements a reflective simulation experiment. A *discrete event simulation* advances by processing events such as a car arriving or a washer becoming available. Each event has a simulated time. Processing an event at time 10 does not require waiting ten real seconds.

Time Warp is an *optimistic* approach: an object processes available events without first proving that no earlier event will arrive later. If one does arrive, the object must restore an earlier state and redo work. This is *rollback*, not an ordinary error retry: the computation may have been correct for the events known then, but its history was incomplete. Reflection lets this example put time and recovery policy in the execution machinery rather than in every application handler.

13.18.1 Local Time, Receive Time, and Three Queues

`LVT`, the local virtual time, belongs to a simulation object's metaobject. The custom evaluator makes that value available as the pseudo-variable `LVT` in its scripts. A timed send has the form:

```
[target <= message @ reply :vrt receive-time]
```

The evaluator sends the target metaobject a six-field envelope:

```
[:message Message Reply Sender VST VRT]
```

`VST` is the sender's virtual send time; `VRT` is the intended virtual receive time. Neither is a wall-clock timestamp. The application still matches `Message`, not this entire envelope. The explicit reply destination remains a separate field.

Distinguish three queues. A simulation metaobject has an ordinary message queue and an undoable, time-ordered event queue. Separately, the evaluator's own metaobject has a queue of evaluation requests. Lowest-LVT-first versus FIFO changes this third queue, not the timestamp semantics of the

event queue. An ordinary send without `:vrt` goes through the ordinary message path; the source does not automatically timestamp every send. For example, the washer's self-messages advance its work loop without advancing virtual time.

13.18.2 Run the Original Car-Wash Experiment

As with pcontrol, start in a scratch copy of the example directory: `tw-load.abclr2` uses relative file names and creates translations and compiled files. The automated check prepares the copy:

```
make -C tutorial timewarp-check
```

In a fresh interpreter whose working directory is that copy:

```
(inc "tw-load" :use-source t)
[starter <== [:start [3 4 6] 3]]
; Runs Lowest LVT first, then FIFO.
; Prints each run's RESULT and counters.
; Replies (:PRIORI (:ROLLBACK ...) (:EXPS ...)
;          :FIFO   (:ROLLBACK ...) (:EXPS ...)).
```

`tw-load.abclr2` first compiles ten files, then incorporates those same ten files in the same order. Compilation alone does not run their object definitions (Section 9). Read the loading sequence as four stages, rather than opening an arbitrary helper first:

Files, in loader order	What the stage supplies
`tw-undo-env`, `tw-undoable-pqueue`, `tw-output-history`	State frames, replayable input, and speculative-output history for recovery.
`tw-pqueue`, `tw-monitor`	Evaluator work queues and the rollback counter, distinct from the replayable input queue.
`tw-meta`, `tw-meta-eval`, `tw-eval`	Application dispatch/recovery, scheduling of evaluator requests, and expression evaluation with time/output context.
`tw-group`, `tw-wash1`	Connect the machinery, then define the timed car-wash application and its controllers.

The names in this table omit `.abclr2`, just as the loader does. It does not load the earlier `tw-wash0`, the independent `tw-test`, or the unfinished `tw-scheduler` fragment.

The service times are 3, 4, and 6 simulated units; the last argument requests three cars. `starter` compares scheduling policies, whereas `starter-1` performs one simulation. Its factories create a new attendant and new washers for each run. The returned report contains counters, not the car assignment list. That list is printed as `RESULT=(:FIN (...))`. Do not assert particular expression counts as a language rule.

`tw-wash0.abclr2` is an earlier, ordinary-message version: two queues pair waiting cars with waiting washers, and a two-signal join completes the starter's request. Its own comment says it has no rollback or Time Warp; its `service-time` argument is not used to model a timed completion. The loader instead selects `tw-wash1.abclr2`, which adds timed events and explicit simulation completion.

13.18.3 Follow Three Cars Through the Application

`washers-gen` numbers the washers from zero. It initially announces washer 0 at virtual time 0, washer 1 at time 1, and washer 2 at time 2. `starter-1` injects car arrivals at times 1, 2, and 3, followed by an arrival sentinel `:fin` at time 4. These initial injections go directly to the attendant's metaobject with `:VRT`; they are not ordinary application messages to the attendant.

The attendant stores pairs `[time car]` and `[time washer]`. When both lists are nonempty, it removes their first entries and starts washing at the maximum of those two times:

```
[(second time+washer)
 <= [:wash (second time+car)]
 :VRT (max (first time+washer)
           (first time+car))]
[job-record
 := (cons [(second time+washer)
           (second time+car)
           (max (first time+washer)
                (first time+car))]
```

This maximum prevents service before either resource is available. `job-record` stores washer, car, and *start* time, not finish time. Each washer executes a short self-message loop to supply runtime work, then announces its next availability at `(+ service-time LVT)`:

```
(=> [:loop 0 car]
    [attendant
     <= [:available Me] :VRT (+ service-time LVT)]
    (format t "~&***Washer(~D) ~S finished washing ~D at ~D"
            Me id car (+ service-time LVT)))
```

For the checked small run, the assignments are:

Washer	Car	Start	Available again
0	1	1	4
1	2	2	6
2	3	3	9

This is a simulated-time trace, not the order in which console lines must appear. `job-record` uses `cons`, so the printed assignment list is in reverse recording order. After the sentinel, the attendant sets `finished` and waits until all washers are available before sending its result to `starter-1`. The latter joins that result with its own end-of-injection signal using `wait-flag`. The final reply therefore does not mean merely that all arrival messages were sent.

13.18.4 What Happens When an Earlier Event Arrives Late?

Open `tw-meta.abclr2`. The ordinary path resembles the metaobjects read earlier: enqueue, find a script, evaluate it, and continue on `:end`. The timed path uses `:begin-tw`. For a positive event with `VRT >= LVT`, it saves the current state frame, sets `LVT` to `VRT`, and evaluates the selected script with output-history and time added to the context.

Suppose an object has processed time 10 and then receives an event for time 5. Continuing at 10 would violate the intended event history. The other branch requests coordinated recovery:

```
[monitor <= :rollback]
[LVT := [state <== [:rollback-to VRT]]]
(format t "~&~S back to ~D by ~S~%"
        [den Me] LVT
        [Type Message Reply Sender VST VRT])
[pqueue <= [:rollback-to VRT]]
[[anti-messenger-gen <== [:new Me]]
 <== [output-history <== [:history-since VRT]]]
[Me <= :end])))))
```

1. The undoable environment restores a saved frame from before the offending receive time and returns its saved time.

2. The input queue rewinds previously dequeued events at or after the cutoff so they can be considered again.
3. The output history supplies sends made at or after that cutoff. `anti-messenger` sends matching cancellation envelopes to their destinations.
4. Dispatch resumes, allowing the revised history to be processed.

An *anti-message* is a cancellation record, not an application request to undo a particular business operation. The queue can remove a matching positive event that has not yet executed. Cancellation of already processed work can require the receiving object to roll back, which can propagate recovery further. The original queue compares message contents, reply, sender, and times; it does not introduce a separate globally unique event identifier. Read these as the experiment's concrete protocols, not a complete general-purpose distributed simulation framework.

13.18.5 Read the Three Histories Separately

In `tw-undo-env.abclr2`, `:push` saves a frame and copies its association-list pairs before later binding updates. `backFrame` selects a saved time strictly less than the rollback cutoff. For a component trace, start with `x = 0`, save at 0, set `x = 5`, save at 10, and set `x = 8`. Rollback to 5 restores the frame at 0 and `x = 0`. It does not interpolate a value between snapshots.

This is a shallow copy of binding pairs, not a deep copy of arbitrary Lisp objects stored in them. Updating a binding through `:set` is different from mutating an array or cons cell reachable through that binding. Updates delegated to an outer lexical environment also do not become part of the local frame's saved history. Do not assume arbitrary host side effects are reversible.

`tw-undoable-pqueue.abclr2` keeps pending events in `queue` and dequeued events in `old`. After processing events at times 2 and 4, rollback to 3 makes the event at 4 available again while retaining the earlier history. It also contains positive/anti-message cancellation and sorting helpers. Some alternative enqueue code is guarded by `#+COMMENT`; read the active handlers rather than combining all printed alternatives.

`tw-output-history.abclr2` records the target, message, reply destination, send time, and receive time. Its cutoff compares the record's *send time*, the fourth field, not its receive time. For sends at times 1 and 5, a cutoff of 4 extracts the second send even if its receive time is 20. Extraction removes those records from the retained history. Restoring bindings alone would leave such speculative output effects uncorrected.

13.18.6 A Complete Two-Object Rollback Cascade

The original three-car run does not force recovery. To see the original machinery actually restore state, replay inputs, and cancel an output, use the supplemental application in `tutorial/examples/08-timewarp-rollback.abclr2`. It introduces no new simulation algorithm: the unchanged original loader, metaobjects, evaluator, environments, and history queues do the recovery. The new application merely makes their effects easy to inspect.

The sink stores the latest received value and a reverse-order trace:

```
[object rollback-sink
 (group TW-group)
 (state [value := 0] [trace := nil])
 (script
  (=> [:value n]
   [value := n]
   [trace := (cons (list LVT n) trace)]))]
```

The source adds each input to its total and sends that total to the sink one simulated unit later:

```
[object rollback-source
 (group TW-group)
 (state [total := 0])
 (script
  (=> [:add n]
   [total := (+ total n)]
   [rollback-sink <= [:value total] :vrt (+ LVT 1)]))]
```

Neither application script contains an undo operation. Assignment goes through the original environment protocol; the timed send is recorded by the original evaluator. The trace uses `cons` to construct a new list and rebinds `trace`, rather than destructively modifying a list shared with earlier snapshots. That distinction matters for shallow binding-history restoration.

Arrange a late event deliberately. Run `make -C tutorial timewarp-check`; the runner copies the original directory and this supplemental file into `build/timewarp-tests/`. For an interactive reproduction, start a fresh interpreter in that prepared directory and load `(inc "08-timewarp-rollback" :use-source t)`. The example loads `tw-load` and resets the rollback monitor before creating the two objects. It initially uses Lowest-LVT-first scheduling.

Inject a time-10 event, let the finite computation settle, and only then inject a time-5 event:

```
[[meta rollback-source] <= [:message [:add 10] nil nil 0 10]]
; Source total becomes 10 at LVT 10.
; Sink receives (:VALUE 10) at LVT 11.

[[meta rollback-source] <= [:message [:add 1] nil nil 0 5]]
; Source rolls back; its earlier output is cancelled at the sink.
; Final source total = 11, sink value = 11.
```

These are deliberate meta-level event injections, not ordinary application requests. The two `nil` fields are the initial reply destination and sender; the final numbers are `VST = 0` and `VRT = 10` or 5. No application reply is requested. The source's subsequent timed sends name the source object as sender automatically. The current interactive top-level drains runnable work after each entered form. The checks exploit that boundary to ensure the first output has *already executed*, rather than merely waiting in the sink's queue. This staged arrival order is a test arrangement, not a promise that arbitrary concurrent clients see the same schedule.

Follow the histories, not just the final sum. After the first injection, the source has saved its initial frame at time 0, set `LVT = 10`, and changed `total` from 0 to 10. Its output history contains `[rollback-sink [:value 10] nil 10 11]`. The sink has likewise saved its initial frame at 0 before processing time 11. Its value is 10 and its trace is `((11 10))`.

Now the late positive event has `VRT = 5 < LVT = 10`. The source's `:begin-tw` handler takes the recovery branch. It restores the saved frame at 0, so both its local time and total return to zero. Its input queue has already dequeued the offending time-5 event and the previous time-10 event. Rewinding to 5 makes both available for replay, sorted by receive time. It must not discard the late event after using it to trigger recovery.

The source extracts outputs whose send time is at least 5. The old `[:value 10]` send qualifies because its send time was 10; its receive time 11 is preserved in the cancellation envelope. Here is the original anti-messenger's active handler:

```
(=> nil !:fin)
(=> (bracket* [Target Message Reply VST VRT]
              rest-history) @ R
    (format t "~&Cancelling ~S" [Target Message Reply VST VRT])
    [[meta Target]
     <= [:anti-message Message Reply [den TW-meta]
                       VST VRT]]
    [Me <= rest-history @ R]))]))]
```

The first pattern ends recursion on an empty history with `:fin`. The second uses `bracket*` to separate one output record from the remaining history. It sends directly to the target's metaobject, changing the envelope's tag to `:anti-message`, but preserving message, reply, sender identity, and both timestamps. Then it processes the remaining records with the same `R`.

The source waits for the anti-messenger's completion before sending itself `:end`. That completion means the cancellation requests were issued, not that every receiving simulation object has completed its own rollback. The target sends are asynchronous. In this checked staged run, the sink receives:

```
[:anti-message [:value 10] nil rollback-source 10 11]
```

Because the positive event has already executed, it is in the sink's `old` input history, not its pending queue. The anti-message therefore cannot simply remove a pending positive event. The sink dequeues the anti-message and rolls back to a frame strictly before 11: its initial frame at 0. This restores `value = 0` and `trace = nil`. Rewinding the input history brings the old positive and its anti-message together; `remove-anti` removes that pair, so the invalid `[:value 10]` event is not replayed. The sink has no timed outputs of its own to cancel in this application.

The source then processes its revised inputs at 5 and 10. At 5 it adds 1 to zero and sends value 1 for receive time 6. At 10 it adds 10 to that corrected state and sends value 11 for receive time 11. These are new outputs of replayed work, not the old output being retained under another name. The sink's revised history is therefore:

Receive time	Application event	Resulting sink value
6	`[:value 1]`	1
11	`[:value 11]`	11

The final reverse trace is `((11 11) (6 1))`; the speculative `(11 10)` entry has disappeared. The source's final local time is 10, the sink's is 11, and the shared monitor records two rollbacks: one caused by a late positive input, one caused by cancellation of already processed output. Console cancellation and rollback lines remain visible: restoring simulation bindings does not erase printing.

Inspect without injecting another application event. The original `TW-meta` does not implement an application-state query. Its own ordinary metaobject can expose the simulation machinery's bindings. For example:

```
[source-meta-state := [[meta [meta rollback-source]] <== :state]]
[source-state := [source-meta-state <== [:value-of 'state]]]
[source-state <== [:value-of 'total]] ; => 11
[source-meta-state <== [:value-of 'LVT]] ; => 10
```

The inner `meta` selects the simulation object's interpreter; the outer selects that interpreter's ordinary metaobject. Its `:state` reply is an environment whose binding named `state` points to the undoable *application* environment. `LVT`, `pqueue`, and `output-history` instead belong to the simulation metaobject's own frame. Do not confuse these two state environments or treat the returned objects as copied snapshots.

Evidence and limits. Twelve assertions run for this cascade with each evaluator scheduler, in separate fresh processes: Lowest-LVT-first and FIFO. They check the speculative values and output record, corrected values and trace, both final local times, the replacement output history, the replayed source input history, the sink's retained corrected inputs, empty pending event queues, and the two-rollback counter. Transcript assertions require the cancellation and both restoration reports. The finite result is the same under these two checked arrangements; the test is not a performance comparison.

This closes the gap between isolated history tests and a concrete metaobject/evaluator recovery cascade. It does not certify every cancellation arrival order, simultaneous or duplicate event, zero-time rollback boundary, or timestamped now-type send. In particular, output-history matching is based on the original event fields rather than unique event identifiers. The same original `remove-anti` helper compares envelope tails, not the tags; do not infer a general duplicate-delivery policy from this one pair. The experimental NR/NS evaluator variants remain separate cases.

Check your understanding. Why is restoring only the source's total insufficient? Its speculative output has already changed the sink. Why is the final value 11 rather than 1? The time-10 input is replayed after the new time-5 input. Why does the sink not end with both time-11 values in its trace? The old input is cancelled during rewind and its earlier state effect is rolled back. Does waiting for the anti-messenger join the entire cascade? No; it joins cancellation issuance, not all recipient work.

13.18.7 The Evaluator and a Source Reading Route

`tw-group.abclr2` connects `TW-meta-gen` and `TW-eval-gen`. Scheduler replacement transfers the old evaluator queue's entries to the new queue before installing it. The queues in `tw-pqueue.abclr2` support `:enq`, `:deq`, `:listify`, and `[:set ...]` for that purpose. The priority variant sorts by an entry's first field; the normal variant appends entries for FIFO order.

`tw-meta-eval.abclr2` wraps evaluation requests as `[LVT Message Reply Sender]` when the message has the extended eight-field context; other requests receive key zero. Its counter counts incoming evaluation requests, not cars, virtual time units, or elapsed execution time. `tw-monitor.abclr2` separately counts rollback announcements.

Read the first evaluator definition in `tw-eval.abclr2` before its variants. It recognizes compiled expressions, variable lookup, sends, object/group creation, sequential execution, and argument-list evaluation. The added `Outputs` and `LVT` fields carry the simulation context. A cache indexed by `Id` supplies those fields to subsequent requests lacking them. Timed past-type sends both construct a timestamped envelope and record the output; ordinary sends use their separate path.

The same file also defines non-reifying and no-scheduling variants, selected by the experimental `NR` and `NS` controllers in `tw-wash1.abclr2`. They are not aliases for the two policies compared by `starter`; Section 13.18.8 explains their separate setup and evaluator architecture. Timestamped now-type sends need separate validation; the car-wash application uses timed past-type sends.

`tw-test.abclr2` supplies independent countdown objects `foo`, `bar`, and `baz`. Their timed self-sends increase virtual receive time by 1, 2, and 3 respectively. Their `start` object's handler dispatches the three requests; it has no final application reply. Do not call it with `<==` and assume it is a completion join. `tw-scheduler.abclr2` is an unfinished object-definition fragment and is not loaded by `tw-load`. The operative scheduler objects come from `tw-pqueue`, not that fragment.

13.18.8 NR and NS: Replacing the Evaluator, Not Just Its Queue

The original `NR` and `NS` controllers in `tw-wash1.abclr2` explore different implementations of the Time Warp evaluator. They are not additional names for the priority and FIFO policies already compared by `starter`. The latter two replace the *queue* of one schedulable evaluator; NR and NS install entirely new evaluator objects.

Variant	Evaluator's metaobject	Meaning of the change
Standard	Created by `TW-meta-eval-gen`.	Its evaluation-request queue can be selected as Lowest-LVT-first or FIFO.
NR	Explicit `non-reifying-meta`.	Evaluation requests use direct, non-reifying handlers instead of the custom scheduling metaobject.
NS	Default ordinary metaobject.	Removes the custom Time Warp evaluation-request scheduler, but keeps ordinary object dispatch.

All three evaluator definitions retain a local virtual-time cache and the extended `Outputs/LVT` context. All are used with `TW-meta-gen` for simulation objects, so selecting NR does not remove those objects' rollback histories. "No scheduling" in NS does not mean that the runtime stops scheduling processes or ignores simulation event times. It refers to the omitted customized evaluator scheduling layer; ordinary queues and the undoable event queue remain.

The group selectors show the replacement directly:

```
(=> :schedulable-eval
    [evaluator := [TW-eval-gen
                   <== [:new [pqueue-gen <== [:new #'< :key #'first]]
                             (create-primary-eval group)]]])
(=> :non-reifying-eval
    [evaluator := [TW-non-reifying-eval-gen
                   <== [:new (create-primary-eval group)]]])
(=> :no-schedule-eval
    [evaluator := [TW-no-schedule-eval-gen
                   <== [:new (create-primary-eval group)]]])
```

Each creates a fresh evaluator and assigns the group's `evaluator` binding. Unlike `:change-scheduler`, these handlers do not transfer the old evaluator's pending request queue. Nor do they rewrite the `evaluator` binding held by an existing simulation metaobject. The checks create an object before switching and another afterwards: the former retains the old evaluator, the latter captures the new one. This is why the original NR/NS controller switches before `starter-1` creates its attendant and washers. A group's configuration is a creation policy, not necessarily a live broadcast to all its members. Use separate fresh sessions for these experiments; restoring the group's standard evaluator does not retrofit older simulation objects.

Read the controller's result protocol carefully. Here is NR's original controller:

```
[object NR                              ; non-reifying
  (script
   (=> [:start service-time-list num-cars]
       (temporary fifo-rollback fifo-exps)
       (format t "~&===Non-Reifying===")
       ;;[tw-group <= [:change-scheduler [normal-queue-gen <== :new]]]
       [tw-group <= :non-reifying-eval]
       [tw-group <= :reset-monitor]
       ;;[tw-group <= :reset-eval]
       (format t "~&RESULT=~S"
               [starter-1 <== [:start service-time-list num-cars]])
       [fifo-rollback := [tw-group <== :count]]
       ;;[fifo-exps := [tw-group <== :eval-count]]
       (format t "~&NUMBER OF ROLLBACKS=~D" fifo-rollback)
       ;;(format t "~&NUMBER OF EXPRESSIONS=~D" fifo-exps)
       ![:fifo [:rollback fifo-rollback] ]))]
```

It selects the new evaluator, resets the rollback monitor, and waits on the familiar `starter-1` simulation. If that request completes, it prints the result, reads the rollback counter, and replies `[:fifo [:rollback count]]`. NS follows the same structure but sends `:no-schedule-eval`. Both reuse the historical `:fifo` result tag, even though neither installs `normal-queue-gen` here. The tag therefore cannot identify the selected evaluator architecture. The expression-counter reset/query and `:exps` result fields are commented out. The custom counter belongs to `TW-meta-eval`; an ordinary or non-reifying evaluator metaobject does not acquire its `:count` handler merely because the group has that query method.

The original entry points, after loading `tw-load` in a scratch directory, are:

```
; Use a fresh session for each variant.
[NR <== [:start [3 4 6] 3]]
; In another fresh session:
[NS <== [:start [3 4 6] 3]]
```

Both complete this small case. Use the returned controller result to identify completion; printed car assignments alone are not a join.

Follow the evaluator's representations. The translator represents a simple literal such as `:fin` as `[:self-evaluate function]`, where the zero-argument function returns the literal. Each evaluator invokes that function with `funcall`; a function-valued payload is returned as data. The standard evaluator shows the protocol directly:

```
(is [:self-evaluate form]
    [C <= (funcall form)])
```

The `[:script scriptObject]` case delegates to `call-script-body`, which accepts internal scripts, object-level `:exec`, or compiled functions. Trace the representation before deciding which case handles a request.

What the variant checks establish. Run `make -C tutorial timewarp-variants-check`. Fifteen probes per variant check a fresh evaluator identity, old/new objects' captured evaluators, the literal's returned representation and payload, a synthetic five-argument compiled body that replies 42, and `LVT` lookup at 7. The compiled-body probe exercises `:compiled`, whereas the literal probe exercises `:self-evaluate`; one working case does not certify the other. Further checks cover numbers, nil, symbols, lists, function-valued payloads, single invocation of an initializer, and an application script returning 42.

Two car-wash sessions add four checks each: controller result shape, historical `:fifo` tag, and a non-negative rollback count. Transcripts also require the `:fin` result and all three expected car assign-

ments. The 53 checks cover these evaluator and application protocols. Inputs and transcripts stay in `build/timewarp-variants/`. Use small runs to understand the control flow before exploring additional cancellation orders or larger simulations.

Check your understanding. Does NR eliminate application-state rollback? No; simulation objects still use the Time Warp metaobject generator. Does NS mean no queues? No; it omits a particular scheduling layer. Does a returned `:fifo` tag prove FIFO evaluator scheduling? No; the old controllers reuse that tag. Can a new group evaluator fix an already created object's evaluator? Not through these selectors: existing metaobjects retain their captured evaluator.

Check your understanding. If a washer starts at time 2 and has service time 4, does its self-loop advance its virtual time ten times? *Answer:* No. The ordinary self-messages do not supply timestamps; the next timed availability is at 6. Why restore both input and output histories? *Answer:* Inputs may need replay after state restoration, while outputs from discarded work may already have affected other objects.

What the checks establish. Forty checks run in isolated scratch sessions: twelve exercise binding restoration, input replay, pending cancellation, and output history extraction; four check the car-wash report and nonnegative rollback counters; twenty-four check the two-object rollback cascade under both evaluator scheduling policies. The transcript additionally checks the three assignment records under both scheduling policies. Files stay in `build/timewarp-tests/`. The small car-wash case can complete without any rollback, so its success does not demonstrate a complete rollback cascade. Section 13.18.6 supplies a dedicated recovery integration check. Other cancellation arrival orders remain outside these recovery checks; the separate variant probes in Section 13.18.8 check small NR/NS runs, not arbitrary rollback schedules.

13.19 Navigating the Original Sources

Appendix B maps every original example file and library to its reading section and role. Use it to return to a helper after following the corresponding application trace; it is not a new instruction to load every file separately. The directory READMEs are historical instructions, built-ins is a schematic protocol sketch, and `tw-scheduler` is unfinished. Their presence in the source tree does not make them independent executable demonstrations.

14 Final Project

Implement a small concurrent job service with three roles: `dispatcher`, `worker`, and `monitor`.

1. The dispatcher accepts `[:run jobs]`, where each job is `[id value]`, creates a batch monitor, and assigns each job to a worker with `[:square id value]`.
2. Each worker computes a square or rejects an invalid value, sending exactly one `[:done id status result]` to the monitor.
3. The monitor counts successful and failed jobs and sends `[:report successful failed results]` to the original caller once the batch is complete. Handle an empty batch explicitly.
4. Use past-type messages between actors and an explicit reply destination for a completion acknowledgment.
5. Optional advanced extension: add a priority or express-message variant using an appropriate library and compare it with ordinary message handling.

This project brings together object definitions, pattern matching, state, replies, past-type and now-type messages, and concurrency. The worked solution below covers this ordinary-message protocol; the optional extension returns to reflective libraries and is not required to complete it.

14.1 A Worked Job-Service Protocol

The complete solution is `tutorial/examples/03-jobs.abclr2`. It uses two workers to show that completion cannot be inferred from dispatch order. Each input is a two-element list `[id value]`, where `id` is an integer. Each worker sends exactly one `[:done id status result]` reply, including when the value is invalid. This rule lets the monitor count completions without confusing a rejected job with a missing response.

Receiver	Message	Reply destination
Dispatcher	`[:run jobs]`	The original caller.
Monitor factory	`[:new count destination]`	The dispatcher; returns a fresh monitor object.
Worker	`[:square id value]`	The batch's monitor.
Monitor	`[:done id status result]`	After the last completion, sends a report to the original caller.

The workers deliberately use the same behavior; the second definition creates another independent object:

```
[object worker-a
 (script
  (=> [:square id value]
      (if (numberp value)
        ![:done id :ok (* value value)]
        ![:done id :failed :not-a-number]))))]

[object worker-b
 (script
  (=> [:square id value]
      (if (numberp value)
        ![:done id :ok (* value value)]
        ![:done id :failed :not-a-number]))))]
```

The monitor factory captures the batch's final destination. Each created monitor has its own counters and result list:

```
[object monitor-factory
 (script
  (=> [:new count destination]
      ![object
        (state [remaining := count]
               [successful := 0]
               [failed := 0]
               [results := nil])
        (script
         (=> [:done id status value]
             [results := (cons (list id status value) results)]
             (if (eq status :ok)
               [successful := (+ successful 1)]
               [failed := (+ failed 1)])
             [remaining := (- remaining 1)]
             (if (= remaining 0)
               [destination <= [:report successful failed
                                (reverse results)]])))]))]
```

The dispatcher creates the monitor before submitting any work. It routes even job identifiers to one worker and odd identifiers to the other, keeping `@ monitor` on every request:

```
[object dispatcher
 (script
  (=> [:run jobs] @ R
      (if (null jobs)
        [R <= [:report 0 0 nil]]
        (let ((monitor [monitor-factory
                        <== [:new (length jobs) R]]))
          (dolist (job jobs)
            (let ((id (first job))
                  (value (second job)))
              (if (evenp id)
                [worker-a <= [:square id value] @ monitor]
                [worker-b <= [:square id value] @ monitor])))))))]
```

The caller waits on the dispatcher, but the dispatcher handler ends after dispatching. The monitor sends the eventual reply to the caller's `R`. The empty case replies immediately because there will be no worker completions to trigger the monitor.

```
(inc "tutorial/examples/03-jobs" :use-source t)
[dispatcher <== [:run [[1 2] [2 3] [3 :bad]]]]
; => (:REPORT 2 1 ((... completion entries ...)))
[dispatcher <== [:run nil]]
; => (:REPORT 0 0 NIL)
```

The completion entries are `(1 :OK 4)`, `(2 :OK 9)`, and `(3 :FAILED :NOT-A-NUMBER)`. Their order is not part of the interface. The automated check sorts entries by identifier before comparing them. The counts, identifiers, statuses, and computed values are what the protocol promises.

14.2 Invariants and Limits

For every monitor, `remaining + successful + failed` equals the number of submitted jobs. A reply is sent only when `remaining` reaches zero. Because each worker responds exactly once in this example, the final report means the whole batch has completed.

The demonstration assumes well-formed pairs with integer identifiers, no duplicate completion messages, and no worker failures outside the declared invalid-value branch. Production extensions would

need duplicate detection, timeout or cancellation policy, and validation of malformed submissions. A timeout is a policy decision, not proof that a remote computation stopped.

15 Worked Exercise Solutions

Run `07-solutions.abclr2` after attempting the exercises:

```
(inc "tutorial/examples/07-solutions" :use-source t)
```

15.1 Exercise 1: Sum of Squares

```
[object sum-of-squares
 (script
  (=> [:sum values]
      (temporary [total := 0])
      (dolist (x values)
        [total := (+ total (* x x))])
      !total))]
```

```
[sum-of-squares <== [:sum [2 3 4]]]   ; => 29
[sum-of-squares <== [:sum nil]]       ; => 0
```

The temporary accumulator belongs to one invocation. An empty `dolist` leaves it at its initial value, so the result is zero without a special case.

Back to the original. In Section 13.1, `tree2leaves` also keeps handler-local working data, but rewrites the remaining tree instead of adding to an accumulator. Why is a fixed `cdr` step not enough there? *Answer:* a matched clause may expose a nested head and tail as new work. Follow the rewritten tree, not just the original list's next element.

15.2 Exercise 2: An Account Factory and a Transfer

The factory returns an anonymous object for each `:new` request. Its initializer is copied into a separate state binding. Initial balances must be nonnegative integers; this is a precondition, not a check performed by the factory. Deposit and withdrawal amounts are checked by the account handlers.

```
[object account-factory
 (script
  (=> [:new initial-balance]
      ![object
        (state [balance := initial-balance])
        (script
         (=> [:deposit amount]
             (if (and (integerp amount) (> amount 0))
               (progn [balance := (+ balance amount)] !balance)
               !:invalid-amount))
         (=> [:withdraw amount]
             (if (and (integerp amount) (> amount 0)
                      (>= balance amount))
               (progn [balance := (- balance amount)] !balance)
               !:rejected))
         (=> [:transfer other amount]
             (if (and (not (eq other Me))
                      (integerp amount) (> amount 0)
                      (>= balance amount))
               (let ((result [other <== [:deposit amount]]))
                 (if (integerp result)
                   (progn [balance := (- balance amount)] !:transferred)
                   !:rejected))
               !:rejected))
         (=> :balance !balance))]))]
```

```
(inc "tutorial/examples/02-accounts" :use-source t)
[alice <== [:transfer bob 30]]     ; => :TRANSFERRED
[alice <== :balance]               ; => 70
[bob <== :balance]                 ; => 50
[alice <== [:transfer alice 10]]   ; => :REJECTED
```

The self-transfer guard avoids a synchronous wait on the current object. The demonstration uses trusted recipients and a sequential caller. The recipient is credited before the sender is debited, and the two changes are not atomic together. Opposing simultaneous transfers can form a wait cycle. A production transfer protocol would need reservations and a defined commit/cancel procedure; changing one send to past-type alone does not solve that problem.

Back to the original. Why can the inherited feather query in Section 13.4 forward a destination, while this transfer waits for the deposit result? *Answer:* the inherited script can answer the original caller itself; the transfer handler still needs the recipient's result before deciding whether to debit its own state and report success.

15.3 Exercise 3: Count Only After Acknowledgment

The solution creates a fresh consumer for each run. Every item returns its post-increment count to a future, and the producer consumes exactly as many acknowledgments as it submitted items:

```
[object counted-producer
 (script
  (=> [:run values]
      (temporary
       [consumer := [object
                     (state [n := 0])
                     (script
                      (=> [:item x] [n := (+ n 1)] !n)
                      (=> :count !n))]]
       [replies := (make-future)])
      (dolist (x values)
        [consumer <= [:item x] $ replies])
      (dotimes (i (length values))
        (next-value replies))
      ![consumer <== :count]))]
```

```
[counted-producer <== [:run [2 3 4]]]   ; => 3
[counted-producer <== [:run nil]]       ; => 0
```

The final count is queried only after all increments have been acknowledged. The local consumer prevents another batch from affecting the result. Each `next-value` may wait, but no worker waits back on the producer, so this protocol does not introduce a circular dependency.

Back to the original. Why does this client count replies, while `print-leaves` waits for `:finish`? *Answer:* this protocol submits a known number of items and requires exactly one reply per item. The original tree walker sends a stream of leaves whose length the consumer does not know in advance. Pcontrol's two-child join (Section 13.17.2) is another known-count protocol; it must collect two answers, not merely the first.

15.4 Exercise 4: A Tagged Stream

```
[object square-stream
 (script
  (=> [:range first last]
      (do ((i first (+ i 1)))
          ((> i last))
        ![:value (* i i)])
      !:done))]

[object square-stream-client
 (script
  (=> [:collect first last]
      (temporary [stream := (make-future)]
                 [items := nil]
                 [message := nil])
      [square-stream <= [:range first last] $ stream]
      [message := (next-value stream)]
      (do ()
          ((eq message :done))
        (match message
          (is [:value x] [items := (cons x items)]))
        [message := (next-value stream)])
      !(reverse items)))]
```

```
[square-stream-client <== [:collect 1 4]]
; => (1 4 9 16)
```

Values arrive inside `[:value x]`, while `:done` is a distinct terminal message. The client unwraps only the value messages. This separation remains useful even if the data domain later includes keywords or lists.

Back to the original. Would renaming `:finish` to another keyword in the original tree walker solve its marker collision? *Answer:* no; that keyword could also occur as a leaf. The value wrapper distinguishes the *role* of a message instead of reserving one possible data value.

15.5 Exercise 5: Unless at the Meta-Level

Add this handler before the choice evaluator's fallback:

```
(=> [:unless [predicate body] Env Id Gid Eval] @ C
    [Eval <= [:do predicate Env Id Gid Eval]
          @ [cont result
             (if result
               [C <= nil]
               [Eval <= [:do body Env Id Gid Eval] @ C])]])
```

When the predicate is true, reply to `C` with `nil`. Otherwise evaluate the compiled body in its captured environment. The test object's `:unless` handler invokes this protocol:

```
[choice-test <== [:unless 3]]   ; => 3
[choice-test <== :selected]     ; => :UNLESS
[choice-test <== [:unless 8]]   ; => NIL
[choice-test <== :selected]     ; => :UNLESS (unchanged)
```

The unchanged state verifies that the body was skipped for an even input. Both control-flow paths still send a result to the caller's continuation.

Back to the original. Compare `my-if` (Section 13.2) and `with-interrupt` (Section 13.3). Where does each reified request go? *Answer:* `my-if`, like our choice form, uses the evaluator; `with-interrupt` uses the application's metaobject. For either route, trace the saved continuation separately from the application handler's eventual `!` reply.

16 Further Reading in the Source Tree

This book is the primary user documentation for the current SBCL port. The historical documents below provide supplementary detail and background; their platform-specific installation instructions describe the original release, not the current build.

- `historical/doc/users-guide-e`: reference for ABCL/R2-specific forms and implementation restrictions.
- `doc/Masuhara_1992.pdf`: Hidehiko Masuhara.
  Study on Implementation of an Object-Oriented Concurrent Reflective Language, University of Tokyo, February 1992. Covers implementation, reflective towers, and lazy creation.
- `examples/misc/`: small programs demonstrating futures, express messages, built-in objects, and reflection.
- `lib/`: library implementations to inspect and load.

A Building the Book and Checking Its Examples

From the repository root, build the PDF and check the examples:

```
make -C tutorial
make -C tutorial check
```

The PDF build runs `lualatex` three times to stabilize the contents, page numbers, and citations. It keeps build intermediates in `build/tutorial-build/` and copies the finished document to `doc/tutorial.pdf`. The build and checks create their scratch directories automatically; no pre-existing contents of `build/` are required. It requires SBCL for source-excerpt preparation and a LuaLaTeX installation with the packages named in the preamble, Libertinus Serif and Libertinus Sans for text, and Noto Sans Mono for code. The native runner `tutorial/tests/run-tests.lisp` needs SBCL and the current ABCL/R2 executable, built with `make` at the root; no Python or external Lisp packages are needed. The checks run example sessions in fresh processes and keep source copies, translations and transcripts under `build/`. Session inputs are in `tutorial/tests/sessions/`; expected results are specified in `tutorial/tests/cases.lisp`. Run `make -C tutorial reading-check` for the Lisp-reading examples, `reflection-check` for reflective protocols, and `group-check` for ownership and identity checks.

Interactive loading creates a neighboring `.lsp` file; use `:use-source t` while editing to avoid cached translations. Printed object identifiers are session-dependent, not stable expected outputs.

B Original Source Reading Map

This map covers all 34 original example files and nine library files. Each entry points to the explanation of its role in a complete example, not necessarily to a separate runnable program. Start with the linked application or loading section before reading its supporting generators. Consult the walkthrough's stated checks and limits: a source-map entry is not a certification that every execution path works on this image.

Individual examples and reflective mechanisms.

Original file	Reading route and role
`examples/misc/future-ex.abclr2`	Section 13.1: Tree traversal, repeated replies, and the terminal marker.
`examples/misc/inherit.abclr2`	Section 13.4: Bird/emu application, inherited scripts, and reflective context.
`examples/misc/built-ins.abclr2`	Section 13.5: Schematic environment, script, and queue protocols; not a runtime to load.
`examples/misc/user-script.abclr2`	Section 13.6: Sender-sensitive script matching, dynamic installation, and separate application/evaluation replies.
`examples/misc/MCP.abclr2`	Section 13.7: Meta-circular group manager, application metaobjects, and evaluator cases.
`examples/misc/meta-meta.abclr2`	Section 13.8: Three levels of identities and the twice computation.
`examples/misc/reify-args.abclr2`	Section 13.2: Eager/lazy branch selection; Section 13.3 follows interruption and the separate control-flow experiment.
`examples/misc/fault-tolerant.abclr2`	Section 13.14: Acceptance state, initialization, rejection, and retry with preserved reply/sender.
`examples/misc/break.abclr2`	Section 13.15: Breakpoint continuation, lexical environment navigation, and resume.

Original file	Reading route and role
`examples/misc/express-ex.abclr2`	Section 13.16: Newer clock protocol, suspension context, atomic regions, and express-message routing.
`examples/misc/express-old-ex.abclr2`	Section 13.16: Legacy clock and reader/writer lock participation; separate fresh session.

Priority control: read the application, then the execution policy.

Original file	Reading route and role
`examples/pcontrol/README.abclr2`	Section 13.17.1: Historical entry instructions, argument shapes, and small inputs.
`examples/pcontrol/start.abclr2`	Section 13.17.1: Compile applications, then load the four machinery files in dependency order.
`examples/pcontrol/fib.abclr2`	Section 13.17.2: Child creation, saved reply destination, and two-answer join.
`examples/pcontrol/qsort.abclr2`	Section 13.17.4: Partition self-messages, shared array, and completion join.
`examples/pcontrol/pcontrol-group.abclr2`	Section 13.17.3: Connect priority metaobjects and four evaluator objects into one group.
`examples/pcontrol/priori-meta.abclr2`	Section 13.17.3: Attach object priority to evaluation and announce dispatch activity.
`examples/pcontrol/pqueue-eval.abclr2`	Section 13.17.3: Priority/FIFO work queues, priority associations, and cyclic evaluator routing.
`examples/pcontrol/para-monitor.abclr2`	Section 13.17.5: Logical clock, active/object counters, and output streams.

Time Warp: separate application, histories, and evaluator scheduling.

Original file	Reading route and role
`examples/time-warp/README.abclr2`	Section 13.18.2: Historical run sequence and virtual-time application interface.
`examples/time-warp/tw-load.abclr2`	Section 13.18.2: Ten-file compile/load sequence; distinguishes loaded machinery from alternatives.
`examples/time-warp/tw-wash1.abclr2`	Section 13.18.3: Timed car/washer pairing, availability, finish sentinel, and controller join.
`examples/time-warp/tw-wash0.abclr2`	Section 13.18.2: Earlier ordinary-message application, without Time Warp or timed service completion.
`examples/time-warp/tw-meta.abclr2`	Section 13.18.4: Ordinary/timed dispatch, stragglers, state restoration, and anti-messages.
`examples/time-warp/tw-undo-env.abclr2`	Section 13.18.5: Saved local binding frames and strict-before-cutoff restoration.
`examples/time-warp/tw-undoable-pqueue.abclr2`	Section 13.18.5: Pending/dequeued inputs, replay, cancellation, and active versus conditional alternatives.
`examples/time-warp/tw-output-history.abclr2`	Section 13.18.5: Record sent events and extract speculative output by send-time cutoff.
`examples/time-warp/tw-group.abclr2`	Section 13.18.7: Connect components, transfer scheduler queues, and select evaluator variants.

Original file	Reading route and role
`examples/time-warp/tw-eval.abclr2`	Section 13.18.7: Expression interpretation and time/output context; Section 13.18.8 compares NR/NS.
`examples/time-warp/tw-meta-eval.abclr2`	Section 13.18.7: Queue evaluator requests by LVT and count evaluation arrivals.
`examples/time-warp/tw-pqueue.abclr2`	Section 13.18.7: Priority and FIFO work queues; not the undoable application input queue.
`examples/time-warp/tw-monitor.abclr2`	Section 13.18.7: Rollback announcements and their counter.
`examples/time-warp/tw-test.abclr2`	Section 13.18.7: Independent countdown objects; the starter is not a completion join.
`examples/time-warp/tw-scheduler.abclr2`	Section 13.18.7: Unfinished fragment, not loaded by the application loader.

Libraries: loading is not the same as installing a policy.

Original file	Reading route and role
`lib/future.abclr2`	Section 13.1: Buffered reply destination, consumption, and explicit application completion; see also Section 10.1.
`lib/multicast.abclr2`	Section 13.10: Evaluator installation, target templates, forwarding, and reply collection.
`lib/heavy-meta.abclr2`	Section 13.9: Guard-aware script selection and candidate fallback.
`lib/heavy-meta-old.abclr2`	Section 13.9: Historical reference only: use the regular variant with the current translator.
`lib/express.abclr2`	Section 13.16: Newer sender/receiver machinery and execution modes.
`lib/express-old.abclr2`	Section 13.16: Different legacy lock/count protocol; not an alias for the newer library.
`lib/copy.abclr2`	Section 13.11: New identities versus shared lexical/script/reply references and state-copy restrictions.
`lib/watch.abclr2`	Section 13.12: Outgoing-send evaluator wrapper, reporters, and restoration boundaries.
`lib/report.abclr2`	Section 13.13: Copy-based arrival/dispatch/completion reports and filtering.

C Syntax at a Glance

This is a reading aid, not a complete grammar. Names such as `target`, `pattern`, and `body` are placeholders; ellipses stand for omitted parts. The linked sections explain evaluation, matching, prerequisites, and the limits of the supplied implementation. Lisp notation and control forms are introduced in Section 2.1.

Objects, state, and matching.

Construct	Meaning	Section
`[object name (script ...)]`	Create an object and bind its name; omit the name for an anonymous object.	3

Construct	Meaning	Section
`(state [n := 0])`	Object-definition entry for persistent state.	4
`(temporary [x := 0])`	Handler-local binding, not persistent state.	3
`[x := expression]`	Change a binding.	4
`(=> pattern body)`	A script handling a matching message.	3
`(=> pattern @ R from S body)`	Additionally bind reply destination and sender; either header field can be omitted.	3
`[head . tail]`	Pattern binding the head and entire tail of a pair; dot notation is not an extra list element.	2.1
`(match value (is pattern body) ...)`	Match a value inside a script; `otherwise` supplies a fallback clause.	3
`(=> pattern where test body)`	Guarded script selection, requiring compatible heavy-meta machinery; not an ordinary `match` constraint.	13.9

Sending, replying, and continuing.

Construct	Meaning	Section
`[target <= message]`	Past-type send; do not wait for a reply.	5
`[target <== message]`	Now-type send; wait for a reply.	5
`[target <= message @ R]`	Past-type send with a specified reply destination.	5
`[target <= message $ f]`	Future-type send; route replies through the loaded future library.	10.1
`!expression`	Send a reply for the current message; not a return from the rest of the handler.	3
`[cont pattern body]`	Create a short, non-reifying continuation object.	5
`[target <= message :express]`	Express send; requires both sender and receiver machinery, not just this extra parameter.	10
`(atomic body ...)`	Receiver-local protection in the express protocol; not a multi-object transaction.	10

Groups, identities, and reflection.

Construct	Meaning	Section
`[group name ...]`	Create a group manager with optional custom kernel objects.	11
`(group manager)`	Object-definition parameter selecting its creation group.	11
`(meta-gen generator)`	Object/group definition parameter selecting a metaobject generator.	11
`(evaluator evaluator-object)`	Group-definition parameter selecting its evaluator.	11
`[meta object]`	Obtain its metaobject.	11
`[den metaobject]`	Obtain its lower-level denotation, not a copy.	11
`[group-of object]`	Query its group manager; custom generators must provide the corresponding identity metadata.	13.4
`Me` / `group`	Current object denoted by `Id` / group manager in `Gid`, not necessarily base-level application objects.	11
`(reify-args tag forms ...)`	Evaluate the tag, then send compiled argument expressions and context to the current evaluator.	11
`(reify-args-to-meta tag forms ...)`	Send them to the current object's metaobject instead.	11

`reify-args` and `reify-args-to-meta` are unavailable in a non-reifying object's script. Extra send/object/assignment parameters are interpreted by the selected machinery, not universally by their spelling. Section 11 explains the evaluation context; Section 9 separately collects loading and compilation commands.

D Troubleshooting Program Protocols

No return messages. A now-type call has no reply. Check that a script matched and every result path either uses `!` or forwards the original reply destination. Also look earlier in the transcript for a Lisp error.

An object is not bound. Its definition may have failed while loading the file. Read the first error rather than treating later missing bindings as independent failures.

A future never finishes. The producer may not send the terminal marker, or a reader may have consumed it already. Futures do not infer completion.

A macro expands into an unsupported special form. ABCL/R2 does not support every Common Lisp special form. Inspect the macro expansion and compare it with the supported forms following Section 2.1.

Script lambdas. Lambdas embedded in scripts cannot capture ABCL lexical variables, and their bodies cannot contain ABCL object creation or message sends in this implementation [1]. Prefer a script object for an ABCL callback.

E Concept and Protocol Index

Use this alphabetical index to return to an explanation, not as a list of every occurrence. Entries give the section and its starting page. For notation use Appendix C; for original filenames use Appendix B.

Concept or protocol	Explanation
Actors and asynchronous messages	5 (p. 12)
Assignment	4 (p. 11)
Atomic regions and their limits	10 (p. 18); 13.16 (p. 67)
Bindings, lexical scope, and Lisp notation	2.1 (p. 5)
Breakpoints and environment inspection	13.15 (p. 64)
Built-in object protocols	13.5 (p. 38)
Callbacks and script lambdas	3 (p. 9); D (p. 95)
Compilation and cached translations	9 (p. 15)
Completion: application versus evaluation	13 (p. 27)
Continuations, `cont`, and explicit reply targets	5 (p. 12)
Copy: identity, state, and shared references	13.11 (p. 49)
Denotation, `den`, and `meta`	11 (p. 20)
Dispatch and script selection	3 (p. 9); 13.9 (p. 42)
Environment frames and inspection	11 (p. 20); 13.15 (p. 64)
Evaluation context: Env, Id, Gid, Eval, C	11 (p. 20)
Express messages and preemption	10 (p. 18); 13.16 (p. 67)
Fault tolerance: rejection and retry	13.14 (p. 61)
Fibonacci: recursive objects and reply joins	13.17.2 (p. 71)
Futures, `make-future`, and `next-value`	10.1 (p. 18); 13.1 (p. 28)
Groups, `group-of`, and object creation	11 (p. 20); 13.4 (p. 35)
Guards, `where`, and heavy-meta	13.9 (p. 42)
Host events and the pending-request slot	11.6 (p. 24)
Identity and metaobject towers	13.8 (p. 41)
Inheritance and forwarding reply destinations	13.4 (p. 35)
Interrupting evaluation	13.3 (p. 31)
Job-service protocol: run, square, done, report	14 (p. 84)
Loading: `inc`, `comp`, and `inc-lib`	9 (p. 15)
Library search path: `*library-directories*`	9.2 (p. 17)
Macros and supported Lisp forms	2 (p. 4); D (p. 95)
Matching, dotted patterns, and fallback	3 (p. 9)
MCP and the meta-circular interpreter	13.7 (p. 40)
Me and the currently executing object	11 (p. 20)
Messages: past-, now-, and future-type	5 (p. 12); 10.1 (p. 18)

Concept or protocol	Explanation
Modules: `require` and `provide`	9 (p. 15)
Monitor measurements and their limits	13.17.5 (p. 74)
Multicast and positional reply collection	13.10 (p. 46)
Non-reifying execution	11 (p. 20); 13.18.8 (p. 81)
Objects, factories, and scripts	3 (p. 9); 6 (p. 13)
Priority queues and scheduling policy	13.17.3 (p. 72)
Quicksort as a message-driven state machine	13.17.4 (p. 73)
Reflection: evaluator and metaobject routes	11 (p. 20); 13.2 (p. 30)
Reification: `reify-args`, `reify-args-to-meta`	11 (p. 20); 13.3 (p. 31)
Replies, `!`, sender, and destination	3 (p. 9); 13 (p. 27)
Report: receiving-side observation and restoration	13.13 (p. 57)
Rollback, anti-messages, and replay	13.18.4 (p. 77); 13.18.6 (p. 78)
Script installation and dynamic updates	13.6 (p. 39)
State versus temporary bindings	4 (p. 11); 3 (p. 9)
Streams and terminal-message protocols	13.1 (p. 28); 15.4 (p. 89)
Time Warp: virtual time and simulation	13.18 (p. 75)
Transactions: limits of an account transfer	15.2 (p. 87)
Watch: outgoing-send observation and restoration	13.12 (p. 53)

References

[1] Hidehiko Masuhara. *ABCL/R2 User's Guide*. English translation, November 1992, with updates through July 1994. Included as `historical/doc/users-guide-e`.

[2] Hidehiko Masuhara. *Study on Implementation of an Object-Oriented Concurrent Reflective Language*. Senior thesis, University of Tokyo, February 6, 1992. Included as `doc/Masuhara_1992.pdf`.

[3] *ABCL/R2 Distribution Readme and Installation Guide*. Included as `historical/doc/readme` and `historical/doc/install`, with the current root Makefile providing the SBCL build.